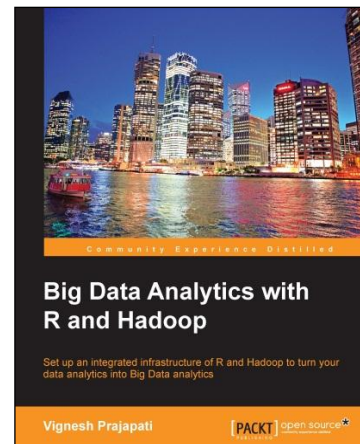


Big Data Analytics with R and Hadoop

Vignesh Prajapati



Chapter No.1 "Getting Ready to Use R and Hadoop"

In this package, you will find:

A Biography of the author of the book

A preview chapter from the book, Chapter NO.1 "Getting Ready to Use R and Hadoop"

A synopsis of the book's content

Information on where to buy this book

About the Author

Vignesh Prajapati, from India, is a Big Data enthusiast, a Pingax (www.pingax.com) consultant and a software professional at Enjay. He is an experienced ML Data engineer. He is experienced with Machine learning and Big Data technologies such as R, Hadoop, Mahout, Pig, Hive, and related Hadoop components to analyse datasets to achieve informative insights by data analytics cycles.

He pursued B.E from Gujarat Technological University in 2012 and started his career as Data Engineer at Tatvic. His professional experience includes working on the development of various Data analytics algorithms for Google Analytics data source, for providing economic value to the products. To get the ML in action, he implemented several analytical apps in collaboration with Google Analytics and Google Prediction API services. He also contributes to the R community by developing the RGoogleAnalytics' R library as an open source code Google project and writes articles on Data-driven technologies.

Vignesh is not limited to a single domain; he has also worked for developing various interactive apps via various Google APIs, such as Google Analytics API, Realtime API, Google Prediction API, Google Chart API, and Translate API with the Java and PHP platforms. He is highly interested in the development of open source technologies.

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

Vignesh has also reviewed the Apache Mahout Cookbook for Packt Publishing. This book provides a fresh, scope-oriented approach to the Mahout world for beginners as well as advanced users. Mahout Cookbook is specially designed to make users aware of the different possible machine learning applications, strategies, and algorithms to produce an intelligent as well as Big Data application.

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

Big Data Analytics with R and Hadoop

The volume of data that enterprises acquire every day is increasing exponentially. It is now possible to store these vast amounts of information on low cost platforms such as Hadoop.

The conundrum these organizations now face is what to do with all this data and how to glean key insights from this data. Thus R comes into picture. R is a very amazing tool that makes it a snap to run advanced statistical models on data, translate the derived models into colorful graphs and visualizations, and do a lot more functions related to data science.

One key drawback of R, though, is that it is not very scalable. The core R engine can process and work on very limited amount of data. As Hadoop is very popular for Big Data processing, corresponding R with Hadoop for scalability is the next logical step.

This book is dedicated to R and Hadoop and the intricacies of how data analytics operations of R can be made scalable by using a platform as Hadoop.

With this agenda in mind, this book will cater to a wide audience including data scientists, statisticians, data architects, and engineers who are looking for solutions to process and analyze vast amounts of information using R and Hadoop.

Using R with Hadoop will provide an elastic data analytics platform that will scale depending on the size of the dataset to be analyzed. Experienced programmers can then write Map/Reduce modules in R and run it using Hadoop's parallel processing Map/Reduce mechanism to identify patterns in the dataset.

Introducing R

R is an open source software package to perform statistical analysis on data. R is a programming language used by data scientist statisticians and others who need to make statistical analysis of data and glean key insights from data using mechanisms, such as regression, clustering, classification, and text analysis. R is registered under **GNU (General Public License)**. It was developed by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand, which is currently handled by the R Development Core Team. It can be considered as a different implementation of S, developed by Johan Chambers at Bell Labs. There are some important differences, but a lot of the code written in S can be unaltered using the R interpreter engine.

R provides a wide variety of statistical, machine learning (linear and nonlinear modeling, classic statistical tests, time-series analysis, classification, clustering) and graphical techniques, and is highly extensible. R has various built-in as well as extended functions for statistical, machine learning, and visualization tasks such as:

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

- Data extraction
- Data cleaning
- Data loading
- Data transformation
- Statistical analysis
- Predictive modeling
- Data visualization

It is one of the most popular open source statistical analysis packages available on the market today. It is crossplatform, has a very wide community support, and a large and ever-growing user community who are adding new packages every day. With its growing list of packages, R can now connect with other data stores, such as MySQL, SQLite, MongoDB, and Hadoop for data storage activities.

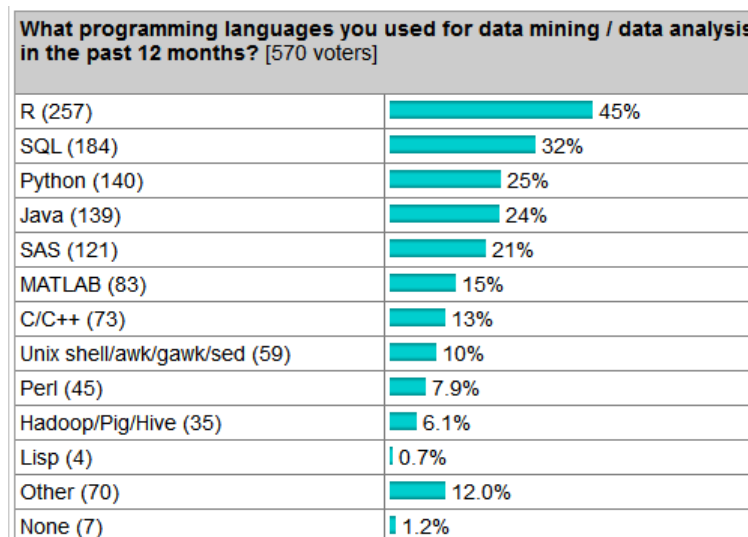
Understanding features of R

Let's see different useful features of R:

- Effective programming language
- Relational database support
- Data analytics
- Data visualization
- Extension through the vast library of R packages

Studying the popularity of R

The graph provided from KD suggests that R is the most popular language for data analysis and mining:

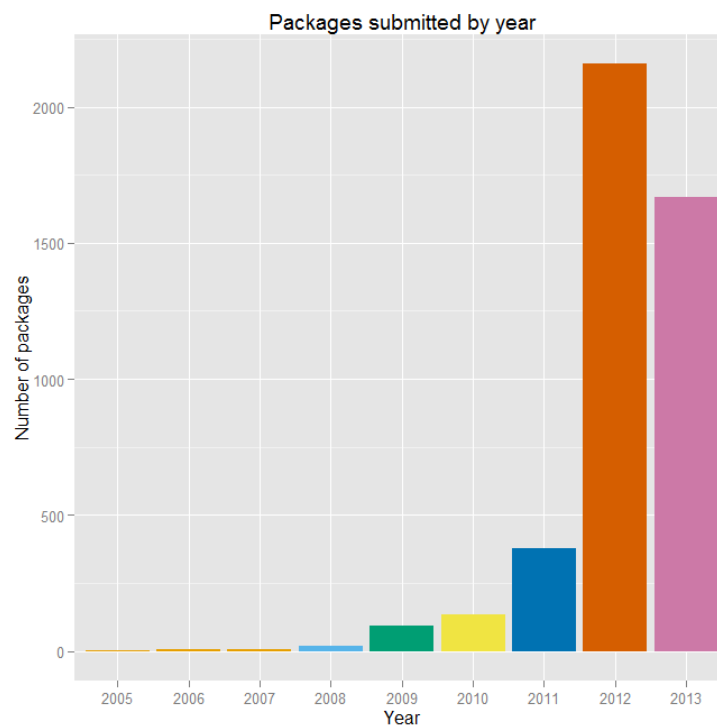


For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

The following graph provides details about the total number of R packages released by R users from 2005 to 2013. This is how we explore R users. The growth was exponential in 2012 and it seems that 2013 is on track to beat that. R allows performing Data analytics by various statistical and machine learning operations as follows:

- Regression
- Classification
- Clustering
- Recommendation
- Text mining



Introducing Big Data

Big Data has to deal with large and complex datasets that can be structured, semi-structured, or unstructured and will typically not fit into memory to be processed. They have to be processed in place, which means that computation has to be done where the data resides for processing. When we talk to developers, the people actually building Big Data systems and applications, we get a better idea of what they mean about 3Vs. They typically would mention the 3Vs model of Big Data, which are velocity, volume, and variety.

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

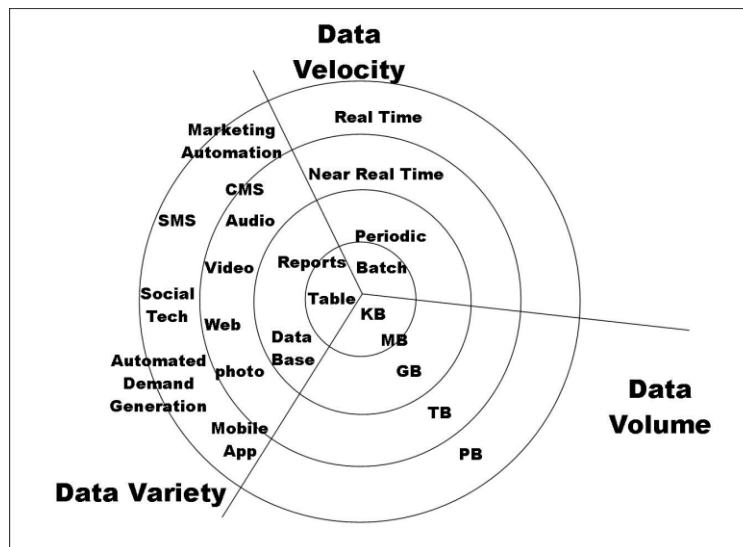
Velocity refers to the low latency, real-time speed at which the analytics need to be applied. A typical example of this would be to perform analytics on a continuous stream of data originating from a social networking site or aggregation of disparate sources of data.

Volume refers to the size of the dataset. It may be in KB, MB, GB, TB, or PB based on the type of the application that generates or receives the data.

Variety refers to the various types of the data that can exist, for example, text, audio, video, and photos.

Big Data usually includes datasets with sizes. It is not possible for such systems to process this amount of data within the time frame mandated by the business. Big

Data volumes are a constantly moving target, as of 2012 ranging from a few dozen terabytes to many petabytes of data in a single dataset. Faced with this seemingly insurmountable challenge, entirely new platforms are called Big Data platforms.



Getting information about popular organizations that hold Big Data

Some of the popular organizations that hold Big Data are as follows:

- Facebook: It has 40 PB of data and captures 100 TB/day
- Yahoo!: It has 60 PB of data
- Twitter: It captures 8 TB/day
- EBay: It has 40 PB of data and captures 50 TB/day

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

How much data is considered as Big Data differs from company to company. Though true that one company's Big Data is another's small, there is something common: doesn't fit in memory, nor disk, has rapid influx of data that needs to be processed and would benefit from distributed software stacks. For some companies, 10 TB of data would be considered Big Data and for others 1 PB would be Big Data.

So only you can determine whether the data is really Big Data. It is sufficient to say that it would start in the low terabyte range.

Also, a question well worth asking is, as you are not capturing and retaining enough of your data do you think you do not have a Big Data problem now? In some scenarios, companies literally discard data, because there wasn't a cost effective way to store and process it. With platforms as Hadoop, it is possible to start capturing and storing all that data.

Introducing Hadoop

Apache Hadoop is an open source Java framework for processing and querying vast amounts of data on large clusters of commodity hardware. Hadoop is a top level

Apache project, initiated and led by Yahoo! and Doug Cutting. It relies on an active community of contributors from all over the world for its success.

With a significant technology investment by Yahoo!, Apache Hadoop has become an enterprise-ready cloud computing technology. It is becoming the industry de facto framework for Big Data processing.

Hadoop changes the economics and the dynamics of large-scale computing. Its impact can be boiled down to four salient characteristics. Hadoop enables scalable, cost-effective, flexible, fault-tolerant solutions.

Exploring Hadoop features

Apache Hadoop has two main features:

- HDFS (Hadoop Distributed File System)
- MapReduce

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

Studying Hadoop components

Hadoop includes an ecosystem of other products built over the core HDFS and MapReduce layer to enable various types of operations on the platform. A few popular Hadoop components are as follows:

- **Mahout:** This is an extensive library of machine learning algorithms.
- **Pig:** Pig is a high-level language (such as PERL) to analyze large datasets with its own language syntax for expressing data analysis programs, coupled with infrastructure for evaluating these programs.
- **Hive:** Hive is a data warehouse system for Hadoop that facilitates easy data summarization, ad hoc queries, and the analysis of large datasets stored in HDFS. It has its own SQL-like query language called **Hive Query Language (HQL)**, which is used to issue query commands to Hadoop.
- **HBase:** HBase (Hadoop Database) is a distributed, column-oriented database. HBase uses HDFS for the underlying storage. It supports both batch style computations using MapReduce and atomic queries (random reads).
- **Sqoop:** Apache Sqoop is a tool designed for efficiently transferring bulk data between Hadoop and Structured Relational Databases. **Sqoop** is an abbreviation for **(SQL)** to **Ha(oop)**.
- **ZooKeeper:** ZooKeeper is a centralized service to maintain configuration information, naming, providing distributed synchronization, and group services, which are very useful for a variety of distributed systems.
- **Ambari:** A web-based tool for provisioning, managing, and monitoring Apache Hadoop clusters, which includes support for Hadoop HDFS, Hadoop MapReduce, Hive, HCatalog, HBase, ZooKeeper, Oozie, Pig, and Sqoop.

Understanding the reason for using R and Hadoop together

I would also say that sometimes the data resides on the HDFS (in various formats). Since a lot of data analysts are very productive in R, it is natural to use R to compute with the data stored through Hadoop-related tools.

As mentioned earlier, the strengths of R lie in its ability to analyze data using a rich library of packages but fall short when it comes to working on very large datasets. The strength of Hadoop on the other hand is to store and process very large amounts of data in the TB and even PB range. Such vast datasets cannot be processed in memory as the RAM of each machine cannot hold such large datasets. The options would be to run analysis on limited chunks also known as sampling or to correspond the analytical power of R with the storage and processing power of Hadoop and you arrive at an ideal solution. Such solutions can also be achieved in the cloud using platforms such as Amazon EMR.

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

What This Book Covers

Chapter 1, Getting Ready to Use R and Hadoop, gives an introduction as well as the process of installing R and Hadoop.

Chapter 2, Writing Hadoop MapReduce Programs, covers basics of Hadoop MapReduce and ways to execute MapReduce using Hadoop.

Chapter 3, Integrating R and Hadoop, shows deployment and running of sample MapReduce programs for RHadoop and RHIPE by various data handling processes.

Chapter 4, Using Hadoop Streaming with R, shows how to use Hadoop Streaming with R.

Chapter 5, Learning Data Analytics with R and Hadoop, introduces the Data analytics project life cycle by demonstrating with real-world Data analytics problems.

Chapter 6, Understanding Big Data Analysis with Machine Learning, covers performing Big Data analytics by machine learning techniques with RHadoop.

Chapter 7, Importing and Exporting Data from Various DBs, covers how to interface with popular relational databases to import and export data operations with R.

Appendix, References, describes links to additional resources regarding the content of all the chapters being present.

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

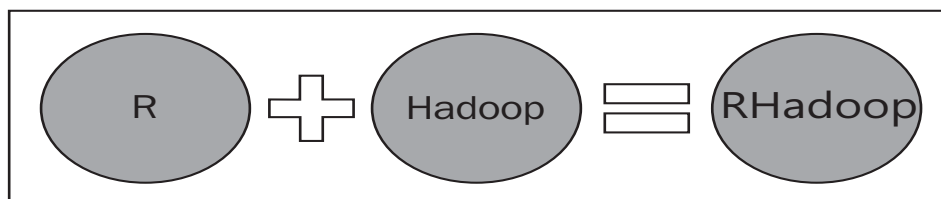
1

Getting Ready to Use R and Hadoop

The first chapter has been bundled with several topics on R and Hadoop basics as follows:

- R Installation, features, and data modeling
- Hadoop installation, features, and components

In the preface, we introduced you to R and Hadoop. This chapter will focus on getting you up and running with these two technologies. Until now, R has been used mainly for statistical analysis, but due to the increasing number of functions and packages, it has become popular in several fields, such as machine learning, visualization, and data operations. R will not load all data (Big Data) into machine memory. So, Hadoop can be chosen to load the data as Big Data. Not all algorithms work across Hadoop, and the algorithms are, in general, not R algorithms. Despite this, analytics with R have several issues related to large data. In order to analyze the dataset, R loads it into the memory, and if the dataset is large, it will fail with exceptions such as "cannot allocate vector of size x". Hence, in order to process large datasets, the processing power of R can be vastly magnified by combining it with the power of a Hadoop cluster. Hadoop is very a popular framework that provides such parallel processing capabilities. So, we can use R algorithms or analysis processing over Hadoop clusters to get the work done.



For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

If we think about a combined RHadoop system, R will take care of data analysis operations with the preliminary functions, such as data loading, exploration, analysis, and visualization, and Hadoop will take care of parallel data storage as well as computation power against distributed data.

Prior to the advent of affordable Big Data technologies, analysis used to be run on limited datasets on a single machine. Advanced machine learning algorithms are very effective when applied to large datasets, and this is possible only with large clusters where data can be stored and processed with distributed data storage systems. In the next section, we will see how R and Hadoop can be installed on different operating systems and the possible ways to link R and Hadoop.

Installing R

You can download the appropriate version by visiting the official R website.

Here are the steps provided for three different operating systems. We have considered Windows, Linux, and Mac OS for R installation. Download the latest version of R as it will have all the latest patches and resolutions to the past bugs.

For Windows, follow the given steps:

1. Navigate to `www.r-project.org`.
2. Click on the **CRAN** section, select **CRAN mirror**, and select your Windows OS (stick to Linux; Hadoop is almost always used in a Linux environment).
3. Download the latest R version from the mirror.
4. Execute the downloaded `.exe` to install R.

For Linux-Ubuntu, follow the given steps:

1. Navigate to `www.r-project.org`.
2. Click on the **CRAN** section, select **CRAN mirror**, and select your OS.
3. In the `/etc/apt/sources.list` file, add the CRAN `<mirror>` entry.
4. Download and update the package lists from the repositories using the `sudo apt-get update` command.
5. Install R system using the `sudo apt-get install r-base` command.

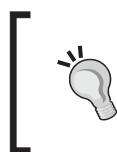
For Linux-RHEL/CentOS, follow the given steps:

1. Navigate to `www.r-project.org`.
2. Click on **CRAN**, select **CRAN mirror**, and select Red Hat OS.
3. Download the `R-*core-*.rpm` file.
4. Install the `.rpm` package using the `rpm -ivh R-*core-*.rpm` command.
5. Install R system using `sudo yum install R`.

For Mac, follow the given steps:

1. Navigate to `www.r-project.org`.
2. Click on **CRAN**, select **CRAN mirror**, and select your OS.
3. Download the following files: `pkg`, `gfortran-*.dmg`, and `tcltk-*.dmg`.
4. Install the `R-*.pkg` file.
5. Then, install the `gfortran-*.dmg` and `tcltk-*.dmg` files.

After installing the base R package, it is advisable to install RStudio, which is a powerful and intuitive **Integrated Development Environment (IDE)** for R.



We can use R distribution of Revolution Analytics as a Modern Data analytics tool for statistical computing and predictive analytics, which is available in free as well as premium versions. Hadoop integration is also available to perform Big Data analytics.

Installing RStudio

To install RStudio, perform the following steps:

1. Navigate to `http://www.rstudio.com/ide/download/desktop`.
2. Download the latest version of RStudio for your operating system.
3. Execute the installer file and install RStudio.

The RStudio organization and user community has developed a lot of R packages for graphics and visualization, such as `ggplot2`, `plyr`, `Shiny`, `Rpubs`, and `devtools`.

Understanding the features of R language

There are over 3,000 R packages and the list is growing day by day. It would be beyond the scope of any book to even attempt to explain all these packages. This book focuses only on the key features of R and the most frequently used and popular packages.

Using R packages

R packages are self-contained units of R functionality that can be invoked as functions. A good analogy would be a .jar file in Java. There is a vast library of R packages available for a very wide range of operations ranging from statistical operations and machine learning to rich graphic visualization and plotting. Every package will consist of one or more R functions. An R package is a re-usable entity that can be shared and used by others. R users can install the package that contains the functionality they are looking for and start calling the functions in the package. A comprehensive list of these packages can be found at <http://cran.r-project.org/> called **Comprehensive R Archive Network (CRAN)**.

Performing data operations

R enables a wide range of operations. Statistical operations, such as mean, min, max, probability, distribution, and regression. Machine learning operations, such as linear regression, logistic regression, classification, and clustering. Universal data processing operations are as follows:

- **Data cleaning:** This option is to clean massive datasets
- **Data exploration:** This option is to explore all the possible values of datasets
- **Data analysis:** This option is to perform analytics on data with descriptive and predictive analytics data visualization, that is, visualization of analysis output programming

To build an effective analytics application, sometimes we need to use the online **Application Programming Interface (API)** to dig up the data, analyze it with expedient services, and visualize it by third-party services. Also, to automate the data analysis process, programming will be the most useful feature to deal with.

R has its own programming language to operate data. Also, the available package can help to integrate R with other programming features. R supports object-oriented programming concepts. It is also capable of integrating with other programming languages, such as Java, PHP, C, and C++. There are several packages that will act as middle-layer programming features to aid in data analytics, which are similar to `sqldf`, `httr`, `RMongo`, `RgoogleMaps`, `RGoogleAnalytics`, and `google-prediction-api-r-client`.

Increasing community support

As the number of R users are escalating, the groups related to R are also increasing. So, R learners or developers can easily connect and get their uncertainty solved with the help of several R groups or communities.

The following are many popular sources that can be found useful:

- **R mailing list:** This is an official R group created by R project owners.
- **R blogs:** R has countless bloggers who are writing on several R applications. One of the most popular blog websites is <http://www.r-bloggers.com/> where all the bloggers contribute their blogs.
- **Stack overflow:** This is a great technical knowledge sharing platform where the programmers can post their technical queries and enthusiast programmers suggest a solution. For more information, visit <http://stats.stackexchange.com/>.
- **Groups:** There are many other groups existing on LinkedIn and Meetup where professionals across the world meet to discuss their problems and innovative ideas.
- **Books:** There are also lot of books about R. Some of the popular books are *R in Action*, by Rob Kabacoff, Manning Publications, *R in a Nutshell*, by Joseph Adler, O'Reilly Media, *R and Data Mining*, by Yanchang Zhao, Academic Press, and *R Graphs Cookbook*, by Hrishu Mittal, Packt Publishing.

Performing data modeling in R

Data modeling is a machine learning technique to identify the hidden pattern from the historical dataset, and this pattern will help in future value prediction over the same data. This technique highly focuses on past user actions and learns their taste. Most of these data modeling techniques have been adopted by many popular organizations to understand the behavior of their customers based on their past transactions. These techniques will analyze data and predict for the customers what they are looking for. Amazon, Google, Facebook, eBay, LinkedIn, Twitter, and many other organizations are using data mining for changing the definition applications.

The most common data mining techniques are as follows:

- **Regression:** In statistics, regression is a classic technique to identify the scalar relationship between two or more variables by fitting the state line on the variable values. That relationship will help to predict the variable value for future events. For example, any variable y can be modeled as linear function of another variable x with the formula $y = mx + c$. Here, x is the predictor variable, y is the response variable, m is slope of the line, and c is the intercept. Sales forecasting of products or services and predicting the price of stocks can be achieved through this regression. R provides this regression feature via the `lm` method, which is by default present in R.
- **Classification:** This is a machine-learning technique used for labeling the set of observations provided for training examples. With this, we can classify the observations into one or more labels. The likelihood of sales, online fraud detection, and cancer classification (for medical science) are common applications of classification problems. Google Mail uses this technique to classify e-mails as spam or not. Classification features can be served by `glm`, `glmnet`, `ksvm`, `svm`, and `randomForest` in R.
- **Clustering:** This technique is all about organizing similar items into groups from the given collection of items. User segmentation and image compression are the most common applications of clustering. Market segmentation, social network analysis, organizing the computer clustering, and astronomical data analysis are applications of clustering. Google News uses these techniques to group similar news items into the same category. Clustering can be achieved through the `knn`, `kmeans`, `dist`, `pvclust`, and `Mclust` methods in R.

- **Recommendation:** The recommendation algorithms are used in recommender systems where these systems are the most immediately recognizable machine learning techniques in use today. Web content recommendations may include similar websites, blogs, videos, or related content. Also, recommendation of online items can be helpful for cross-selling and up-selling. We have all seen online shopping portals that attempt to recommend books, mobiles, or any items that can be sold on the Web based on the user's past behavior. Amazon is a well-known e-commerce portal that generates 29 percent of sales through recommendation systems. Recommender systems can be implemented via `Recommender()` with the `recommenderlab` package in R.

Installing Hadoop

Now, we presume that you are aware of R, what it is, how to install it, what its key features are, and why you may want to use it. Now we need to know the limitations of R (this is a better introduction to Hadoop). Before processing the data; R needs to load the data into **random access memory (RAM)**. So, the data needs to be smaller than the available machine memory. For data that is larger than the machine memory, we consider it as Big Data (only in our case as there are many other definitions of Big Data).

To avoid this Big Data issue, we need to scale the hardware configuration; however, this is a temporary solution. To get this solved, we need to get a Hadoop cluster that is able to store it and perform parallel computation across a large computer cluster. Hadoop is the most popular solution. Hadoop is an open source Java framework, which is the top level project handled by the Apache software foundation. Hadoop is inspired by the Google filesystem and MapReduce, mainly designed for operating on Big Data by distributed processing.

Hadoop mainly supports Linux operating systems. To run this on Windows, we need to use VMware to host Ubuntu within the Windows OS. There are many ways to use and install Hadoop, but here we will consider the way that supports R best. Before we combine R and Hadoop, let us understand what Hadoop is.



Machine learning contains all the data modeling techniques that can be explored with the web link http://en.wikipedia.org/wiki/Machine_learning.

The structure blog on Hadoop installation by Michael Noll can be found at <http://www.michael-noll.com/tutorials/running-hadoop-on-ubuntu-linux-single-node-cluster/>.

Understanding different Hadoop modes

Hadoop is used with three different modes:

- **The standalone mode:** In this mode, you do not need to start any Hadoop daemons. Instead, just call `~/Hadoop-directory/bin/hadoop` that will execute a Hadoop operation as a single Java process. This is recommended for testing purposes. This is the default mode and you don't need to configure anything else. All daemons, such as NameNode, DataNode, JobTracker, and TaskTracker run in a single Java process.
- **The pseudo mode:** In this mode, you configure Hadoop for all the nodes. A separate **Java Virtual Machine (JVM)** is spawned for each of the Hadoop components or daemons like mini cluster on a single host.
- **The full distributed mode:** In this mode, Hadoop is distributed across multiple machines. Dedicated hosts are configured for Hadoop components. Therefore, separate JVM processes are present for all daemons.

Understanding Hadoop installation steps

Hadoop can be installed in several ways; we will consider the way that is better to integrate with R. We will choose Ubuntu OS as it is easy to install and access it.

1. Installing Hadoop on Linux, Ubuntu flavor (single and multinode cluster).
2. Installing Cloudera Hadoop on Ubuntu.

Installing Hadoop on Linux, Ubuntu flavor (single node cluster)

To install Hadoop over Ubuntu OS with the pseudo mode, we need to meet the following prerequisites:

- Sun Java 6
- Dedicated Hadoop system user
- Configuring SSH
- Disabling IPv6



The provided Hadoop installation will be supported with Hadoop MRv1.

Follow the given steps to install Hadoop:

1. Download the latest Hadoop sources from the Apache software foundation. Here we have considered Apache Hadoop 1.0.3, whereas the latest version is 1.1.x.

```
// Locate to Hadoop installation directory
$ cd /usr/local

// Extract the tar file of Hadoop distribution
$ sudo tar xzf hadoop-1.0.3.tar.gz

// To move Hadoop resources to hadoop folder
$ sudo mv hadoop-1.0.3 hadoop

// Make user-hduser from group-hadoop as owner of hadoop directory
$ sudo chown -R hduser:hadoop hadoop
```

2. Add the \$JAVA_HOME and \$HADOOP_HOME variables to the .bashrc file of Hadoop system user and the updated .bashrc file looks as follows:

```
// Setting the environment variables for running Java and Hadoop
commands
export HADOOP_HOME=/usr/local/hadoop
export JAVA_HOME=/usr/lib/jvm/java-6-sun

// alias for Hadoop commands
unalias fs &> /dev/null
alias fs="hadoop fs"
unalias hls &> /dev/null
alias hls="fs -ls"

// Defining the function for compressing the MapReduce job output
by lzop command
lzohead () {
hadoopfs -cat $1 | lzop -dc | head -1000 | less
}

// Adding Hadoop_HoME variable to PATH
export PATH=$PATH:$HADOOP_HOME/bin
```

3. Update the Hadoop configuration files with the conf/*-site.xml format.

Finally, the three files will look as follows:

- `conf/core-site.xml`:

```
<property>
<name>hadoop.tmp.dir</name>
<value>/app/hadoop/tmp</value>
<description>A base for other temporary directories.</description>
</property>
<property>
<name>fs.default.name</name>
<value>hdfs://localhost:54310</value>
<description>The name of the default filesystem. A URI whose
scheme and authority determine the FileSystem implementation. The
uri's scheme determines the config property (fs.SCHEME.impl)
naming
theFileSystem implementation class. The uri's authority is used to
determine the host, port, etc. for a filesystem.</description>
</property>
```
- `conf/mapred-site.xml`:

```
<property>
<name>mapred.job.tracker</name>
<value>localhost:54311</value>
<description>The host and port that the MapReduce job tracker runs
at. If "local", then jobs are run in-process as a single map
and reduce task.
</description>
</property>
```
- `conf/hdfs-site.xml`:

```
<property>
<name>dfs.replication</name>
<value>1</value>
<description>Default block replication.
  The actual number of replications can be specified when the file
is created.
  The default is used if replication is not specified in create
time.
</description>
```

After completing the editing of these configuration files, we need to set up the distributed filesystem across the Hadoop clusters or node.

- Format **Hadoop Distributed File System (HDFS)** via NameNode by using the following command line:

```
hduser@ubuntu:~$ /usr/local/hadoop/bin/hadoopnamenode -format
```

- Start your single node cluster by using the following command line:

```
hduser@ubuntu:~$ /usr/local/hadoop/bin/start-all.sh
```



Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at <http://www.packtpub.com>. If you purchased this book elsewhere, you can visit <http://www.packtpub.com/support> and register to have the files e-mailed directly to you.

Installing Hadoop on Linux, Ubuntu flavor (multinode cluster)

We learned how to install Hadoop on a single node cluster. Now we will see how to install Hadoop on a multinode cluster (the full distributed mode).

For this, we need several nodes configured with a single node Hadoop cluster. To install Hadoop on multinodes, we need to have that machine configured with a single node Hadoop cluster as described in the last section.

After getting the single node Hadoop cluster installed, we need to perform the following steps:

1. In the networking phase, we are going to use two nodes for setting up a full distributed Hadoop mode. To communicate with each other, the nodes need to be in the same network in terms of software and hardware configuration.
2. Among these two, one of the nodes will be considered as master and the other will be considered as slave. So, for performing Hadoop operations, master needs to be connected to slave. We will enter 192.168.0.1 in the master machine and 192.168.0.2 in the slave machine.
3. Update the /etc/hosts directory in both the nodes. It will look as 192.168.0.1 master and 192.168.0.2 slave.



You can perform the **Secure Shell (SSH)** setup similar to what we did for a single node cluster setup. For more details, visit <http://www.michael-noll.com>.

4. Updating `conf/*-site.xml`: We must change all these configuration files in all of the nodes.
 - `conf/core-site.xml` and `conf/mapred-site.xml`: In the single node setup, we have updated these files. So, now we need to just replace `localhost` by `master` in the value tag.
 - `conf/hdfs-site.xml`: In the single node setup, we have set the value of `dfs.replication` as 1. Now we need to update this as 2.

5. In the formatting HDFS phase, before we start the multinode cluster, we need to format HDFS with the following command (from the master node):

```
bin/hadoop namenode -format
```

Now, we have completed all the steps to install the multinode Hadoop cluster. To start the Hadoop clusters, we need to follow these steps:

1. Start HDFS daemons:

```
hduser@master:/usr/local/hadoop$ bin/start-dfs.sh
```
2. Start MapReduce daemons:

```
hduser@master:/usr/local/hadoop$ bin/start-mapred.sh
```
3. Alternatively, we can start all the daemons with a single command:

```
hduser@master:/usr/local/hadoop$ bin/start-all.sh
```
4. To stop all these daemons, fire:

```
hduser@master:/usr/local/hadoop$ bin/stop-all.sh
```

These installation steps are reproduced after being inspired by the blogs (<http://www.michael-noll.com>) of Michael Noll, who is a researcher and Software Engineer based in Switzerland, Europe. He works as a Technical lead for a large scale computing infrastructure on the Apache Hadoop stack at VeriSign.

Now the Hadoop cluster has been set up on your machines. For the installation of the same Hadoop cluster on single node or multinode with extended Hadoop components, try the Cloudera tool.

Installing Cloudera Hadoop on Ubuntu

Cloudera Hadoop (CDH) is Cloudera's open source distribution that targets enterprise class deployments of Hadoop technology. Cloudera is also a sponsor of the Apache software foundation. CDH is available in two versions: CDH3 and CDH4. To install one of these, you must have Ubuntu with either 10.04 LTS or 12.04 LTS (also, you can try CentOS, Debian, and Red Hat systems). Cloudera manager will make this installation easier for you if you are installing a Hadoop on cluster of computers, which provides GUI-based Hadoop and its component installation over a whole cluster. This tool is very much recommended for large clusters.

We need to meet the following prerequisites:

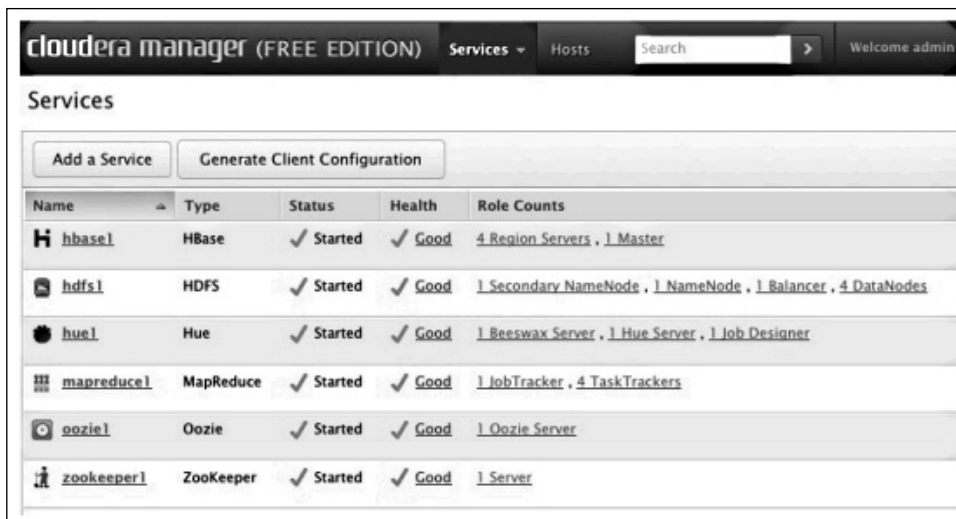
- Configuring SSH
- OS with the following criteria:
 - Ubuntu 10.04 LTS or 12.04 LTS with 64 bit
 - Red Hat Enterprise Linux 5 or 6
 - CentOS 5 or 6
 - Oracle Enterprise Linux 5
 - SUSE Linux Enterprise server 11 (SP1 or lasso)
 - Debian 6.0

The installation steps are as follows:

1. Download and run the Cloudera manager installer: To initialize the Cloudera manager installation process, we need to first download the `cloudera-manager-installer.bin` file from the download section of the Cloudera website. After that, store it at the cluster so that all the nodes can access this. Allow ownership for execution permission of `cloudera-manager-installer.bin` to the user. Run the following command to start execution.

```
$ sudo ./cloudera-manager-installer.bin
```
2. Read the Cloudera manager **Readme** and then click on **Next**.
3. Start the Cloudera manager admin console: The Cloudera manager admin console allows you to use Cloudera manager to install, manage, and monitor Hadoop on your cluster. After accepting the license from the Cloudera service provider, you need to traverse to your local web browser by entering `http://localhost:7180` in your address bar. You can also use any of the following browsers:
 - Firefox 11 or higher
 - Google Chrome
 - Internet Explorer
 - Safari

4. Log in to the Cloudera manager console with the default credentials using `admin` for both the username and password. Later on you can change it as per your choice.
5. Use the Cloudera manager for automated CDH3 installation and configuration via browser: This step will install most of the required Cloudera Hadoop packages from Cloudera to your machines. The steps are as follows:
 1. Install and validate your Cloudera manager license key file if you have chosen a full version of software.
 2. Specify the hostname or IP address range for your CDH cluster installation.
 3. Connect to each host with SSH.
 4. Install the **Java Development Kit (JDK)** (if not already installed), the Cloudera manager agent, and CDH3 or CDH4 on each cluster host.
 5. Configure Hadoop on each node and start the Hadoop services.
6. After running the wizard and using the Cloudera manager, you should change the default administrator password as soon as possible. To change the administrator password, follow these steps:
 1. Click on the icon with the gear sign to display the administration page.
 2. Open the **Password** tab.
 3. Enter a new password twice and then click on **Update**.
7. Test the Cloudera Hadoop installation: You can check the Cloudera manager installation on your cluster by logging into the Cloudera manager admin console and by clicking on the **Services** tab. You should see something like the following screenshot:



Name	Type	Status	Health	Role Counts
hbase1	HBase	✓ Started	✓ Good	4 Region Servers, 1 Master
hdfs1	HDFS	✓ Started	✓ Good	1 Secondary NameNode, 1 NameNode, 1 Balancer, 4 DataNodes
hue1	Hue	✓ Started	✓ Good	1 Beeswax Server, 1 Hue Server, 1 Job Designer
mapreduce1	MapReduce	✓ Started	✓ Good	1 JobTracker, 4 TaskTrackers
oozie1	Oozie	✓ Started	✓ Good	1 Oozie Server
zookeeper1	ZooKeeper	✓ Started	✓ Good	1 Server

Cloudera manager admin console

8. You can also click on each service to see more detailed information. For example, if you click on the **hdfs1** link, you might see something like the following screenshot:



Status and Health Summary		
	Status	Health
NameNode	✓ Started	✓ Good
Secondary NameNode	✓ Started	✓ Good
DataNode	✓ 4 Started	✓ 4 Good
Balancer	⊘ Stopped	✓ Good

Cloudera manger admin console – HDFS service



To avoid these installation steps, use preconfigured Hadoop instances with Amazon Elastic MapReduce and MapReduce.

If you want to use Hadoop on Windows, try the HDP tool by Hortonworks. This is 100 percent open source, enterprise grade distribution of Hadoop. You can download the HDP tool at <http://hortonworks.com/download/>.

Understanding Hadoop features

Hadoop is specially designed for two core concepts: HDFS and MapReduce. Both are related to distributed computation. MapReduce is believed as the heart of Hadoop that performs parallel processing over distributed data.

Let us see more details on Hadoop's features:

- HDFS
- MapReduce

Understanding HDFS

HDFS is Hadoop's own rack-aware filesystem, which is a UNIX-based data storage layer of Hadoop. HDFS is derived from concepts of Google filesystem. An important characteristic of Hadoop is the partitioning of data and computation across many (thousands of) hosts, and the execution of application computations in parallel, close to their data. On HDFS, data files are replicated as sequences of blocks in the cluster. A Hadoop cluster scales computation capacity, storage capacity, and I/O bandwidth by simply adding commodity servers. HDFS can be accessed from applications in many different ways. Natively, HDFS provides a Java API for applications to use.

The Hadoop clusters at Yahoo! span 40,000 servers and store 40 petabytes of application data, with the largest Hadoop cluster being 4,000 servers. Also, one hundred other organizations worldwide are known to use Hadoop.

Understanding the characteristics of HDFS

Let us now look at the characteristics of HDFS:

- Fault tolerant
- Runs with commodity hardware
- Able to handle large datasets
- Master slave paradigm
- Write once file access only

Understanding MapReduce

MapReduce is a programming model for processing large datasets distributed on a large cluster. MapReduce is the heart of Hadoop. Its programming paradigm allows performing massive data processing across thousands of servers configured with Hadoop clusters. This is derived from Google MapReduce.

Hadoop MapReduce is a software framework for writing applications easily, which process large amounts of data (multiterabyte datasets) in parallel on large clusters (thousands of nodes) of commodity hardware in a reliable, fault-tolerant manner. This MapReduce paradigm is divided into two phases, Map and Reduce that mainly deal with key and value pairs of data. The Map and Reduce task run sequentially in a cluster; the output of the Map phase becomes the input for the Reduce phase. These phases are explained as follows:

- **Map phase:** Once divided, datasets are assigned to the task tracker to perform the Map phase. The data functional operation will be performed over the data, emitting the mapped key and value pairs as the output of the Map phase.
- **Reduce phase:** The master node then collects the answers to all the subproblems and combines them in some way to form the output; the answer to the problem it was originally trying to solve.

The five common steps of parallel computing are as follows:

1. Preparing the `Map()` input: This will take the input data row wise and emit key value pairs per rows, or we can explicitly change as per the requirement.
 - Map input: list (k1, v1)
2. Run the user-provided `Map()` code
 - Map output: list (k2, v2)
3. Shuffle the Map output to the Reduce processors. Also, shuffle the similar keys (grouping them) and input them to the same reducer.
4. Run the user-provided `Reduce()` code: This phase will run the custom reducer code designed by developer to run on shuffled data and emit key and value.
 - Reduce input: (k2, list(v2))
 - Reduce output: (k3, v3)
5. Produce the final output: Finally, the master node collects all reducer output and combines and writes them in a text file.



The reference links to review on Google filesystem can be found at <http://research.google.com/archive/gfs.html> and Google MapReduce can be found at <http://research.google.com/archive/mapreduce.html>.

Learning the HDFS and MapReduce architecture

Since HDFS and MapReduce are considered to be the two main features of the Hadoop framework, we will focus on them. So, let's first start with HDFS.

Understanding the HDFS architecture

HDFS can be presented as the master/slave architecture. HDFS master is named as NameNode whereas slave as DataNode. NameNode is a server that manages the filesystem namespace and adjusts the access (open, close, rename, and more) to files by the client. It divides the input data into blocks and announces which data block will be stored in which DataNode. DataNode is a slave machine that stores the replicas of the partitioned dataset and serves the data as the request comes. It also performs block creation and deletion.

The internal mechanism of HDFS divides the file into one or more blocks; these blocks are stored in a set of data nodes. Under normal circumstances of the replication factor three, the HDFS strategy is to place the first copy on the local node, second copy on the local rack with a different node, and a third copy into different racks with different nodes. As HDFS is designed to support large files, the HDFS block size is defined as 64 MB. If required, this can be increased.

Understanding HDFS components

HDFS is managed with the master-slave architecture included with the following components:

- **NameNode:** This is the master of the HDFS system. It maintains the directories, files, and manages the blocks that are present on the DataNodes.
- **DataNode:** These are slaves that are deployed on each machine and provide actual storage. They are responsible for serving read-and-write data requests for the clients.
- **Secondary NameNode:** This is responsible for performing periodic checkpoints. So, if the NameNode fails at any time, it can be replaced with a snapshot image stored by the secondary NameNode checkpoints.

Understanding the MapReduce architecture

MapReduce is also implemented over master-slave architectures. Classic MapReduce contains job submission, job initialization, task assignment, task execution, progress and status update, and job completion-related activities, which are mainly managed by the JobTracker node and executed by TaskTracker. Client application submits a job to the JobTracker. Then input is divided across the cluster. The JobTracker then calculates the number of map and reducer to be processed. It commands the TaskTracker to start executing the job. Now, the TaskTracker copies the resources to a local machine and launches JVM to map and reduce program over the data. Along with this, the TaskTracker periodically sends update to the JobTracker, which can be considered as the heartbeat that helps to update JobID, job status, and usage of resources.

Understanding MapReduce components

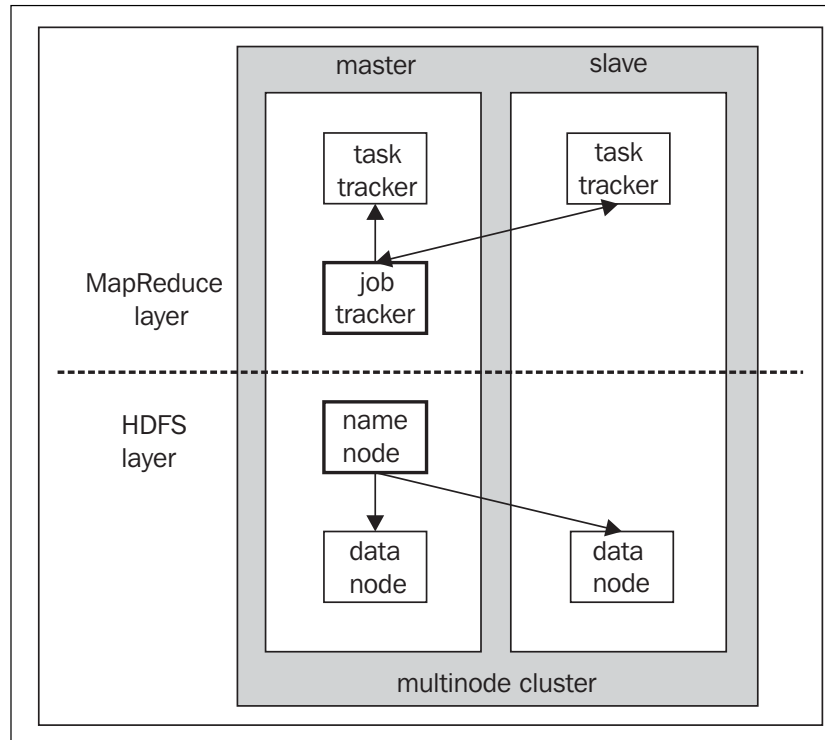
MapReduce is managed with master-slave architecture included with the following components:

- **JobTracker:** This is the master node of the MapReduce system, which manages the jobs and resources in the cluster (TaskTrackers). The JobTracker tries to schedule each map as close to the actual data being processed on the TaskTracker, which is running on the same DataNode as the underlying block.
- **TaskTracker:** These are the slaves that are deployed on each machine. They are responsible for running the map and reducing tasks as instructed by the JobTracker.

Understanding the HDFS and MapReduce architecture by plot

In this plot, both HDFS and MapReduce master and slave components have been included, where NameNode and DataNode are from HDFS and JobTracker and TaskTracker are from the MapReduce paradigm.

Both paradigms consisting of master and slave candidates have their own specific responsibility to handle MapReduce and HDFS operations. In the next plot, there is a plot with two sections: the preceding one is a MapReduce layer and the following one is an HDFS layer.



The HDFS and MapReduce architecture

Hadoop is a top-level Apache project and is a very complicated Java framework. To avoid technical complications, the Hadoop community has developed a number of Java frameworks that has added an extra value to Hadoop features. They are considered as Hadoop subprojects. Here, we are departing to discuss several Hadoop components that can be considered as an abstraction of HDFS or MapReduce.

Understanding Hadoop subprojects

Mahout is a popular data mining library. It takes the most popular data mining scalable machine learning algorithms for performing clustering, classification, regression, and statistical modeling to prepare intelligent applications. Also, it is a scalable machine-learning library.

Apache Mahout is distributed under a commercially friendly Apache software license. The goal of Apache Mahout is to build a vibrant, responsive, and diverse community to facilitate discussions not only on the project itself but also on potential use cases.

The following are some companies that are using Mahout:

- **Amazon:** This is a shopping portal for providing personalization recommendation
- **AOL:** This is a shopping portal for shopping recommendations
- **Drupal:** This is a PHP content management system using Mahout for providing open source content-based recommendation
- **iOffer:** This is a shopping portal, which uses Mahout's Frequent Pattern Set Mining and collaborative filtering to recommend items to users
- **LucidWorks Big Data:** This is a popular analytics firm, which uses Mahout for clustering, duplicate document detection, phrase extraction, and classification
- **Radoop:** This provides a drag-and-drop interface for Big Data analytics, including Mahout clustering and classification algorithms
- **Twitter:** This is a social networking site, which uses Mahout's **Latent Dirichlet Allocation (LDA)** implementation for user interest modeling and maintains a fork of Mahout on GitHub.
- **Yahoo!:** This is the world's most popular web service provider, which uses Mahout's Frequent Pattern Set Mining for Yahoo! Mail



The reference links on the Hadoop ecosystem can be found at <http://www.revelytix.com/?q=content/hadoop-ecosystem>.

Apache HBase is a distributed Big Data store for Hadoop. This allows random, real-time read/write access to Big Data. This is designed as a column-oriented data storage model innovated after inspired by Google BigTable.

The following are the companies using HBase:

- **Yahoo!:** This is the world's popular web service provider for near duplicate document detection
- **Twitter:** This is a social networking site for version control storage and retrieval
- **Mahalo:** This is a knowledge sharing service for similar content recommendation
- **NING:** This is a social network service provider for real-time analytics and reporting
- **StumbleUpon:** This is a universal personalized recommender system, real-time data storage, and data analytics platform
- **Veoh:** This is an online multimedia content sharing platform for user profiling system



For Google Big Data, distributed storage system for structured data, refer the link <http://research.google.com/archive/bigtable.html>.

Hive is a Hadoop-based data warehousing like framework developed by Facebook. It allows users to fire queries in SQL-like languages, such as HiveQL, which are highly abstracted to Hadoop MapReduce. This allows SQL programmers with no MapReduce experience to use the warehouse and makes it easier to integrate with business intelligence and visualization tools for real-time query processing.

Pig is a Hadoop-based open source platform for analyzing the large scale datasets via its own SQL-like language: Pig Latin. This provides a simple operation and programming interface for massive, complex data-parallelization computation. This is also easier to develop; it's more optimized and extensible. Apache Pig has been developed by Yahoo!. Currently, Yahoo! and Twitter are the primary Pig users.

For developers, the direct use of Java APIs can be tedious or error-prone, but also limits the Java programmer's use of Hadoop programming's flexibility. So, Hadoop provides two solutions that enable making Hadoop programming for dataset management and dataset analysis with MapReduce easier — these are Pig and Hive, which are always confusing.

Apache Sqoop provides Hadoop data processing platform and relational databases, data warehouse, and other non-relational databases quickly transferring large amounts of data in a new way. Apache Sqoop is a mutual data tool for importing data from the relational databases to Hadoop HDFS and exporting data from HDFS to relational databases.

It works together with most modern relational databases, such as MySQL, PostgreSQL, Oracle, Microsoft SQL Server, and IBM DB2, and enterprise data warehouse. Sqoop extension API provides a way to create new connectors for the database system. Also, the Sqoop source comes up with some popular database connectors. To perform this operation, Sqoop first transforms the data into Hadoop MapReduce with some logic of database schema creation and transformation.

Apache Zookeeper is also a Hadoop subproject used for managing Hadoop, Hive, Pig, HBase, Solr, and other projects. Zookeeper is an open source distributed applications coordination service, which is designed with Fast Paxos algorithm-based synchronization and configuration and naming services such as maintenance of distributed applications. In programming, Zookeeper design is a very simple data model style, much like the system directory tree structure.

Zookeeper is divided into two parts: the server and client. For a cluster of Zookeeper servers, only one acts as a leader, which accepts and coordinates all rights. The rest of the servers are read-only copies of the master. If the leader server goes down, any other server can start serving all requests. Zookeeper clients are connected to a server on the Zookeeper service. The client sends a request, receives a response, accesses the observer events, and sends a heartbeat via a TCP connection with the server.

For a high-performance coordination service for distributed applications, Zookeeper is a centralized service for maintaining configuration information, naming, and providing distributed synchronization and group services. All these kinds of services are used in some form or another by distributed applications. Each time they are implemented, there is a lot of work that goes into fixing the bugs and race conditions that are inevitable. These services lead to management complexity when the applications are deployed.

Apache Solr is an open source enterprise search platform from the Apache license project. Apache Solr is highly scalable, supporting distributed search and index replication engine. This allows building web application with powerful text search, faceted search, real-time indexing, dynamic clustering, database integration, and rich document handling.

Apache Solr is written in Java, which runs as a standalone server to serve the search results via REST-like HTTP/XML and JSON APIs. So, this Solr server can be easily integrated with an application, which is written in other programming languages. Due to all these features, this search server is used by Netflix, AOL, CNET, and Zappos.

Ambari is very specific to Hortonworks. Apache Ambari is a web-based tool that supports Apache Hadoop cluster supply, management, and monitoring. Ambari handles most of the Hadoop components, including HDFS, MapReduce, Hive, Pig, HBase, Zookeeper, Sqoop, and HCatlog as centralized management.

In addition, Ambari is able to install security based on the Kerberos authentication protocol over the Hadoop cluster. Also, it provides role-based user authentication, authorization, and auditing functions for users to manage integrated LDAP and Active Directory.

Summary

In this chapter, we learned what is R, Hadoop, and their features, and how to install them before going on to analyzing the datasets with R and Hadoop. In the next chapter, we are going to learn what MapReduce is and how to develop MapReduce programs with Apache Hadoop.

Where to buy this book

You can buy Big Data Analytics with R and Hadoop from the Packt Publishing website:
<http://www.packtpub.com/big-data-analytics-with-r-and-hadoop/book>

Free shipping to the US, UK, Europe and selected Asian countries. For more information, please read our [shipping policy](#).

Alternatively, you can buy the book from Amazon, BN.com, Computer Manuals and most internet book retailers.



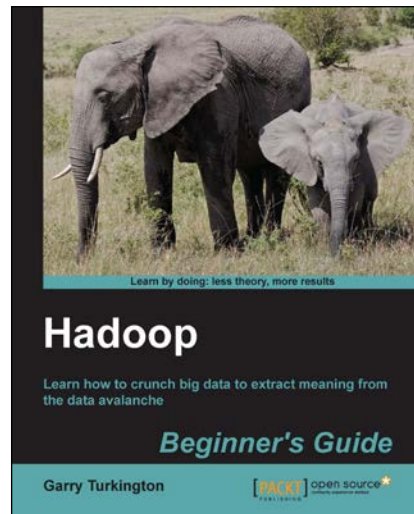
www.PacktPub.com

For More Information:

www.packtpub.com/big-data-analytics-with-r-and-hadoop/book

Hadoop Beginner's Guide

Garry Turkington



Chapter No. 4

"Developing MapReduce Programs"

In this package, you will find:

A Biography of the author of the book

A preview chapter from the book, Chapter NO.4 "Developing MapReduce Programs"

A synopsis of the book's content

Information on where to buy this book

About the Author

Garry Turkington has 14 years of industry experience, most of which has been focused on the design and implementation of large-scale distributed systems. In his current roles as VP Data Engineering and Lead Architect at Improve Digital, he is primarily responsible for the realization of systems that store, process, and extract value from the company's large data volumes. Before joining Improve Digital, he spent time at Amazon.co.uk, where he led several software development teams building systems that process Amazon catalog data for every item worldwide. Prior to this, he spent a decade in various government positions in both the UK and USA.

He has BSc and PhD degrees in Computer Science from the Queens University of Belfast in Northern Ireland and an MEng in Systems Engineering from Stevens Institute of Technology in the USA.

I would like to thank my wife Lea for her support and encouragement—not to mention her patience—throughout the writing of this book and my daughter, Maya, whose spirit and curiosity is more of an inspiration than she could ever imagine.

For More Information:

www.packtpub.com/hadoop-beginners-guide/book

Hadoop Beginner's Guide

This book is here to help you make sense of Hadoop and use it to solve your big data problems. It's a really exciting time to work with data processing technologies such as Hadoop. The ability to apply complex analytics to large data sets—once the monopoly of large corporations and government agencies—is now possible through free **open source software (OSS)**.

But because of the seeming complexity and pace of change in this area, getting a grip on the basics can be somewhat intimidating. That's where this book comes in, giving you an understanding of just what Hadoop is, how it works, and how you can use it to extract value from your data now.

In addition to an explanation of core Hadoop, we also spend several chapters exploring other technologies that either use Hadoop or integrate with it. Our goal is to give you an understanding not just of what Hadoop is but also how to use it as a part of your broader technical infrastructure.

A complementary technology is the use of cloud computing, and in particular, the offerings from Amazon Web Services. Throughout the book, we will show you how to use these services to host your Hadoop workloads, demonstrating that not only can you process large data volumes, but also you don't actually need to buy any physical hardware to do so.

What This Book Covers

This book comprises of three main parts: chapters 1 through 5, which cover the core of Hadoop and how it works, chapters 6 and 7, which cover the more operational aspects of Hadoop, and chapters 8 through 11, which look at the use of Hadoop alongside other products and technologies.

Chapter 1, What It's All About, gives an overview of the trends that have made Hadoop and cloud computing such important technologies today.

Chapter 2, Getting Hadoop Up and Running, walks you through the initial setup of a local Hadoop cluster and the running of some demo jobs. For comparison, the same work is also executed on the hosted Hadoop Amazon service.

Chapter 3, Understanding MapReduce, goes inside the workings of Hadoop to show how MapReduce jobs are executed and shows how to write applications using the Java API.

Chapter 4, Developing MapReduce Programs, takes a case study of a moderately sized data set to demonstrate techniques to help when deciding how to approach the processing and analysis of a new data source.

For More Information:

www.packtpub.com/hadoop-beginners-guide/book

Chapter 5, Advanced MapReduce Techniques, looks at a few more sophisticated ways of applying MapReduce to problems that don't necessarily seem immediately applicable to the Hadoop processing model.

Chapter 6, When Things Break, examines Hadoop's much-vaunted high availability and fault tolerance in some detail and sees just how good it is by intentionally causing havoc through killing processes and intentionally using corrupt data.

Chapter 7, Keeping Things Running, takes a more operational view of Hadoop and will be of most use for those who need to administer a Hadoop cluster. Along with demonstrating some best practice, it describes how to prepare for the worst operational disasters so you can sleep at night.

Chapter 8, A Relational View On Data With Hive, introduces Apache Hive, which allows Hadoop data to be queried with a SQL-like syntax.

Chapter 9, Working With Relational Databases, explores how Hadoop can be integrated with existing databases, and in particular, how to move data from one to the other.

Chapter 10, Data Collection with Flume, shows how Apache Flume can be used to gather data from multiple sources and deliver it to destinations such as Hadoop.

Chapter 11, Where To Go Next, wraps up the book with an overview of the broader Hadoop ecosystem, highlighting other products and technologies of potential interest. In addition, it gives some ideas on how to get involved with the Hadoop community and to get help.

For More Information:

www.packtpub.com/hadoop-beginners-guide/book

4

Developing MapReduce Programs

Now that we have explored the technology of MapReduce, we will spend this chapter looking at how to put it to use. In particular, we will take a more substantial dataset and look at ways to approach its analysis by using the tools provided by MapReduce.

In this chapter we will cover the following topics:

- ◆ Hadoop Streaming and its uses
- ◆ The UFO sighting dataset
- ◆ Using Streaming as a development/debugging tool
- ◆ Using multiple mappers in a single job
- ◆ Efficiently sharing utility files and data across the cluster
- ◆ Reporting job and task status and log information useful for debugging

Throughout this chapter, the goal is to introduce both concrete tools and ideas about how to approach the analysis of a new data set. We shall start by looking at how to use scripting programming languages to aid MapReduce prototyping and initial analysis. Though it may seem strange to learn the Java API in the previous chapter and immediately move to different languages, our goal here is to provide you with an awareness of different ways to approach the problems you face. Just as many jobs make little sense being implemented in anything but the Java API, there are other situations where using another approach is best suited. Consider these techniques as new additions to your tool belt and with that experience you will know more easily which is the best fit for a given scenario.

For More Information:

www.packtpub.com/hadoop-beginners-guide/book

Using languages other than Java with Hadoop

We have mentioned previously that MapReduce programs don't have to be written in Java. Most programs are written in Java, but there are several reasons why you may want or need to write your map and reduce tasks in another language. Perhaps you have existing code to leverage or need to use third-party binaries—the reasons are varied and valid.

Hadoop provides a number of mechanisms to aid non-Java development, primary amongst these are **Hadoop Pipes** that provides a native C++ interface to Hadoop and **Hadoop Streaming** that allows any program that uses standard input and output to be used for map and reduce tasks. We will use Hadoop Streaming heavily in this chapter.

How Hadoop Streaming works

With the MapReduce Java API, both map and reduce tasks provide implementations for methods that contain the task functionality. These methods receive the input to the task as method arguments and then output results via the `Context` object. This is a clear and type-safe interface but is by definition Java specific.

Hadoop Streaming takes a different approach. With Streaming, you write a map task that reads its input from standard input, one line at a time, and gives the output of its results to standard output. The reduce task then does the same, again using only standard input and output for its data flow.

Any program that reads and writes from standard input and output can be used in Streaming, such as compiled binaries, Unixshell scripts, or programs written in a dynamic language such as Ruby or Python.

Why to use Hadoop Streaming

The biggest advantage to Streaming is that it can allow you to try ideas and iterate on them more quickly than using Java. Instead of a compile/jar/submit cycle, you just write the scripts and pass them as arguments to the Streaming jar file. Especially when doing initial analysis on a new dataset or trying out new ideas, this can significantly speed up development.

The classic debate regarding dynamic versus static languages balances the benefits of swift development against runtime performance and type checking. These dynamic downsides also apply when using Streaming. Consequently, we favor use of Streaming for up-front analysis and Java for the implementation of jobs that will be executed on the production cluster.

We will use Ruby for Streaming examples in this chapter, but that is a personal preference. If you prefer shell scripting or another language, such as Python, then take the opportunity to convert the scripts used here into the language of your choice.

Time for action – implementing WordCount using Streaming

Let's flog the dead horse of WordCount one more time and implement it using Streaming by performing the following steps:

1. Save the following file to `wcmapper.rb`:

```
#!/bin/env ruby

while line = gets
  words = line.split("\t")
  words.each{ |word| puts word.strip+"\t1"}}
end
```

2. Make the file executable by executing the following command:

```
$ chmod +x wcmapper.rb
```

3. Save the following file to `wcreducer.rb`:

```
#!/usr/bin/env ruby

current = nil
count = 0

while line = gets
  word, counter = line.split("\t")

  if word == current
    count = count+1
  else
    puts current+"\t"+count.to_s if current
    current = word
    count = 1
  end
end
puts current+"\t"+count.to_s
```

4. Make the file executable by executing the following command:

```
$ chmod +x wcreducer.rb
```

5. Execute the scripts as a Streaming job using the datafile from the previous chapter:

```
$ hadoop jar hadoop/contrib/streaming/hadoop-streaming-1.0.3.jar
-file wcmapper.rb -mapper wcmapper.rb -file wcreducer.rb
-reducer wcreducer.rb -input test.txt -output output
packageJobJar: [wcmapper.rb, wcreducer.rb, /tmp/hadoop-
hadoop/hadoop-unjar1531650352198893161/] [] /tmp/
streamjob937274081293220534.jar tmpDir=null

12/02/05 12:43:53 INFO mapred.FileInputFormat: Total input paths
to process : 1

12/02/05 12:43:53 INFO streaming.StreamJob: getLocalDirs(): [/var/
hadoop/mapred/local]

12/02/05 12:43:53 INFO streaming.StreamJob: Running job:
job_201202051234_0005

...

12/02/05 12:44:01 INFO streaming.StreamJob: map 100% reduce 0%
12/02/05 12:44:13 INFO streaming.StreamJob: map 100% reduce 100%
12/02/05 12:44:16 INFO streaming.StreamJob: Job complete:
job_201202051234_0005

12/02/05 12:44:16 INFO streaming.StreamJob: Output: wcoutput
```

6. Check the result file:

```
$ hadoop fs -cat output/part-00000
```

What just happened?

Ignore the specifics of Ruby. If you don't know the language, it isn't important here.

Firstly, we created the script that will be our mapper. It uses the `gets` function to read a line from standard input, splits this into words, and uses the `puts` function to write the word and the value 1 to the standard output. We then made the file executable.

Our reducer is a little more complex for reasons we will describe in the next section. However, it performs the job we would expect, it counts the number of occurrences for each word, reads from standard input, and gives the output as the final value to standard output. Again we made sure to make the file executable.

Note that in both cases we are implicitly using Hadoop input and output formats discussed in the earlier chapters. It is the `TextInputFormat` property that processes the source file and provides each line one at a time to the map script. Conversely, the `TextOutputFormat` property will ensure that the output of reduce tasks is also correctly written as textual data. We can of course modify these if required.

Next, we submitted the Streaming job to Hadoop via the rather cumbersome command line shown in the previous section. The reason for each file to be specified twice is that any file not available on each node must be packaged up by Hadoop and shipped across the cluster, which requires it to be specified by the `-file` option. Then, we also need to tell Hadoop which script performs the mapper and reducer roles.

Finally, we looked at the output of the job, which should be identical to the previous Java-based WordCount implementations

Differences in jobs when using Streaming

The Streaming WordCount mapper looks a lot simpler than the Java version, but the reducer appears to have more logic. Why? The reason is that the implied contract between Hadoop and our tasks changes when we use Streaming.

In Java we knew that our `map()` method would be invoked once for each input key/value pair and our `reduce()` method would be invoked for each key and its set of values.

With Streaming we don't have the concept of the `map` or `reduce` methods anymore, instead we have written scripts that process streams of received data. This changes how we need to write our reducer. In Java the grouping of values to each key was performed by Hadoop; each invocation of the `reduce` method would receive a single key and all its values. In Streaming, each instance of the `reduce` task is given the individual ungathered values one at a time.

Hadoop Streaming does sort the keys, for example, if a mapper emitted the following data:

```
First    1
Word     1
Word     1
A        1
First    1
```

The Streaming reducer would receive this data in the following order:

```
A        1
First    1
First    1
Word     1
Word     1
```

Hadoop still collects the values for each key and ensures that each key is passed only to a single reducer. In other words, a reducer gets all the values for a number of keys and they are grouped together; however, they are not packaged into individual executions of the reducer, that is, one per key, as with the Java API.

This should explain the mechanism used in the Ruby reducer; it first sets empty default values for the current word; then after reading each line it determines if this is another value for the current key, and if so, increments the count. If not, then there will be no more values for the previous key and its final output is sent to standard output and the counting begins again for the new word.

After reading so much in the earlier chapters about how great it is for Hadoop to do so much for us, this may seem a lot more complex, but after you write a few Streaming reducers it's actually not as bad as it may first appear. Also remember that Hadoop does still manage the assignment of splits to individual map tasks and the necessary coordination that sends the values for a given key to the same reducer. This behavior can be modified through configuration settings to change the number of mappers and reducers just as with the Java API.

Analyzing a large dataset

Armed with our abilities to write MapReduce jobs in both Java and Streaming, we'll now explore a more significant dataset than any we've looked at before. In the following section, we will attempt to show how to approach such analysis and the sorts of questions Hadoop allows you to ask of a large dataset.

Getting the UFO sighting dataset

We will use a public domain dataset of over 60,000 UFO sightings. This is hosted by InfoChimps at <http://www.infochimps.com/datasets/60000-documented-ufo-sightings-with-text-descriptions-and-metadata>.

You will need to register for a free InfoChimps account to download a copy of the data.

The data comprises a series of UFO sighting records with the following fields:

1. **Sighting date:** This field gives the date when the UFO sighting occurred.
2. **Recorded date:** This field gives the date when the sighting was reported, often different to the sighting date.
3. **Location:** This field gives the location where the sighting occurred.
4. **Shape:** This field gives a brief summary of the shape of the UFO, for example, diamond, lights, cylinder.
5. **Duration:** This field gives the duration of how long the sighting lasted.
6. **Description:** This field gives free text details of the sighting.

Once downloaded, you will find the data in a few formats. We will be using the `.tsv` (tab-separated value) version.

Getting a feel for the dataset

When faced with a new dataset it is often difficult to get a feel for the nature, breadth, and quality of the data involved. There are several questions, the answers to which will affect how you approach the follow-on analysis, in particular:

- ◆ How big is the dataset?
- ◆ How complete are the records?
- ◆ How well do the records match the expected format?

The first is a simple question of scale; are we talking hundreds, thousands, millions, or more records? The second question asks how complete the records are. If you expect each record to have 10 fields (if this is structured or semi-structured data), how many have key fields populated with data? The last question expands on this point, how well do the records match your expectations of format and representation?

Time for action – summarizing the UFO data

Now we have the data, let's get an initial summarization of its size and how many records may be incomplete:

1. With the UFO **tab-separated value (TSV)** file on HDFS saved as `ufo.tsv`, save the following file to `summarymapper.rb`:

```
#!/usr/bin/env ruby

while line = gets
  puts "total\t1"
  parts = line.split("\t")
  puts "badline\t1" if parts.size != 6
  puts "sighted\t1" if !parts[0].empty?
  puts "recorded\t1" if !parts[1].empty?
  puts "location\t1" if !parts[2].empty?
  puts "shape\t1" if !parts[3].empty?
  puts "duration\t1" if !parts[4].empty?
  puts "description\t1" if !parts[5].empty?
end
```

2. Make the file executable by executing the following command:

```
$ chmod +x summarymapper.rb
```

3. Execute the job as follows by using Streaming:

```
$ hadoop jar hadoop/contrib/streaming/hadoop-streaming-1.0.3.jar  
-file summarymapper.rb -mapper summarymapper.rb -file wcReducer.rb  
-reducer wcReducer.rb -input ufo.tsv -output ufosummary
```

4. Retrieve the summary data:

```
$ hadoop fs -cat ufosummary/part-0000
```

What just happened?

Remember that our UFO sightings should have six fields as described previously. They are listed as follows:

- ◆ The date of the sighting
- ◆ The date the sighting was reported
- ◆ The location of the sighting
- ◆ The shape of the object
- ◆ The duration of the sighting
- ◆ A free text description of the event

The mapper examines the file and counts the total number of records in addition to identifying potentially incomplete records.

We produce the overall count by simply recording how many distinct records are encountered while processing the file. We identify potentially incomplete records by flagging those that either do not contain exactly six fields or have at least one field that has a null value.

Therefore, the implementation of the mapper reads each line and does three things as it proceeds through the file:

- ◆ It gives the output of a token to be incremented in the total number of records processed
- ◆ It splits the record on tab boundaries and records any occurrence of lines which do not result in six fields' values

- ◆ For each of the six expected fields it reports when the values present are other than an empty string, that is, there is data in the field, though this doesn't actually say anything about the quality of that data

We wrote this mapper intentionally to produce the output of the form `(token, count)`. Doing this allowed us to use our existing WordCount reducer from our earlier implementations as the reducer for this job. There are certainly more efficient implementations, but as this job is unlikely to be frequently executed, the convenience is worth it.

At the time of writing, the result of this job was as follows:

```
badline324
description61372
duration58961
location61377
recorded61377
shape58855
sighted61377
total61377
```

We see from these figures that we have 61,300 records. All of these provide values for the sighted date, reported date, and location fields. Around 58,000-59,000 records have values for shape and duration, and almost all have a description.

When split on tab characters there were 372 lines found to not have exactly six fields. However, since only five records had no value for description, this suggests that the bad records typically have too many tabs as opposed to too few. We could of course alter our mapper to gather detailed information on this fact. This is likely due to tabs being used in the free text description, so for now we will do our analysis expecting most records to have correctly placed values for all the six fields, but not make any assumptions regarding further tabs in each record.

Examining UFO shapes

Out of all the fields in these reports, it was shape that immediately interested us most, as it could offer some interesting ways of grouping the data depending on what sort of information we have in that field.

Time for action – summarizing the shape data

Just as we provided a summarization for the overall UFO data set earlier, let's now do a more focused summarization on the data provided for UFO shapes:

1. Save the following to `shapemapper.rb`:

```
#!/usr/bin/env ruby

while line = gets
  parts = line.split("\t")
  if parts.size == 6
    shape = parts[3].strip
    puts shape+"\t1" if !shape.empty?
  end
end
```

2. Make the file executable:

```
$ chmod +x shapemapper.rb
```

3. Execute the job once again using the WordCount reducer:

```
$ hadoop jar hadoop/contrib/streaming/hadoop-streaming-1.0.3.jar
--file shapemapper.rb -mapper shapemapper.rb -file wcreducer.rb
-reducer wcreducer.rb -input ufo.tsv -output shapes
```

4. Retrieve the shape info:

```
$ hadoop fs -cat shapes/part-00000
```

What just happened?

Our mapper here is pretty simple. It breaks each record into its constituent fields, discards any without exactly six fields, and gives a counter as the output for any non-empty shape value.

For our purposes here, we are happy to ignore any records that don't precisely match the specification we expect. Perhaps one record is the single UFO sighting that will prove it once and for all, but even so it wouldn't likely make much difference to our analysis. Think about the potential value of individual records before deciding to so easily discard some. If you are working primarily on large aggregations where you care mostly about trends, individual records likely don't matter. But in cases where single individual values could materially affect the analysis or must be accounted for, an approach of trying to parse and recover more conservatively rather than discard may be best. We'll talk more about this trade-off in *Chapter 6, When Things Break*.

After the usual routine of making the mapper executable and running the job we produced, data showing 29 different UFO shapes were reported. Here's some sample output tabulated in compact form for space reasons:

```
changed1 changing1533 chevron758 cigar1774
circle5250 cone265 crescent2 cross177
cylinder981 delta8 diamond909 disk4798
dome1 egg661 fireball13437 flare1
flash988 formation1775 hexagon1 light12140
other4574 oval2859 pyramid1 rectangle957
round2 sphere3614 teardrop592 triangle6036
unknown4459
```

As we can see, there is a wide variance in sighting frequency. Some such as pyramid occur only once, while light comprises more than a fifth of all reported shapes. Considering many UFO sightings are at night, it could be argued that a description of light is not terribly useful or specific and when combined with the values for other and unknown we see that around 21000 of our 58000 reported shapes may not actually be of any use. Since we are not about to run out and do additional research, this doesn't matter very much, but what's important is to start thinking of your data in these terms. Even these types of summary analysis can start giving an insight into the nature of the data and indicate what quality of analysis may be possible. In the case of reported shapes, for example, we have already discovered that out of our 61000 sightings only 58000 reported the shape and of these 21000 are of dubious value. We have already determined that our 61000 sample set only provides 37000 shape reports that we may be able to work with. If your analysis is predicated on a minimum number of samples, always be sure to do this sort of summarization up-front to determine if the data set will actually meet your needs.

Time for action – correlating of sighting duration to UFO shape

Let's do a little more detailed analysis in regards to this shape data. We wondered if there was any correlation between the duration of a sighting to the reported shape. Perhaps cigar-shaped UFOs hang around longer than the rest or formations always appear for the exact amount of time.

1. Save the following to `shapetimemapper.rb`:

```
#!/usr/bin/env ruby

pattern = Regexp.new /\d* ?((min) | (sec))/

while line = gets
  parts = line.split("\t")
  if parts.size == 6
```

```
shape = parts[3].strip
duration = parts[4].strip.downcase
if !shape.empty? && !duration.empty?
  match = pattern.match(duration)
  time = /\d*/.match(match[0])[0]
  unit = match[1]
  time = Integer(time)
  time = time * 60 if unit == "min"
  puts shape+"\t"+time.to_s
end
end
end
```

2. Make the file executable by executing the following command:

```
$ chmod +x shapetimemapper.rb
```

3. Save the following to shapetimereducer.rb:

```
#!/usr/bin/env ruby

current = nil
min = 0
max = 0
mean = 0
total = 0
count = 0

while line = gets
  word, time = line.split("\t")
  time = Integer(time)

  if word == current
    count = count+1
    total = total+time
    min = time if time < min
    max = time if time > max
  else
    puts current+"\t"+min.to_s+" "+max.to_s+" "+(total/count).to_s if
    current
    current = word
    count = 1
    total = time
    min = time
    max = time
  end
end
puts current+"\t"+min.to_s+" "+max.to_s+" "+(total/count).to_s
```

4. Make the file executable by executing the following command:

```
$ chmod +x shapetimereducer.rb
```

5. Run the job:

```
$ hadoop jar hadoop/contrib/streaminghadoop-streaming-1.0.3.jar  
-file shapetimemapper.rb -mapper shapetimemapper.rb -file  
shapetimereducer.rb -reducer shapetimereducer.rb -input ufo.tsv  
-output shapetime
```

6. Retrieve the results:

```
$ hadoop fs -cat shapetime/part-00000
```

What just happened?

Our mapper here is a little more involved than previous examples due to the nature of the duration field. Taking a quick look at some sample records, we found values as follows:

```
15 seconds  
2 minutes  
2 min  
2minutes  
5-10 seconds
```

In other words, there was a mixture of range and absolute values, different formatting and inconsistent terms for time units. Again for simplicity we decided on a limited interpretation of the data; we will take the absolute value if present, and the upper part of a range if not. We would assume that the strings `min` or `sec` would be present for the time units and would convert all timings into seconds. With some regular expression magic, we unpack the duration field into these parts and do the conversion. Note again that we simply discard any record that does not work as we expect, which may not always be appropriate.

The reducer follows the same pattern as our earlier example, starting with a default key and reading values until a new one is encountered. In this case, we want to capture the minimum, maximum, and mean for each shape, so use numerous variables to track the needed data.

Remember that Streaming reducers need to handle a series of values grouped into their associated keys and must identify when a new line has a changed key, and hence indicates the last value for the previous key that has been processed. In contrast, a Java reducer would be simpler as it only deals with the values for a single key in each execution.

After making both files executable we run the job and get the following results, where we removed any shape with less than 10 sightings and again made the output more compact for space reasons. The numbers for each shape are the minimum value, the maximum value, and mean respectively:

```
changing0 5400 670 chevron0 3600 333
cigar0 5400 370 circle0 7200 423
cone0 4500 498 cross2 3600 460
cylinder0 5760 380 diamond0 7800 519
disk0 5400 449 egg0 5400 383
fireball0 5400 236 flash0 7200 303
formation0 5400 434 light0 9000 462
other0 5400 418 oval0 5400 405
rectangle0 4200 352 sphere0 14400 396
teardrop0 2700 335 triangle0 18000 375
unknown0 6000 470
```

It is surprising to see the relatively narrow variance in the mean sighting duration across all shape types; most have the mean value between 350 and 430 seconds. Interestingly, we also see that the shortest mean duration is for fireballs and the maximum for changeable objects, both of which make some degree of intuitive sense. A fireball by definition wouldn't be a long-lasting phenomena and a changeable object would need a lengthy duration for its changes to be noticed.

Using Streaming scripts outside Hadoop

This last example with its more involved mapper and reducer is a good illustration of how Streaming can help MapReduce development in another way; you can execute the scripts outside of Hadoop.

It's generally good practice during MapReduce development to have a sample of the production data against which to test your code. But when this is on HDFS and you are writing Java map and reduce tasks, it can be difficult to debug problems or refine complex logic. With map and reduce tasks that read input from the command line, you can directly run them against some data to get quick feedback on the result. If you have a development environment that provides Hadoop integration or are using Hadoop in standalone mode, the problems are minimized; just remember that Streaming does give you this ability to try the scripts outside of Hadoop; it may be useful some day.

While developing these scripts the author noticed that the last set of records in his UFO datafile had data in a better structured manner than those at the start of the file. Therefore, to do a quick test on the mapper all that was required was:

```
$ tail ufo.tsv | shapetimemapper.rb
```

This principle can be applied to the full workflow to exercise both the map and reduce script.

Time for action – performing the shape/time analysis from the command line

It may not be immediately obvious how to do this sort of local command-line analysis, so let's look at an example.

With the UFO datafile on the local filesystem, execute the following command:

```
$ cat ufo.tsv | shapetimemapper.rb | sort | shapetimereducer.rb
```

What just happened?

With a single Unix command line, we produced output identical to our previous full MapReduce job. If you look at what the command line does, this makes sense.

Firstly, the input file is sent—a line at a time—to the mapper. The output of this is passed through the Unix sort utility and this sorted output is passed a line at a time to the reducer. This is of course a very simplified representation of our general MapReduce job workflow.

Then the obvious question is why should we bother with Hadoop if we can do equivalent analysis at the command line. The answer of course is our old friend, scale. This simple approach works fine for a file such as the UFO sightings, which though non-trivial, is only 71MB in size. To put this into context we could hold thousands of copies of this dataset on a single modern disk drive.

So what if the dataset was 71GB in size instead, or even 71TB? In the latter case, at least we would have to spread the data across multiple hosts, and then decide how to split the data, combine partial answers, and deal with the inevitable failures along the way. In other words, we would need something like Hadoop.

However, don't discount the use of command-line tools like this, such approaches should be well used during MapReduce development.

Java shape and location analysis

Let's return to the Java MapReduce API and consider some analysis of the shape and location data within the reports.

However, before we start writing code, let's think about how we've been approaching the per-field analysis of this dataset. The previous mappers have had a common pattern:

- ◆ Discard records determined to be corrupt
- ◆ Process valid records to extract the field of interest
- ◆ Output a representation of the data we care about for the record

Now if we were to write Java mappers to analyze location and then perhaps the sighting and reported time columns, we would follow a similar pattern. So can we avoid any of the consequent code duplication?

The answer is yes, through the use of `org.apache.hadoop.mapred.lib.ChainMapper`. This class provides a means by which multiple mappers are executed in sequence and it is the output of the final mapper that is passed to the reducer. `ChainMapper` is applicable not just for this type of data clean-up; when analyzing particular jobs, it is not an uncommon pattern that is useful to perform multiple map-type tasks before applying a reducer.

An example of this approach would be to write a validation mapper that could be used by all future field analysis jobs. This mapper would discard lines deemed corrupt, passing only valid lines to the actual business logic mapper that can now be focused on analyzing data instead of worrying about coarse-level validation.

An alternative approach here would be to do the validation within a custom `InputFormat` class that discards non-valid records; which approach makes the most sense will depend on your particular situation.

Each mapper in the chain is executed within a single JVM so there is no need to worry about the use of multiple mappers increasing our filesystem I/O load.

Time for action – using ChainMapper for field validation/analysis

Let's use this principle and employ the `ChainMapper` class to help us provide some record validation within our job:

1. Create the following class as `UFORecordValidationMapper.java`:

```
import java.io.IOException;

import org.apache.hadoop.io.* ;
import org.apache.hadoop.mapred.* ;
import org.apache.hadoop.mapred.lib.* ;

public class UFORecordValidationMapper extends MapReduceBase
implements Mapper<LongWritable, Text, LongWritable, Text>
{

    public void map(LongWritable key, Text value,
        OutputCollector<LongWritable, Text> output,
        Reporter reporter) throws IOException
    {
        String line = value.toString();
```

```

        if (validate(line))
            output.collect(key, value);
    }

    private boolean validate(String str)
    {
        String[] parts = str.split("\t") ;

        if (parts.length != 6)
            return false ;

        return true ;
    }
}

```

2. Create the following as UFOLocation.java:

```

import java.io.IOException;
import java.util.Iterator ;
import java.util.regex.* ;

import org.apache.hadoop.conf.* ;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.* ;
import org.apache.hadoop.mapred.* ;
import org.apache.hadoop.mapred.lib.* ;

public class UFOLocation
{

    public static class MapClass extends MapReduceBase
    implements Mapper<LongWritable, Text, Text, LongWritable>
    {

        private final static LongWritable one = new LongWritable(1);
        private static Pattern locationPattern = Pattern.compile(
            "[a-zA-Z]{2}[^a-zA-Z]*$" ) ;

        public void map(LongWritable key, Text value,
            OutputCollector<Text, LongWritable> output,
            Reporter reporter) throws IOException
        {
            String line = value.toString();
            String[] fields = line.split("\t") ;
            String location = fields[2].trim() ;

```

```
        if (location.length() >= 2)
        {

            Matcher matcher = locationPattern.matcher(location) ;
            if (matcher.find() )
            {
                int start = matcher.start() ;
                String state = location.substring(start,start+2);

                output.collect(new Text(state.toUpperCase()),
                               One);
            }
        }
    }
}

public static void main(String[] args) throws Exception
{
    Configuration config = new Configuration() ;
    JobConf conf = new JobConf(config, UFOLocation.class);
    conf.setJobName("UFOLocation");

    conf.setOutputKeyClass(Text.class);
    conf.setOutputValueClass(LongWritable.class);

    JobConf mapconf1 = new JobConf(false) ;
    ChainMapper.addMapper( conf, UFORecordValidationMapper.class,
        LongWritable.class, Text.class, LongWritable.class,
        Text.class, true, mapconf1) ;

    JobConf mapconf2 = new JobConf(false) ;
    ChainMapper.addMapper( conf, MapClass.class,
        LongWritable.class, Text.class,
        Text.class, LongWritable.class, true, mapconf2) ;
    conf.setMapperClass(ChainMapper.class);
    conf.setCombinerClass(LongSumReducer.class);
    conf.setReducerClass(LongSumReducer.class);

    FileInputFormat.setInputPaths(conf,args[0]) ;
    FileOutputFormat.setOutputPath(conf, new Path(args[1])) ;

    JobClient.runJob(conf);
}
}
```

3. Compile both files:

```
$ javac UFORecordValidationMapper.java UFOLocation.java
```

4. Jar up the class files and submit the job to Hadoop:

```
$ Hadoop jar ufo.jar UFOLocation ufo.tsv output
```

5. Copy the output file to the local filesystem and examine it:

```
$ Hadoop fs -get output/part-00000 locations.txt  
$ more locations.txt
```

What just happened?

There's quite a bit happening here, so let's look at it one piece at a time.

The first mapper is our simple validation mapper. The class follows the same interface as the standard MapReduce API and the `map` method simply returns the result of a utility validation method. We split this out into a separate method to highlight the functionality of the mapper, but the checks could easily have been within the main `map` method itself. For simplicity, we keep to our previous validation strategy of looking for the number of fields and discarding lines that don't break into exactly six tab-delimited fields.

Note that the `ChainMapper` class has unfortunately been one of the last components to be migrated to the context object API and as of Hadoop 1.0, it can only be used with the older API. It remains a valid concept and useful tool but until Hadoop 2.0, where it will finally be migrated into the `org.apache.hadoop.mapreduce.lib.chain` package, its current use requires the older approach.

The other file contains another mapper implementation and an updated driver in the main method. The mapper looks for a two-letter sequence at the end of the location field in a UFO sighting report. From some manual examination of data, it is obvious that most location fields are of the form `city, state`, where the standard two-character abbreviation is used for the state.

Some records, however, add trailing parenthesis, periods, or other punctuation. Some others are simply not in this format. For our purposes, we are happy to discard those records and focus on those that have the trailing two-character state abbreviation we are looking for.

The `map` method extracts this from the location field using another regular expression and gives the output as the capitalized form of the abbreviation along with a simple count.

The driver for the job has the most changes as the previous configuration involving a single `map` class is replaced with multiple calls on the `ChainMapper` class.

The general model is to create a new configuration object for each mapper, then add the mapper to the `ChainMapper` class along with a specification of its input and output, and a reference to the overall job configuration object.

Notice that the two mappers have different signatures. Both input a key of type `LongWritable` and value of type `Text` which are also the output types of `UFORecordValidationMapper`. `UFOLocationMapper` however outputs the reverse with a key of type `Text` and a value of type `LongWritable`.

The important thing here is to match the input from the final mapper in the chain (`UFOLocationMapper`) with the inputs expected by the `reduce` class (`LongSumReducer`). When using the `ChainMapper` class the mappers in the chain can have different input and output as long as the following are true:

- ◆ For all but the final mapper each map output matches the input of the subsequent mapper in the chain
- ◆ For the final mapper, its output matches the input of the reducer

We compile these classes and put them in the same jar file. This is the first time we have bundled the output from more than one Java source file together. As may be expected, there is no magic here; the usual rules on jar files, path, and class names apply. Because in this case we have both our classes in the same package, we don't have to worry about an additional import in the driver class file.

We then run the MapReduce job and examine the output, which is not quite as expected.

Have a go hero

Use the Java API and the previous `ChainMapper` example to reimplement the mappers previously written in Ruby that produce the shape frequency and duration reports.

Too many abbreviations

The following are the first few entries from our result file of the previous job:

AB	286
AD	6
AE	7
AI	6
AK	234
AL	548
AM	22
AN	161
...	

The file had 186 different two-character entries. Plainly, our approach of extracting the final character digraph from the location field was not sufficiently robust.

We have a number of issues with the data which becomes apparent after a manual analysis of the source file:

- ◆ There is inconsistency in the capitalization of the state abbreviations
- ◆ A non-trivial number of sightings are from outside the U.S. and though they may follow a similar (`city`, `area`) pattern, the abbreviation is not one of the 50 we'd expect
- ◆ Some fields simply don't follow the pattern at all, yet would still be captured by our regular expression

We need to filter these results, ideally by normalizing the U.S. records into correct state output and by gathering everything else into a broader category.

To perform this task we need to add to the mapper some notion of what the valid U.S. state abbreviations are. We could of course hardcode this into the mapper but that does not seem right. Although we are for now going to treat all non-U.S. sightings as a single category, we may wish to extend that over time and perhaps do a breakdown by country. If we hardcode the abbreviations, we would need to recompile our mapper each time.

Using the Distributed Cache

Hadoop gives us an alternative mechanism to achieve the goal of sharing reference data across all tasks in the job, the Distributed Cache. This can be used to efficiently make available common read-only files that are used by the map or reduce tasks to all nodes. The files can be text data as in this case but could also be additional jars, binary data, or archives; anything is possible.

The files to be distributed are placed on HDFS and added to the DistributedCache within the job driver. Hadoop copies the files onto the local filesystem of each node prior to job execution, meaning every task has local access to the files.

An alternative is to bundle needed files into the job jar submitted to Hadoop. This does tie the data to the job jar making it more difficult to share across jobs and requires the jar to be rebuilt if the data changes.

Time for action – using the Distributed Cache to improve location output

Let's now use the Distributed Cache to share a list of U.S. state names and abbreviations across the cluster:

1. Create a datafile called `states.txt` on the local filesystem. It should have the state abbreviation and full name tab separated, one per line. Or retrieve the file from this book's homepage. The file should start like the following:

```
AL      Alabama
AK      Alaska
AZ      Arizona
AR      Arkansas
CA      California
```

...

2. Place the file on HDFS:

```
$ hadoop fs -put states.txt states.txt
```
3. Copy the previous `UFOLocation.java` file to `UFOLocation2.java` file and make the changes by adding the following import statements:

```
import java.io.* ;
import java.net.* ;
import java.util.* ;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.filecache.DistributedCache ;
```

4. Add the following line to the driver main method after the job name is set:

```
DistributedCache.addCacheFile(new URI ("/user/hadoop/states.txt"),
conf) ;
```

5. Replace the map class as follows:

```
public static class MapClass extends MapReduceBase
implements Mapper<LongWritable, Text, Text, LongWritable>
{

    private final static LongWritable one = new
LongWritable(1);
    private static Pattern locationPattern = Pattern.compile(
"[a-zA-Z]{2}[^a-zA-Z]*$") ;
    private Map<String, String> stateNames ;

    @Override
```

```

        public void configure( JobConf job)
        {
            try
            {
                Path[] cacheFiles = DistributedCache.
getLocalCacheFiles(job) ;
                setupStateMap( cacheFiles[0].toString()) ;
            } catch (IOException e)
            {
                System.err.println("Error reading state file.") ;
                System.exit(1) ;
            }
        }

        private void setupStateMap(String filename)
        throws IOException
        {
            Map<String, String> states = new HashMap<String,
String>() ;
            BufferedReader reader = new BufferedReader( new
FileReader(filename)) ;
            String line = reader.readLine() ;
            while (line != null)
            {
                String[] split = line.split("\t") ;
                states.put(split[0], split[1]) ;
                line = reader.readLine() ;
            }

            stateNames = states ;
        }

        public void map(LongWritable key, Text value,
            OutputCollector<Text, LongWritable> output,
            Reporter reporter) throws IOException
        {
            String line = value.toString();
            String[] fields = line.split("\t") ;
            String location = fields[2].trim() ;
            if (location.length() >= 2)
            {
                Matcher matcher = locationPattern.matcher(location) ;
                if (matcher.find() )

```

```
        {
            int start = matcher.start() ;
            String state = location.substring(start, start+2)
        }
;

        output.collect(newText(lookupState(state.
toUpperCase()), one);
    }
}

private String lookupState( String state)
{
    String fullName = stateNames.get(state) ;

    return fullName == null? "Other": fullName ;
}
}
```

6. Compile these classes and submit the job to Hadoop. Then retrieve the result file.

What just happened?

We first created the lookup file we will use in our job and placed it on HDFS. Files to be added to the Distributed Cache must initially be copied onto the HDFS filesystem.

After creating our new job file, we added the required class imports. Then we modified the driver class to add the file we want on each node to be added to the DistributedCache. The filename can be specified in multiple ways, but the easiest way is with an absolute path to the file location on HDFS.

There were a number of changes to our mapper class. We added an overridden `configure` method, which we use to populate a map that will be used to associate state abbreviations with their full name.

The `configure` method is called on task startup and the default implementation does nothing. In our overridden version, we retrieve the array of files that have been added to the Distributed Cache. As we know there is only one file in the cache we feel safe in using the first index in this array, and pass that to a `utility` method that parses the file and uses the contents to populate the state abbreviation lookup map. Notice that once the file reference is retrieved, we can access the file with standard Java I/O classes; it is after all just a file on the local filesystem.

We add another method to perform the lookup that takes the string extracted from the location field and returns either the full name of the state if there is a match or the string `Other` otherwise. This is called prior to the map result being written via the `OutputCollector` class.

The result of this job should be similar to the following data:

```
Alabama    548
Alaska     234
Arizona    2097
Arkansas   534
California 7679
...
Other      4531...
...
```

This works fine but we have been losing some information along the way. In our validation mapper, we simply drop any lines which don't meet our six field criteria. Though we don't care about individual lost records, we may care if the number of dropped records is very large. Currently, our only way of determining that is to sum the number of records for each recognized state and subtract from the total number of records in the file. We could also try to have this data flow through the rest of the job to be gathered in a special reduced key but that also seems wrong. Fortunately, there is a better way.

Counters, status, and other output

At the end of every MapReduce job, we see output related to counters such as the following output:

```
12/02/12 06:28:51 INFO mapred.JobClient: Counters: 22
12/02/12 06:28:51 INFO mapred.JobClient:   Job Counters
12/02/12 06:28:51 INFO mapred.JobClient:     Launched reduce tasks=1
12/02/12 06:28:51 INFO mapred.JobClient:     Launched map tasks=18
12/02/12 06:28:51 INFO mapred.JobClient:     Data-local map tasks=18
12/02/12 06:28:51 INFO mapred.JobClient:     SkippingTaskCounters
12/02/12 06:28:51 INFO mapred.JobClient:     MapProcessedRecords=61393
...
```

It is possible to add user-defined counters that will likewise be aggregated from all tasks and reported in this final output as well as in the MapReduce web UI.

Time for action – creating counters, task states, and writing log output

We'll modify our UFORecordValidationMapper to report statistics about skipped records and also highlight some other facilities for recording information about a job:

1. Create the following as the UFOCountingRecordValidationMapper.java file:

```
import java.io.IOException;

import org.apache.hadoop.io.* ;
import org.apache.hadoop.mapred.* ;
import org.apache.hadoop.mapred.lib.* ;

public class UFOCountingRecordValidationMapper extends
MapReduceBase
implements Mapper<LongWritable, Text, LongWritable, Text>
{

    public enum LineCounters
    {
        BAD_LINES,
        TOO_MANY_TABS,
        TOO_FEW_TABS
    } ;

    public void map(LongWritable key, Text value,
        OutputCollector<LongWritable, Text> output,
        Reporter reporter) throws IOException
    {
        String line = value.toString();

        if (validate(line, reporter))
        Output.collect(key, value);
    }

    private boolean validate(String str, Reporter reporter)
    {
        String[] parts = str.split("\t") ;

        if (parts.length != 6)
        {
            if (parts.length < 6)
            {
```

```

reporter.incrCounter(LineCounters.TOO_FEW_TABS, 1) ;
    }
    else
    {
        reporter.incrCounter(LineCounters.TOO_MANY_TABS,
1) ;
    }

    reporter.incrCounter(LineCounters.BAD_LINES, 1) ;

    if((reporter.getCounter(
LineCounters.BAD_LINES).getCounter()%10)
== 0)
    {
        reporter.setStatus("Got 10 bad lines.") ;
        System.err.println("Read another 10 bad lines.") ;
    }

    return false ;
}
return true ;
}
}

```

2. Make a copy of the `UFOLocation2.java` file as the `UFOLocation3.java` file to use this new mapper instead of `UFORecordValidationMapper`:

```

...
    JobConf mapconf1 = new JobConf(false) ;
    ChainMapper.addMapper( conf,
UFOCountingRecordValidationMapper.class,
        LongWritable.class, Text.class, LongWritable.class,
Text.class,
        true, mapconf1) ;

```

3. Compile the files, jar them up, and submit the job to Hadoop:

```

...
12/02/12 06:28:51 INFO mapred.JobClient: Counters: 22
12/02/12 06:28:51 INFO mapred.JobClient:   UFOCountingRecordValida
tionMapper$LineCounters
12/02/12 06:28:51 INFO mapred.JobClient:       TOO_MANY_TABS=324
12/02/12 06:28:51 INFO mapred.JobClient:       BAD_LINES=326
12/02/12 06:28:51 INFO mapred.JobClient:       TOO_FEW_TABS=2
12/02/12 06:28:51 INFO mapred.JobClient:   Job Counters

```

4. Use a web browser to go to the MapReduce web UI (remember by default it is on port 50030 on the JobTracker host). Select the job at the bottom of the **Completed Jobs** list and you should see a screen similar to the following screenshot:

Hadoop job_201202111619_0023 on head

User: hadoop
 Job Name: UFOLocation
 Job File: hdfs://head:9000/var/hadoop/mapred/system/job_201202111619_0023/job.xml
 Job Setup: [Successful](#)
 Status: Succeeded
 Started at: Sun Feb 12 15:00:35 PST 2012
 Finished at: Sun Feb 12 15:01:55 PST 2012
 Finished in: 1mins, 19sec
 Job Cleanup: [Successful](#)

Kind	% Complete	Num Tasks	Pending	Running	Complete	Killed	Failed/Killed Task Attempts
map	100.00%	18	0	0	18	0	0 / 2
reduce	100.00%	1	0	0	1	0	0 / 0

	Counter	Map	Reduce	Total
UFOLocationRecordValidationMapper\$LineCounters	TOO_MANY_TABS	324	0	324
	TOO_FEW_TABS	2	0	2
	BAD_LINES	326	0	326
	Launched reduce tasks	0	0	1

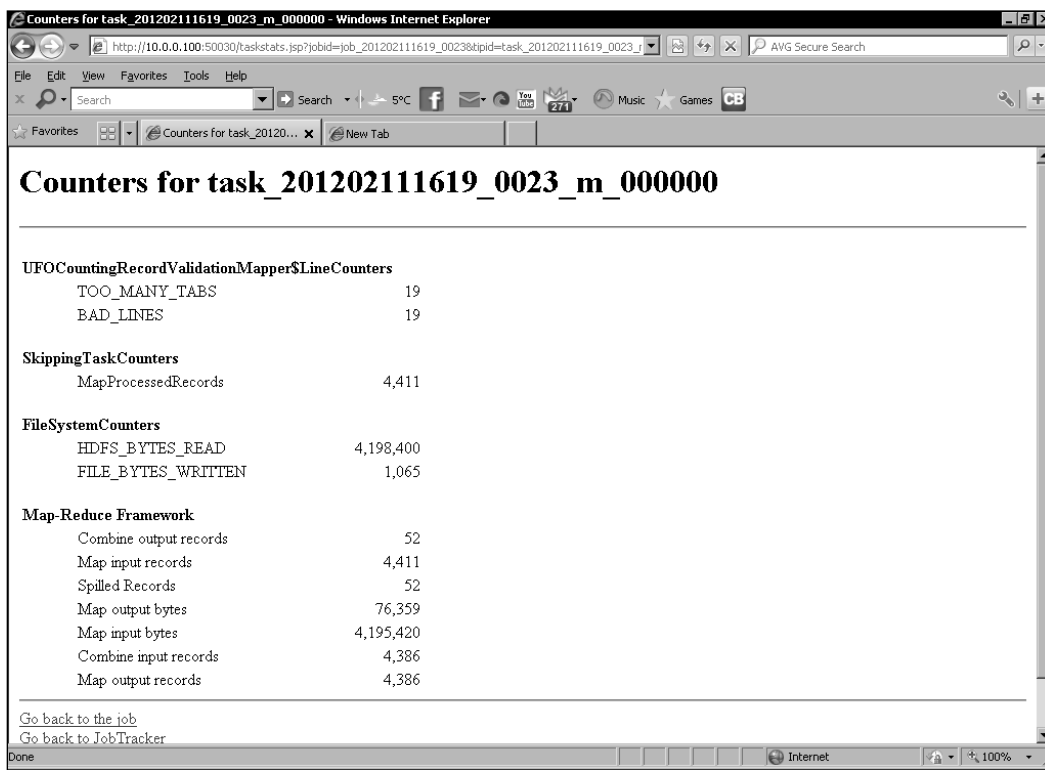
5. Click on the link to the map tasks and you should see an overview screen like the following screenshot:

Hadoop map task list for job_201202111619_0023 on head

Completed Tasks

Task	Complete	Status	Start Time	Finish Time	Errors
task_201202111619_0023_m_000000	100.00%	Got 10 bad lines.	12-Feb-2012 15:00:40	12-Feb-2012 15:00:49 (9sec)	
task_201202111619_0023_m_000001	100.00%	Got 10 bad lines.	12-Feb-2012 15:00:40	12-Feb-2012 15:00:49 (9sec)	
task_201202111619_0023_m_000002	100.00%	Got 10 bad lines.	12-Feb-2012 15:00:41	12-Feb-2012 15:01:26 (44sec)	
task_201202111619_0023_m_000003	100.00%	hdfs://head:9000/user/hadoop/ufo.tsv:12582912+4194304	12-Feb-2012 15:00:41	12-Feb-2012 15:01:26 (44sec)	

6. For one of the tasks with our custom status message, click on the link to its counters. This should give a screen similar to the one shown as follows:



7. Go back to the task list and click on the task ID to get the task overview similar to the following screenshot:

Job job_201202111619_0023

All Task Attempts

Task Attempts	Machine	Status	Progress	Start Time	Finish Time	Errors	Task Logs	Counters	Acti
attempt_201202111619_0023_m_000000_0	/default-rack/node1	SUCCEEDED	100.00%	12-Feb-2012 15:00:40	12-Feb-2012 15:00:46 (6sec)		Last 4kB Last 3kB All	12	

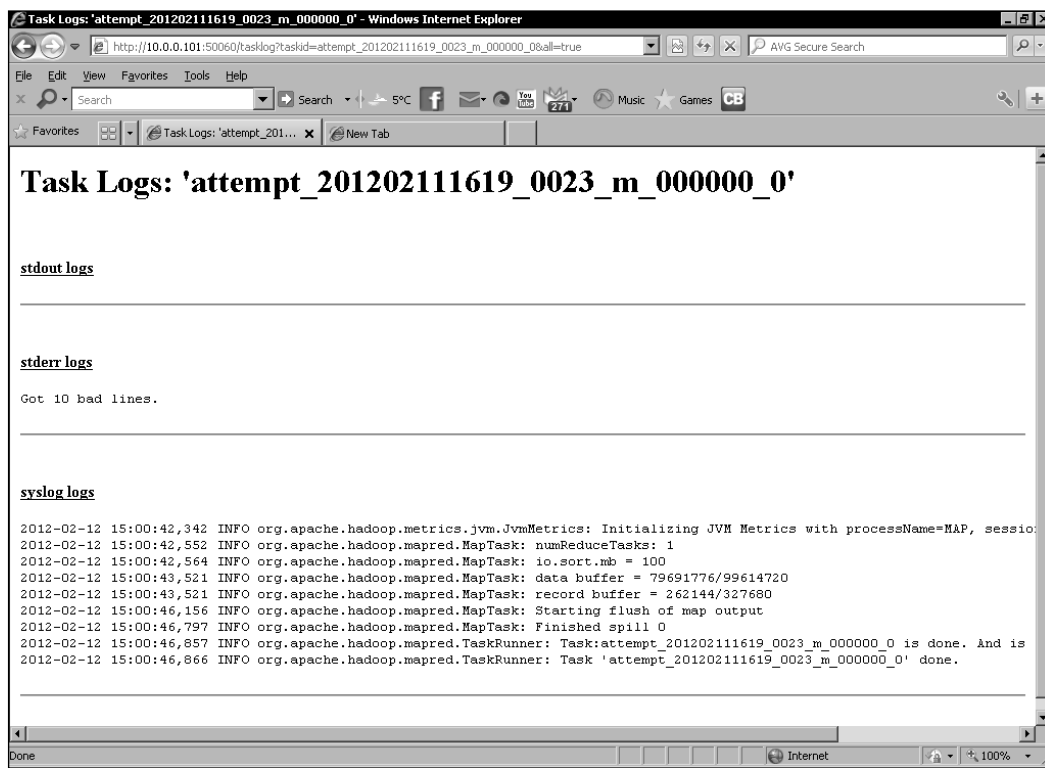
Input Split Locations

- [/default-rack/node3](#)
- [/default-rack/node1](#)
- [/default-rack/node4](#)

[Go back to the job](#)
[Go back to JobTracker](#)

Hadoop, 2012.

- Under the **Task Logs** column are options for the amount of data to be displayed. Click on **All** and the following screenshot should be displayed:



- Now log into one of the task nodes and look through the files stored under `hadoop/logs/userlogs`. There is a directory for each task attempt and several files within each; the one to look for is `stderr`.

What just happened?

The first thing we need to do in order to add new counters is to create a standard Java enumeration that will hold them. In this case we created what Hadoop would consider a counter group called **LineCounters** and within that there are three counters for the total number of bad lines, and finer grained counters for the number of lines with either too few or too many fields. This is all you need to do to create a new set of counters; define the enumeration and once you start setting the counter values, they will be automatically understood by the framework.

To add to a counter we simply increment it via the `Reporter` object, in each case here we add one each time we encounter a bad line, one with fewer than six fields, and one with more than six fields.

We also retrieve the `BAD_LINE` counter for a task and if it is a multiple of 10, do the following:

- ◆ Set the task status to reflect this fact
- ◆ Write a similar message to `stderr` with the standard Java `System.err.println` mechanism

We then go to the MapReduce UI and validate whether we can see both the counter totals in the job overview as well as tasks with the custom state message in the task list.

We then explored the web UI, looking at the counters for an individual job, then under the detail page for a task we saw, we can click on through the log files for the task.

We then looked at one of the nodes to see that Hadoop also captures the logs from each task in a directory on the filesystem under the `{HADOOP_HOME}/logs/userlogs` directory. Under subdirectories for each task attempt, there are files for the standard streams as well as the general task logs. As you will see, a busy node can end up with a large number of task log directories and it is not always easy to identify the task directories of interest. The web interface proved itself to be a more efficient view on this data.



If you are using the Hadoop context object API, then counters are accessed through the `Context.getCounter().increment()` method.

Too much information!

After not worrying much about how to get status and other information out of our jobs, it may suddenly seem like we've got too many confusing options. The fact of the matter is that when running a fully distributed cluster in particular, there really is no way around the fact that the data may be spread across every node. With Java code we can't as easily mock its usage on the command line as we did with our Ruby Streaming tasks; so care needs to be taken to think about what information will be needed at runtime. This should include details concerning both the general job operation (additional statistics) as well as indicators of problems that may need further investigation.

Counters, task status messages, and good old-fashioned Java logging can work together. If there is a situation you care about, set it as a counter that will record each time it occurs and consider setting the status message of the task that encountered it. If there is some specific data, write that to `stderr`. Since counters are so easily visible, you can know pretty quickly post job completion if the situation of interest occurred. From this, you can go to the web UI and see all the tasks in which the situation was encountered at a glance. From here, you can click through to examine the more detailed logs for the task.

In fact, you don't need to wait until the job completes; counters and task status messages are updated in the web UI as the job proceeds, so you can start the investigation as soon as either counters or task status messages alert you to the situation. This is particularly useful in very long running jobs where the errors may cause you to abort the job.

Summary

This chapter covered development of a MapReduce job, highlighting some of the issues and approaches you are likely to face frequently. In particular, we learned how Hadoop Streaming provides a means to use scripting languages to write map and reduce tasks, and how using Streaming can be an effective tool for early stages of job prototyping and initial data analysis.

We also learned that writing tasks in a scripting language can provide the additional benefit of using command-line tools to directly test and debug the code. Within the Java API, we looked at the `ChainMapper` class that provides an efficient way of decomposing a complex map task into a series of smaller, more focused ones.

We then saw how the Distributed Cache provides a mechanism for efficient sharing of data across all nodes. It copies files from HDFS onto the local filesystem on each node, providing local access to the data. We also learned how to add job counters by defining a Java enumeration for the counter group and using framework methods to increment their values, and how to use a combination of counters, task status messages, and debug logs to develop an efficient job analysis workflow.

We expect most of these techniques and ideas to be the ones that you will encounter frequently as you develop MapReduce jobs. In the next chapter, we will explore a series of more advanced techniques that are less often encountered but are invaluable when they are.

Where to buy this book

You can buy Hadoop Beginner's Guide from the Packt Publishing website:

<http://www.packtpub.com/hadoop-beginners-guide/book>.

Free shipping to the US, UK, Europe and selected Asian countries. For more information, please read our [shipping policy](#).

Alternatively, you can buy the book from Amazon, BN.com, Computer Manuals and most internet book retailers.



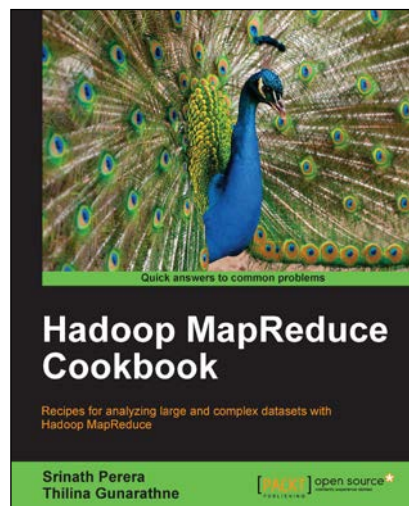
www.PacktPub.com

For More Information:

www.packtpub.com/hadoop-beginners-guide/book

Hadoop MapReduce Cookbook

Srinath Perera
Thilina Gunarathne



Chapter No. 6

"Analytics"

In this package, you will find:

A Biography of the authors of the book

A preview chapter from the book, Chapter NO.6 "Analytics"

A synopsis of the book's content

Information on where to buy this book

About the Authors

Srinath Perera is a Senior Software Architect at WSO2 Inc., where he overlooks the overall WSO2 platform architecture with the CTO. He also serves as a Research Scientist at Lanka Software Foundation and teaches as a visiting faculty at Department of Computer Science and Engineering, University of Moratuwa. He is a co-founder of Apache Axis2 open source project, and he has been involved with the Apache Web Service project since 2002, and is a member of Apache Software foundation and Apache Web Service project PMC. Srinath is also a committer of Apache open source projects Axis, Axis2, and Geronimo.

He received his Ph.D. and M.Sc. in Computer Sciences from Indiana University, Bloomington, USA and received his Bachelor of Science in Computer Science and Engineering from University of Moratuwa, Sri Lanka.

Srinath has authored many technical and peer reviewed research articles, and more detail can be found from his website. He is also a frequent speaker at technical venues.

He has worked with large-scale distributed systems for a long time. He closely works with Big Data technologies, such as Hadoop and Cassandra daily. He also teaches a parallel programming graduate class at University of Moratuwa, which is primarily based on Hadoop.

For More Information:

www.packtpub.com/hadoop-mapreduce-cookbook/book

I would like to thank my wife Miyuru and my parents, whose never-ending support keeps me going. I also like to thanks Sanjiva from WSO2 who encourage us to make our mark even though project like these are not in the job description. Finally I would like to thank my colleges at WSO2 for ideas and companionship that have shaped the book in many ways.

Thilina Gunarathne is a Ph.D. candidate at the School of Informatics and Computing of Indiana University. He has extensive experience in using Apache Hadoop and related technologies for large-scale data intensive computations. His current work focuses on developing technologies to perform scalable and efficient large-scale data intensive computations on cloud environments.

Thilina has published many articles and peer reviewed research papers in the areas of distributed and parallel computing, including several papers on extending MapReduce model to perform efficient data mining and data analytics computations on clouds. Thilina is a regular presenter in both academic as well as industry settings.

Thilina has contributed to several open source projects at Apache Software Foundation as a committer and a PMC member since 2005. Before starting the graduate studies, Thilina worked as a Senior Software Engineer at WSO2 Inc., focusing on open source middleware development. Thilina received his B.Sc. in Computer Science and Engineering from University of Moratuwa, Sri Lanka, in 2006 and received his M.Sc. in Computer Science from Indiana University, Bloomington, in 2009. Thilina expects to receive his doctorate in the field of distributed and parallel computing in 2013.

For More Information:

www.packtpub.com/hadoop-mapreduce-cookbook/book

This book would not have been a success without the direct and indirect help from many people. Thanks to my wife and my son for putting up with me for all the missing family times and for providing me with love and encouragement throughout the writing period. Thanks to my parents, without whose love, guidance and encouragement, I would not be where I am today.

Thanks to my advisor Prof. Geoffrey Fox for his excellent guidance and providing me with the environment to work on Hadoop and related technologies. Thanks to the HBase, Mahout, Pig, Hive, Nutch, and Lucene communities for developing great open source products. Thanks to Apache Software Foundation for fostering vibrant open source communities.

Thanks to the editorial staff at Packt, for providing me the opportunity to write this book and for providing feedback and guidance throughout the process. Thanks to the reviewers for reviewing this book, catching my mistakes, and for the many useful suggestions.

Thanks to all of my past and present mentors and teachers, including Dr. Sanjiva Weerawarana of WSO2, Prof. Dennis Gannon, Prof. Judy Qiu, Prof. Beth Plale, all my professors at Indiana University and University of Moratuwa for all the knowledge and guidance they gave me. Thanks to all my past and present colleagues for many insightful discussions and the knowledge they shared with me.

For More Information:

www.packtpub.com/hadoop-mapreduce-cookbook/book

Hadoop MapReduce Cookbook

Hadoop MapReduce Cookbook helps readers learn to process large and complex datasets. The book starts in a simple manner, but still provides in-depth knowledge of Hadoop. It is a simple one-stop guide on how to get things done. It has 90 recipes, presented in a simple and straightforward manner, with step-by-step instructions and real world examples.

This product includes software developed at The Apache Software Foundation (<http://www.apache.org/>).

What This Book Covers

Chapter 1, Getting Hadoop Up and Running in a Cluster, explains how to install and run Hadoop both as a single node as well as a cluster.

Chapter 2, Advanced HDFS, introduces a set of advanced HDFS operations that would be useful when performing large-scale data processing with Hadoop MapReduce as well as with non-MapReduce use cases.

Chapter 3, Advanced Hadoop MapReduce Administration, explains how to change configurations and security of a Hadoop installation and how to debug.

Chapter 4, Developing Complex Hadoop MapReduce Applications, introduces you to several advanced Hadoop MapReduce features that will help you to develop highly customized, efficient MapReduce applications.

Chapter 5, Hadoop Ecosystem, introduces the other projects related to Hadoop such as HBase, Hive, and Pig.

Chapter 6, Analytics, explains how to calculate basic analytics using Hadoop.

Chapter 7, Searching and Indexing, introduces you to several tools and techniques that you can use with Apache Hadoop to perform large-scale searching and indexing.

For More Information:

www.packtpub.com/hadoop-mapreduce-cookbook/book

Chapter 8, Classifications, Recommendations, and Finding Relationships, explains how to implement complex algorithms such as classifications, recommendations, and finding relationships using Hadoop.

Chapter 9, Mass Text Data Processing, explains how to use Hadoop and Mahout to process large text datasets, and how to perform data preprocessing and loading operations using Hadoop.

Chapter 10, Cloud Deployments: Using Hadoop on Clouds, explains how to use Amazon Elastic MapReduce (EMR) and Apache Whirr to deploy and execute Hadoop MapReduce, Pig, Hive, and HBase computations on cloud infrastructures.

For More Information:

www.packtpub.com/hadoop-mapreduce-cookbook/book

6

Analytics

In this chapter, we will cover:

- ▶ Simple analytics using MapReduce
- ▶ Performing Group-By using MapReduce
- ▶ Calculating frequency distributions and sorting using MapReduce
- ▶ Plotting the Hadoop results using GNU Plot
- ▶ Calculating histograms using MapReduce
- ▶ Calculating scatter Plots using MapReduce
- ▶ Parsing a complex dataset with Hadoop
- ▶ Joining two datasets using MapReduce

Introduction

This chapter discusses how we can process a dataset and understand its basic characteristics. We will cover more complex methods like data mining, classification, and so on, in later chapters.

Following are a few instances of basic analytics:

- ▶ Calculating Minimum, Maximum, Mean, Median, Standard deviation, and so on of a dataset. Given a dataset, generally there are multiple dimensions (for example, while processing HTTP access logs, names of the web page, the size of the web page, access time, and so on). We can measure the mentioned analytics using one or more dimensions. For example, we can group the data into multiple groups and calculate the mean value in each case.
- ▶ Histograms used in finding out how many occurrences happen within different value ranges (for example, how many hits happen within each 6-hour period).

For More Information:

www.packtpub.com/hadoop-mapreduce-cookbook/book

- ▶ Frequency distributions used in finding out how many occurrences of a value happened (for example, how many hits were received by each web page in a site).
- ▶ Finding a correlation between two dimensions (for example, correlation between access count and the file size of web accesses).
- ▶ Hypothesis testing, that is, trying to verify or disprove a hypothesis using a given dataset.

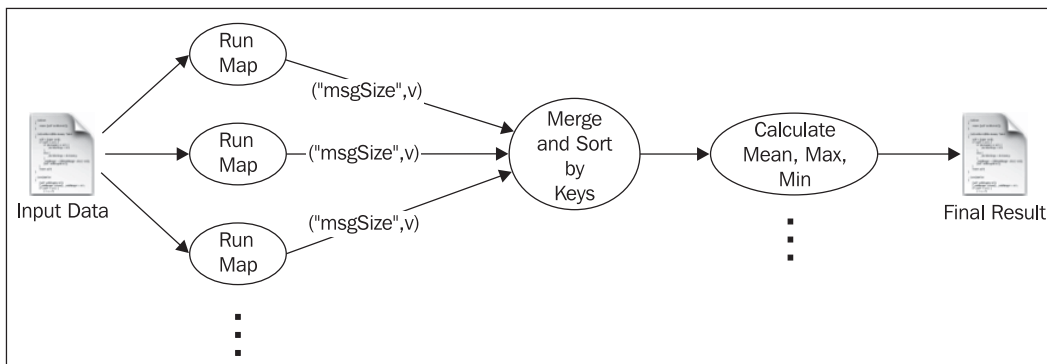
This chapter will show how you can calculate basic analytics using a given dataset. For recipes in this chapter, we will use two datasets:

- ▶ NASA weblog dataset available from <http://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html> is a real-life dataset collected using the requests received by NASA web servers.
- ▶ Apache tomcat developer list e-mail archives available from http://mail-archives.apache.org/mod_mbox/tomcat-users/, which is in MBOX format.

Simple analytics using MapReduce

Aggregative values (for example, Mean, Max, Min, standard deviation, and so on) provide the basic analytics about a dataset. You may perform these calculations, either for the whole dataset or a part of the dataset.

In this recipe, we will use Hadoop to calculate the minimum, maximum, and average size of a file downloaded from the NASA servers, by processing the NASA weblog dataset. The following figure shows a summary of the execution:



As shown in the figure, mapper task will emit all message sizes under the key **msgSize**, and they are all sent to a one-reducer job. Then the reducer will walk through all of the data and will calculate the aggregate values.

Getting ready

- ▶ This recipe assumes that you have followed the first chapter and have installed Hadoop. We will use `HADOOP_HOME` to refer to the Hadoop installation folder.
- ▶ Start Hadoop by following the instructions in the first chapter.
- ▶ This recipe assumes that you are aware of how Hadoop processing works. If you have not already done so, you should follow the recipe *Writing a WordCount MapReduce sample, bundling it and running it using standalone Hadoop* from Chapter 1, *Getting Hadoop Up and Running in a Cluster*.

How to do it...

The following steps describe how to use MapReduce to calculate simple analytics about the weblog dataset:

1. Download the weblog dataset from `ftp://ita.ee.lbl.gov/traces/NASA_access_log_Jul95.gz` and unzip it. We call the extracted folder as `DATA_DIR`.
2. Upload the data to HDFS by running the following commands from `HADOOP_HOME`. If `/data` is already there, clean it up:


```
>bin/hadoopdfs -mkdir /data
> bin/hadoopdfs -mkdir /data/input1
> bin/hadoopdfs -put <DATA_DIR>/NASA_access_log_Jul95 /data/input1
```
3. Unzip the source code of this chapter (`chapter6.zip`). We will call that folder `CHAPTER_6_SRC`.
4. Change the `hadoop.home` property in the `CHAPTER_6_SRC/build.xml` file to point to your Hadoop installation folder.
5. Compile the source by running the `ant build` command from the `CHAPTER_6_SRC` folder.
6. Copy the `build/lib/hadoop-cookbook-chapter6.jar` to your `HADOOP_HOME`.
7. Run the MapReduce job through the following command from `HADOOP_HOME`:


```
>bin/hadoop jar hadoop-cookbook-chapter6.jar chapter6.
WebLogMessageSizeAggregator/data/input1 /data/output1
```
8. Read the results by running the following command:


```
$bin/hadoopdfs -cat /data/output1/*
```

You will see that it will print the results as following:

```
Mean      1150
Max       6823936
Min        0
```

How it works...

You can find the source for the recipe from `src/chapter6/WebLogMessageSizeAggregator.java`.

HTTP logs follow a standard pattern where each log looks like the following. Here the last token includes the size of the web page retrieved:

```
205.212.115.106 - - [01/Jul/1995:00:00:12 -0400] "GET /shuttle/countdown/countdown.html HTTP/1.0" 200 3985
```

We will use the Java regular expressions' support to parse the log lines, and the `Pattern.compile()` method in the top of the class defines the regular expression. Since most Hadoop jobs involve text processing, regular expressions are a very useful tool while writing Hadoop Jobs:

```
private final static IntWritable one = new IntWritable(1);
public void map(Object key, Text value,
    Context context) throws
    IOException, InterruptedException
{
    Matcher matcher = httplogPattern.matcher(value.
        toString());
    if (matcher.matches())
    {
        int size = Integer.parseInt(matcher.group(5));
        context.write(new Text("msgSize"), one);
    }
}
```

The map task receives each line in the log file as a different key-value pair. It parses the lines using regular expressions and emits the file size against the key `msgSize`.

Then, Hadoop collects all values for the key and invokes the reducer. Reducer walks through all the values and calculates the minimum, maximum, and mean file size of the file downloaded from the web server. It is worth noting that by making the values available as an iterator, Hadoop gives the programmer a chance to process the data without storing them in memory. You should therefore try to process values without storing them in memory whenever possible.

```
public static class AReducer
    extends Reducer<Text, IntWritable, Text, IntWritable>
{
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException
    {
        double tot = 0;
        int count = 0;
```

```

int min = Integer.MAX_VALUE;
int max = 0;
Iterator<IntWritable> iterator = values.iterator();
while (iterator.hasNext())
{
    int value = iterator.next().get();
    tot = tot + value;
    count++;
    if (value < min)
    {
        min = value;
    }
    if (value > max)
    {
        max = value;
    }
}
context.write(new Text("Mean"),
new IntWritable((int) tot / count));
context.write(new Text("Max"),
    new IntWritable(max));
context.write(new Text("Min"),
    new IntWritable(min));
}
}

```

The `main()` method of the job looks similar to the `WordCount` example, except for the highlighted lines that has been changed to accommodate the input and output datatype changes:

```

Job job = new Job(conf, "LogProcessingMessageSizeAggregation");
job.setJarByClass(WebLogMessageSizeAggregator.class);
job.setMapperClass(AMapper.class);
job.setReducerClass(AReducer.class);
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job, new Path(otherArgs[0]));
FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);

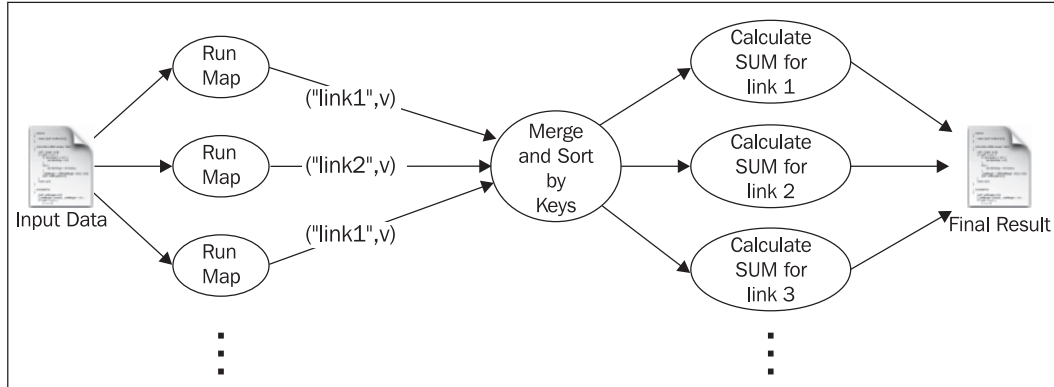
```

There's more...

You can learn more about Java regular expressions from the Java tutorial, <http://docs.oracle.com/javase/tutorial/essential/regex/>.

Performing Group-By using MapReduce

This recipe shows how we can use MapReduce to group data into simple groups and calculate the analytics for each group. We will use the same HTTP log dataset. The following figure shows a summary of the execution:



As shown in the figure, the mapper task groups the occurrence of each link under different keys. Then, Hadoop sorts the keys and provides all values for a given key to a reducer, who will count the number of occurrences.

Getting ready

- ▶ This recipe assumes that you have followed the first chapter and have installed Hadoop. We will use the `HADOOP_HOME` to refer to the Hadoop installation folder.
- ▶ Start Hadoop following the instructions in the first chapter.
- ▶ This recipe assumes that you are aware of how Hadoop processing works. If you have not already done so, you should follow the recipe *Writing a WordCount MapReduce sample, bundling it and running it using standalone Hadoop* from Chapter 1, *Getting Hadoop Up and Running in a Cluster*.

How to do it...

The following steps show how we can group weblog data and calculate analytics.

1. Download the weblog dataset from `ftp://ita.ee.lbl.gov/traces/NASA_access_log_Jul95.gz` and unzip it. We will call this `DATA_DIR`.
2. Upload the data to HDFS by running the following commands from `HADOOP_HOME`. If `/data` is already there, clean it up:


```
>bin/hadoopdfs -mkdir /data
>bin/hadoopdfs -mkdir /data/input1
>bin/hadoopdfs -put <DATA_DIR>/NASA_access_log_Jul95 /data/input1
```
3. Unzip the source code of this chapter (`chapter6.zip`). We will call that folder `CHAPTER_6_SRC`.
4. Change the `hadoop.home` property in the `CHAPTER_6_SRC/build.xml` file to point to your Hadoop installation folder.
5. Compile the source by running the `ant build` command from the `CHAPTER_6_SRC` folder.
6. Copy the `build/lib/hadoop-cookbook-chapter6.jar` to `HADOOP_HOME`.
7. Run the MapReduce job using the following command from `HADOOP_HOME`:


```
>bin/hadoop jar hadoop-cookbook-chapter6.jar chapter6.
WeblogHitsByLinkProcessor/data/input1 /data/output2
```
8. Read the results by running the following command:


```
>bin/hadoopdfs -cat /data/output2/*
```

You will see that it will print the results as following:

```
/base-ops/procurement/procurement.html 28
/biomed/ 1
/biomed/bibliography/biblio.html 7
/biomed/climate/airqual.html 4
/biomed/climate/climate.html 5
/biomed/climate/gif/f16pcfinmed.gif 4
/biomed/climate/gif/f22pcfinmed.gif 3
/biomed/climate/gif/f23pcfinmed.gif 3
/biomed/climate/gif/ozonehrlyfin.gif 3
```

How it works...

You can find the source for the recipe from `src/chapter6/WeblogHitsByLinkProcessor.java`.

As described in the earlier recipe, we will use regular expressions to parse HTTP logs. In the following sample the log line `/shuttle/countdown/countdown.html` shows the link (URL) being retrieved.

```
205.212.115.106 - - [01/Jul/1995:00:00:12 -0400] "GET /shuttle/countdown/
countdown.html HTTP/1.0" 200 3985
```

The following code segment shows the mapper:

```
public void map(Object key, Text value,
    Context context) throws IOException,
    InterruptedException
{
    Matcher matcher = httplogPattern.matcher(value.toString());
    if(matcher.matches())
    {
        String linkUrl = matcher.group(4);
        word.set(linkUrl);
        context.write(word, one);
    }
}
```

Map task receives each line in the log file as a different key-value pair. It parses the lines using regular expressions and emits the link as the key, and number `one` as the value.

Then, Hadoop collects all values for different keys (link) and invokes the reducer once for each link. Then each Reducer counts the number of hits for each link.

```
public void reduce(Text key, Iterable<IntWritable> values,
    Context context) throws IOException, InterruptedException
{
    int sum = 0;
    for (IntWritable val : values)
    {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}
```

The `main()` method of the job works similar to the earlier recipe.

Calculating frequency distributions and sorting using MapReduce

Frequency distribution is the number of hits received by each URL sorted in the ascending order, by the number hits received by a URL. We have already calculated the number of hits in the earlier recipe. This recipe will sort the list.

Getting ready

- ▶ This recipe assumes that you have followed the first chapter and have installed Hadoop. We will use the `HADOOP_HOME` to refer to Hadoop installation folder.
- ▶ Start Hadoop by following the instructions in the first chapter.
- ▶ This recipe assumes that you are aware of how Hadoop processing works. If you have not already done so, you should follow the recipe *Writing a WordCount MapReduce sample, bundling it and running it using standalone Hadoop* from Chapter 1, *Getting Hadoop Up and Running in a Cluster*.
- ▶ This recipe will use the results from the recipe *Performing Group-By using MapReduce* of this chapter. Follow it if you have not done so already.

How to do it...

The following steps show how to calculate frequency distribution using MapReduce:

1. We will use the data from the previous recipe here. So follow the recipe if you have not already done so.
2. Run the MapReduce job using the following command from `HADOOP_HOME`:

```
> bin/hadoop jar hadoop-cookbook-chapter6.jar chapter6.
WeblogFrequencyDistributionProcessor/data/output2 /data/output3
```
3. Read the results by running the following command:

```
> bin/hadoopdfs -cat /data/output3/*
```

You will see that it will print the results as following:

```
/cgi-bin/imagemap/countdown?91,175      12
/cgi-bin/imagemap/countdown?105,143     13
/cgi-bin/imagemap/countdown70?177,284   14
```

How it works...

The second recipe of this chapter calculated the number of hits received by each link, and the frequency distribution as a sorted list of those results in that recipe. Therefore, let us sort the results of the second recipe.

MapReduce always sorts the key-value pairs emitted by the mappers by their keys before delivering them to the reducers. We will use this to sort the results.

You can find the source for the recipe from `src/chapter6/WeblogFrequencyDistributionProcessor.java`.

Map task for the job will look like the following:

```
public static class AMapper extends Mapper<Object,
    Text, IntWritable, Text>
{
    public void map(Object key, Text value, Context context) throws
        IOException, InterruptedException
    {
        String[] tokens = value.toString().split("\\s");
        context.write(
            new IntWritable(Integer.parseInt(tokens[1])),
            new Text(tokens[0]));
    }
}
```

Map task receives each line in the log file as a different key-value pair. It parses the lines using regular expressions and emits the number of hits as the key and the URL name as the value. Hadoop sorts the key-value pairs emitted by the mapper before calling the reducers, and therefore the reducer will receive the pairs in sorted order. Hence, it just has to emit them as they arrive.

```
public static class AReducer extends
    Reducer<IntWritable, Text, Text, IntWritable>
{
    public void reduce(IntWritable key, Iterable<Text> values,
        Context context) throws IOException, InterruptedException
    {
        Iterator<Text> iterator = values.iterator();
        if (iterator.hasNext())
        {
            context.write(iterator.next(), key);
        }
    }
}
```

The `main()` method of the job will work similar to the one in the earlier recipe.

Plotting the Hadoop results using GNU Plot

Although Hadoop jobs can generate interesting analytics, making sense of those results and getting a detailed understanding about the data often require us to see the overall trends in the data. We often do that by plotting the data.

The human eye is remarkably good at detecting patterns, and plotting the data often yields us a deeper understanding of the data. Therefore, we often plot the results of Hadoop jobs using some plotting program.

This recipe explains how to use GNU Plot, which is a free and powerful plotting program, to plot Hadoop results.

Getting ready

- ▶ This recipe assumes that you have followed the previous recipe, *Calculating frequency distributions and sorting using MapReduce*. If you have not done so, please follow the recipe.
- ▶ We will use the `HADOOP_HOME` variable to refer to the Hadoop installation folder.
- ▶ Install the GNU Plot plotting program by following the instructions in <http://www.gnuplot.info/>.

How to do it...

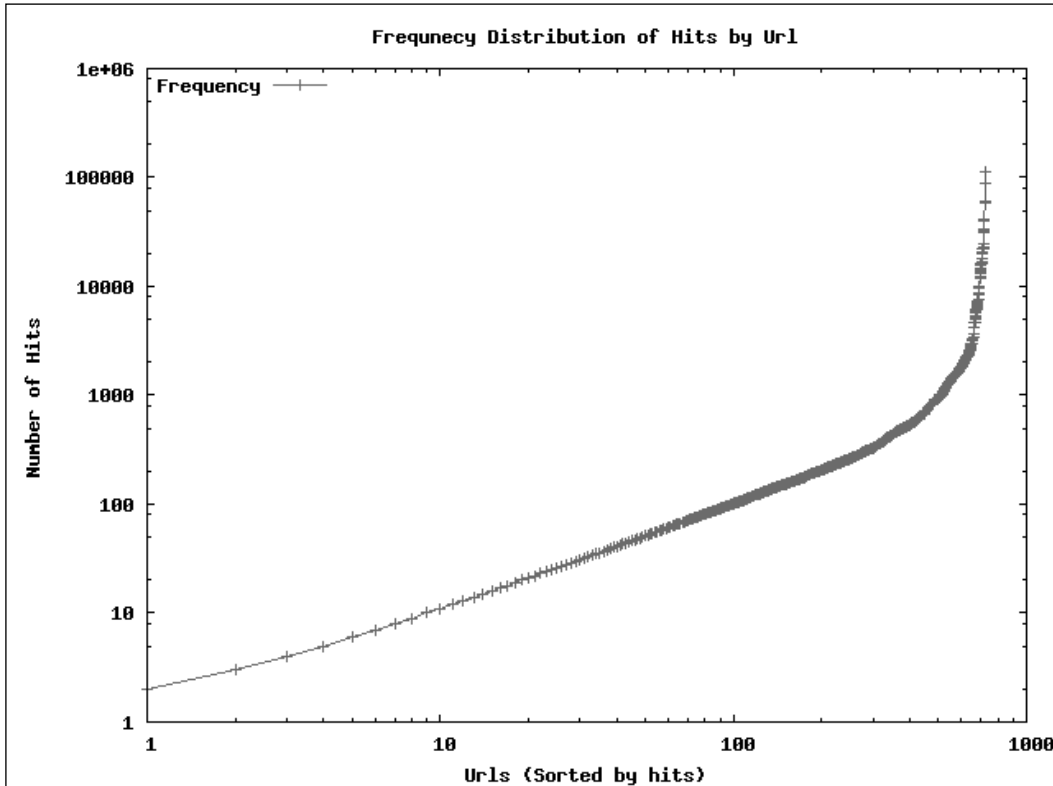
The following steps show how to plot Hadoop job results using GNU Plot.

1. Download the results of the last recipe to a local computer by running the following command from `HADOOP_HOME`:

```
> bin/hadoopdfs -get /data/output3/part-r-00000 2.data
```
2. Copy all the `*.plot` files from `CHAPTER_6_SRC` to `HADOOP_HOME`.
3. Generate the plot by running the following command from `HADOOP_HOME`:

```
>gnuplot httpfreqdist.plot
```

4. It will generate a file called `freqdist.png`, which will look like the following:



The preceding plot is plotted in log-log scale, and the first part of the distribution follows the **zipf** (power law) distribution, which is a common distribution seen in the web. The last few most popular links have much higher rates than expected from a zipf distribution.

Discussion about more details on this distribution is out of scope of this book. However, this plot demonstrates the kind of insights we can get by plotting the analytical results. In most of the future recipes, we will use the GNU plot to plot and to analyze the results.

How it works...

The following steps describe how plotting with GNU plot works:

1. You can find the source for the GNU plot file from `src/chapter6/resources/httpfreqdist.plot`. The source for the plot will look like the following:


```
set terminal png
set output "freqdist.png"

set title "Frequency Distribution of Hits by Url";
set ylabel "Number of Hits";
set xlabel "Urls (Sorted by hits)";
set key left top
set log y
set log x

plot "2.data" using 2 title "Frequency" with linespoints
```
2. Here the first two lines define the output format. This example uses PNG, but GNU plot supports many other terminals like SCREEN, PDF, EPS, and so on.
3. Next four lines define the axis labels and the title.
4. Next two lines define the scale of each axis, and this plot uses log scale for both.
5. Last line defines the plot. Here it is asking GNU plot to read the data from the `2.data` file, and use the data in the second column of the file via `using 2` and to plot it using lines. Columns must be separated by whitespaces.
6. Here if you want to plot one column against other, for example, data from column 1 against column 2, you should write `using 1:2` instead of `using 2`.

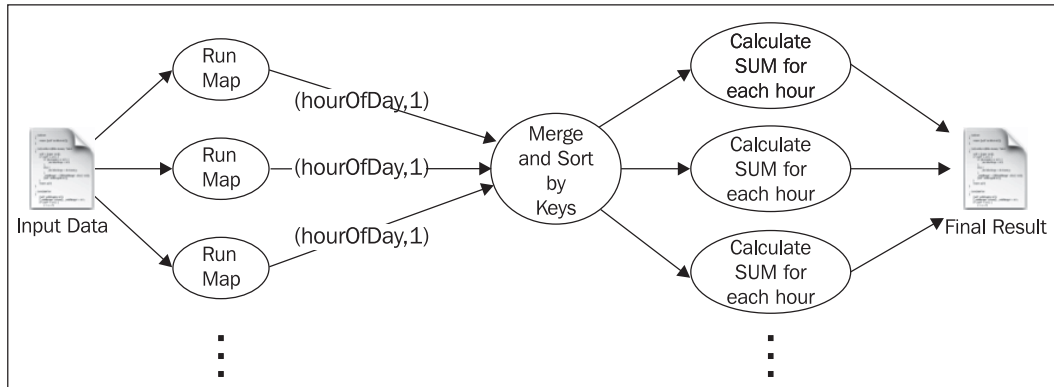
There's more...

You can learn more about GNU plot from <http://www.gnuplot.info/>.

Calculating histograms using MapReduce

Another interesting view into a dataset is a **histogram**. Histogram makes sense only under a continuous dimension (for example, access time and file size). It groups the number of occurrences of some event into several groups in the dimension. For example, in this recipe, if we take the access time from weblogs as the dimension, then we will group the access time by the hour.

The following figure shows a summary of the execution. Here the mapper calculates the hour of the day and emits the "hour of the day" and **1** as the key and value respectively. Then each reducer receives all the occurrences of one hour of a day, and calculates the number of occurrences:



Getting ready

- ▶ This recipe assumes that you have followed the first chapter and have installed Hadoop. We will use the `HADOOP_HOME` variable to refer to the Hadoop installation folder.
- ▶ Start Hadoop by following the instructions in the first chapter.
- ▶ This recipe assumes that you are aware of how Hadoop processing works. If you have not already done so, you should follow the recipe *Writing a WordCount MapReduce sample, bundling it and running it using standalone Hadoop* from Chapter 1, *Getting Hadoop Up and Running in a Cluster*.

How to do it...

The following steps show how to calculate and plot a Histogram:

1. Download the weblog dataset from `ftp://ita.ee.lbl.gov/traces/NASA_access_log_Jul95.gz` and unzip it. We will call this `DATA_DIR`.
2. Upload the data to HDFS by running the following commands from `HADOOP_HOME`. If `/data` is already there, clean it up:


```

>bin/hadoopdfs -mkdir /data
>bin/hadoopdfs -mkdir /data/input1
>bin/hadoopdfs -put <DATA_DIR>/NASA_access_log_Jul95 /data/input1
      
```

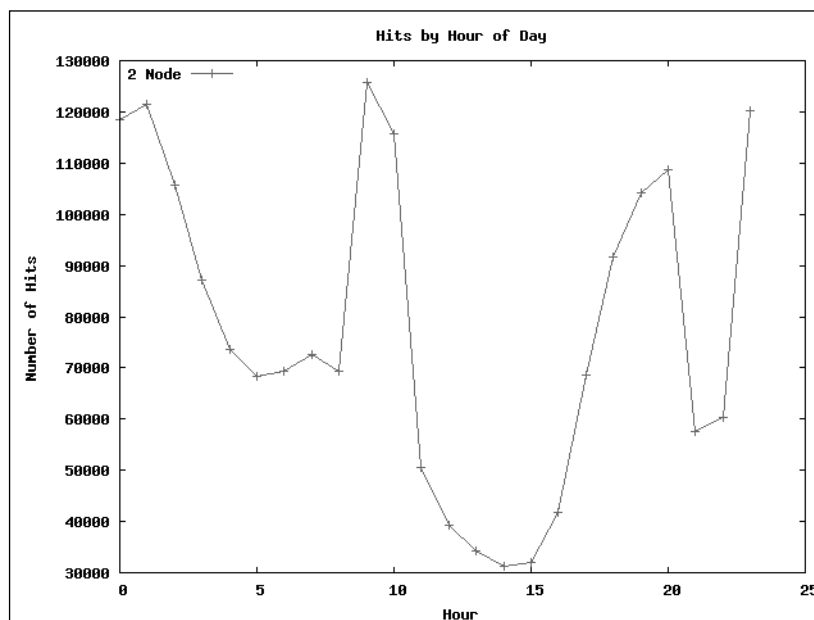
3. Unzip the source code of this chapter (chapter6.zip). We will call that folder CHAPTER_6_SRC.
4. Change the `hadoop.home` property in the `CHAPTER_6_SRC/build.xml` file to point to your Hadoop installation folder.
5. Compile the source by running the `ant build` command from the `CHAPTER_6_SRC` folder.
6. Copy the `build/lib/hadoop-cookbook-chapter6.jar` file to `HADOOP_HOME`.
7. Run the MapReduce job through the following command from `HADOOP_HOME`:

```
> bin/hadoop jar hadoop-cookbook-chapter6.jar chapter6.  
WeblogTimeOfDayHistogramCreator/data/input1 /data/output4
```
8. Read the results by running the following command:

```
>bin/hadoopdfs -cat /data/output4/*
```
9. Download the results of the last recipe to a local computer by running the following command from `HADOOP_HOME`:

```
> bin/hadoopdfs -get /data/output4/part-r-00000 3.data
```
10. Copy all the *.plot files from `CHAPTER_6_SRC` to `HADOOP_HOME`.
11. Generate the plot by running the following command from `HADOOP_HOME`:

```
>gnuplot httphistbyhour.plot
```
12. It will generate a file called `hitsbyHour.png`, which will look like following:



As you can see from the figure, most of the access to NASA is at night, whereas there noontime. Also, two peaks roughly follow the tea times.

How it works...

You can find the source for the recipe from `src/chapter6/WeblogTimeOfDayHistogramCreator.java`. As explained in the first recipe of this chapter, we will use regular expressions to parse the log file and extract the access time from the log files.

The following code segment shows the mapper function:

```
public void map(Object key, Text value,
    Context context) throws IOException, InterruptedException
{
    Matcher matcher = httplogPattern.matcher(value.toString());
    if (matcher.matches())
    {
        String timeAsStr = matcher.group(2);
        Date time = dateFormatter.parse(timeAsStr);
        Calendar calendar = GregorianCalendar.getInstance();
        calendar.setTime(time);
        int hours = calendar.get(Calendar.HOUR_OF_DAY);
        context.write(new IntWritable(hours), one);
    }
}
```

Map task receives each line in the log file as a different key-value pair. It parses the lines using regular expressions and extracts the access time for each web page access. Then, the mapper function extracts the hour of the day from the access time and emits the hour of the day and one as output of the mapper function.

Then, Hadoop collects all key-value pairs, sorts them, and then invokes the reducer once for each key. Each reducer walks through the values and calculates the count of page accesses for each hour.

```
public void reduce(IntWritable key,
    Iterable<IntWritable> values,
    Context context) throws IOException, InterruptedException
{
    int sum = 0;
    for (IntWritable val : values)
    {
        sum += val.get();
    }
    context.write(key, new IntWritable(sum));
}
```

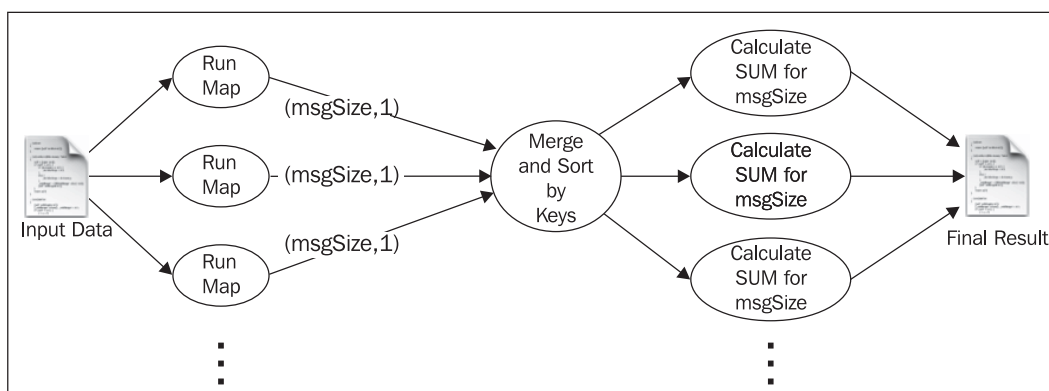
The `main()` method of the job looks similar to the WordCount example as described in the earlier recipe.

Calculating scatter plots using MapReduce

Another useful tool while analyzing data is a **Scatter plot**. We use Scatter plot to find the relationship between two measurements (dimensions). It plots the two dimensions against each other.

For an example, this recipe analyzes the data to find the relationship between the size of the web pages and the number of hits received by the web page.

The following figure shows a summary of the execution. Here, the mapper calculates and emits the message size (rounded to 1024 bytes) as the key and `one` as the value. Then the reducer calculates the number of occurrences for each message size:



Getting ready

- ▶ This recipe assumes that you have followed the first chapter and have installed Hadoop. We will use the `HADOOP_HOME` variable to refer to the Hadoop installation folder.
- ▶ Start Hadoop by following the instructions in the first chapter.
- ▶ This recipe assumes you are aware of how Hadoop processing works. If you have not already done so, you should follow the recipe *Writing a WordCount MapReduce sample, bundling it and running it using standalone Hadoop* from Chapter 1, *Getting Hadoop Up and Running in a Cluster*.

How to do it...

The following steps show how to use MapReduce to calculate the correlation between two datasets:

1. Download the weblog dataset from `ftp://ita.ee.lbl.gov/traces/NASA_access_log_Jul95.gz` and unzip it. We will call this `DATA_DIR`.
2. Upload the data to HDFS by running following commands from `HADOOP_HOME`. If `/data` is already there, clean it up:

```
>bin/hadoopdfs -mkdir /data  
>bin/hadoopdfs -mkdir /data/input1  
>bin/hadoopdfs -put <DATA_DIR>/NASA_access_log_Jul95 /data/input1
```
3. Unzip the source code of this chapter (`chapter6.zip`). We will call that folder `CHAPTER_6_SRC`.
4. Change the `hadoop.home` property in the `CHAPTER_6_SRC/build.xml` file to point to your Hadoop installation folder.
5. Compile the source by running the `ant build` command from the `CHAPTER_6_SRC` folder.
6. Copy the `build/lib/hadoop-cookbook-chapter6.jar` file to your `HADOOP_HOME`.
7. Run the MapReduce job through following command from `HADOOP_HOME`:

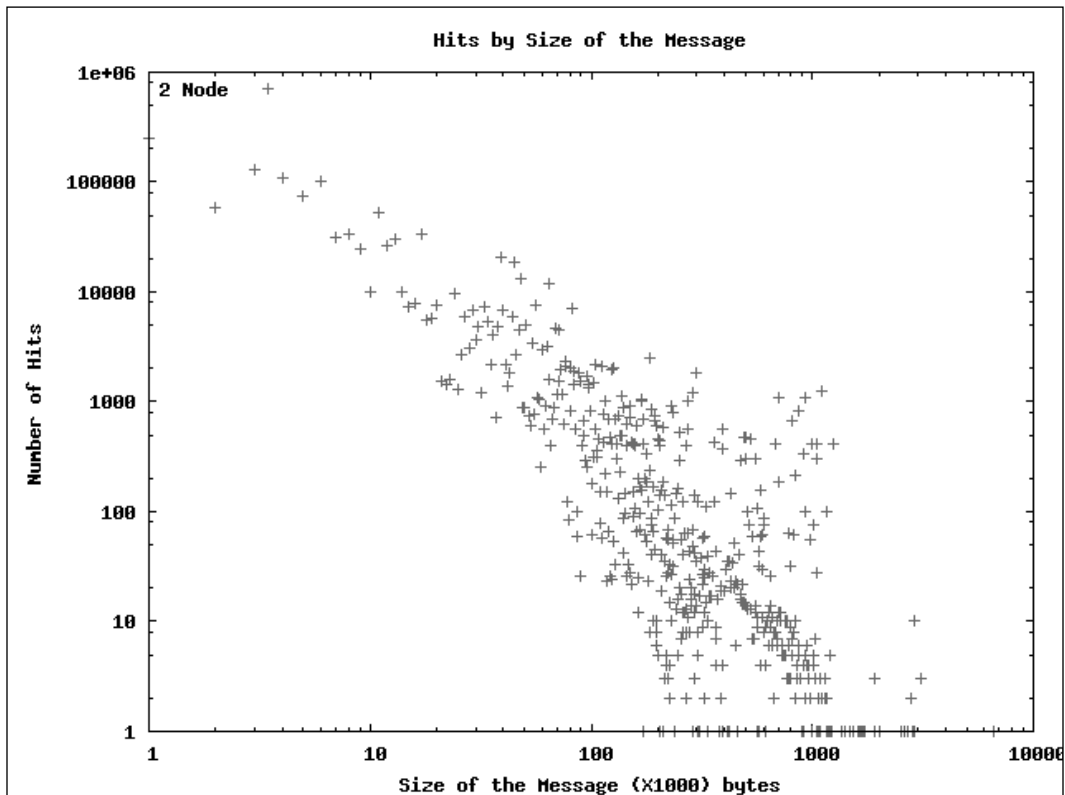
```
> bin/hadoop jar hadoop-cookbook-chapter6.jar chapter6.  
WeblogMessagesizevsHitsProcessor/data/input1 /data/output5
```
8. Read the results by running the following command:

```
>bin/hadoopdfs -cat /data/output5/*
```
9. Download the results of the last recipe to the local computer by running the following command from `HADOOP_HOME`:

```
> bin/hadoopdfs -get /data/output5/part-r-00000 5.data
```
10. Copy all the `*.plot` files from `CHAPTER_6_SRC` to `HADOOP_HOME`.
11. Generate the plot by running the following command from `HADOOP_HOME`.

```
>gnuplot httphitsvsmssize.plot
```

12. It will generate a file called `hitsbysize.png`, which will look like following screenshot:



The plot shows a negative correlation between the number of hits and the size of the messages in the log scales, which also suggest a power law distribution.

How it works...

You can find the source for the recipe from `src/chapter6/WeblogMessagesizevsHitsProcessor.java`.

The following code segment shows the code for the mapper. Just like earlier recipes, we will use regular expressions to parse the log entries from log files:

```
public void map(Object key, Text value,
    Context context) throws IOException, InterruptedException
{
    Matcher matcher = httplogPattern.matcher(value.toString());
    if (matcher.matches())
```

```
{  
    int size = Integer.parseInt(matcher.group(5));  
    context.write(new IntWritable(size / 1024), one);  
}  
}
```

Map task receives each line in the log file as a different key-value pair. It parses the lines using regular expressions and emits the file size as 1024-byte blocks as the key and one as the value.

Then, Hadoop collects all key-value pairs, sorts them, and then invokes the reducer once for each key. Each reducer walks through the values and calculates the count of page accesses for each file size.

```
public void reduce(IntWritable key, Iterable<IntWritable> values,  
    Context context) throws IOException, InterruptedException  
{  
    int sum = 0;  
    for (IntWritable val : values)  
    {  
        sum += val.get();  
    }  
    context.write(key, new IntWritable(sum));  
}
```

The `main()` method of the job looks similar to the earlier recipes.

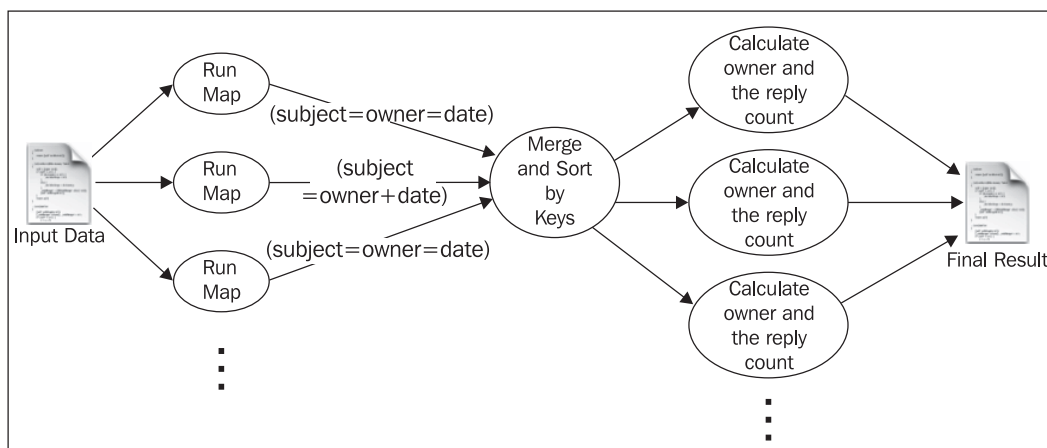
Parsing a complex dataset with Hadoop

Datasets we parsed so far were simple, where each data item was contained in a single line. Therefore, we were able to use Hadoop default parsing support to parse those datasets. However, some datasets have much complex formats.

In this recipe, we will analyze Tomcat developer mailing list archives. In the archive, each e-mail is composed of by multiple lines in the log file. Therefore, we will write a Hadoop input formatter to process the e-mail archive.

This recipe parses the complex e-mail list archives, and finds the owner (person who started the thread) and the number of replies received by each e-mail thread.

The following figure shows a summary of the execution. Here the mapper emits the subject of the mail as key and the sender's e-mail address and date as the value. Then Hadoop groups data by the e-mail subject and sends all the data related to that thread to the same reducer.



Then, the reducer calculates the owner of the thread it received, and the number of replies received by the thread.

Getting ready

- ▶ This recipe assumes that you have followed the first chapter and have installed Hadoop. We will use the `HADOOP_HOME` variable to refer to the Hadoop installation folder.
- ▶ Start Hadoop by following the instructions in the first chapter.
- ▶ This recipe assumes you are aware of how Hadoop processing works. If you have not already done so, you should follow the recipe *Writing a WordCount MapReduce sample, bundling it and running it using standalone Hadoop* from Chapter 1, *Getting Hadoop Up and Running in a Cluster*.

How to do it...

The following steps describe how to parse the Tomcat e-mail list dataset that has complex data format using Hadoop by writing an input formatter:

1. Download the Apache Tomcat developer list e-mail archives for the year 2012 available from http://mail-archives.apache.org/mod_mbox/tomcat-users/. We call the destination folder as `DATA_DIR`.

2. Upload the data to HDFS by running the following commands from `HADOOP_HOME`. If `/data` is already there, clean it up:


```
>bin/hadoopdfs -mkdir /data
>bin/hadoopdfs -mkdir /data/input2
>bin/hadoopdfs -put <DATA_DIR>/*.mbox /data/input2
```
3. Unzip the source code for this chapter (`chapter6.zip`). We will call that folder `CHAPTER_6_SRC`.
4. Change the `hadoop.home` property in the `CHAPTER_6_SRC/build.xml` file to point to your Hadoop installation folder.
5. Compile the source by running the `ant build` command from the `CHAPTER_6_SRC` folder.
6. Copy the `build/lib/hadoop-cookbook-chapter6.jar` file to `HADOOP_HOME`.
7. Run the MapReduce job through the following command from `HADOOP_HOME`:


```
> bin/hadoop jar hadoop-cookbook-chapter6.jar chapter6.
MLReceiveReplyProcessor/data/input2 /data/output6
```
8. Read the results by running the following command:


```
>bin/hadoopdfs -cat /data/output6/*
```

How it works...

As explained before, this dataset has data items that span multiple lines. Therefore, we have to write a custom data formatter to parse the data. You can find the source for the recipe from `src/chapter6/WebLogMessageSizeAggregator.java`, `src/chapter6/MboxFileFormat.java`, `src/chapter6/MboxFileReader.java`.

When the Hadoop job starts, it invokes the formatter to parse the input files. We add a new formatter via the `main()` method as highlighted in the following code snippet:

```
Job job = new Job(conf, "LogProcessingHitsByLink");
job.setJarByClass(MLReceiveReplyProcessor.class);
job.setMapperClass(AMapper.class);
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(Text.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);
job.setReducerClass(AReducer.class);
job.setInputFormatClass(MboxFileFormat.class);
FileInputFormat.addInputPath(job, new Path(otherArgs[0]));
FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
```

As shown by the following code, the new formatter creates a record reader, which is used by Hadoop to read input keys and values:

```
public class MboxFileFormat extends
    FileInputFormat<Text, Text>
{
    private MBoxFileReader boxFileReader = null;
    public RecordReader<Text, Text> createRecordReader(
        InputSplit inputSplit, TaskAttemptContext attempt)
        throws IOException, InterruptedException
    {
        boxFileReader = new MBoxFileReader();
        boxFileReader.initialize(inputSplit, attempt);
        return boxFileReader;
    }
}
```

The following code snippet shows the record reader:

```
public class MBoxFileReader extends
    RecordReader<Text, Text>
{
    public void initialize(InputSplit inputSplit,
        TaskAttemptContext attempt) throws IOException,
        InterruptedException
    {
        Path path = ((FileSplit) inputSplit).getPath();
        FileSystem fs = FileSystem.get(attempt.getConfiguration());
        FSDataInputStream fsStream = fs.open(path);
        reader = new BufferedReader(
            new InputStreamReader(fsStream));
    }
}
```

The `initialize()` method reads the file from HDFS:

```
public Boolean nextKeyValue() throws IOException,
    InterruptedException
{
    if (email == null)
    {
        return false;
    }
    count++;
    while ((line = reader.readLine()) != null)
    {
```

```
Matcher matcher = pattern1.matcher(line);
if (!matcher.matches())
{
    email.append(line).append("\n");
}
else
{
    parseEmail(email.toString());
    email = new StringBuffer();
    email.append(line).append("\n");
    return true;
}
}
parseEmail(email.toString());
email = null; return true;
}
```

Finally, the `nextKeyValue()` method parses the file, and gives users access to the key and values for this dataset. Value has the *from*, *subject*, and *date* of each e-mail separated by a #.

The following code snippet shows the map task source code:

```
public void map(Object key, Text value,
    Context context) throws IOException, InterruptedException
{
    String[] tokens = value.toString().split("#");
    String from = tokens[0];
    String subject = tokens[1];
    String date = tokens[2].replaceAll(",", " ");
    subject = subject.replaceAll("Re:", "");
    context.write(new Text(subject), new Text(date + "#" + from));
}
```

The map task receives each line in the log file as a different key-value pair. It parses the lines by breaking it by the #, and emits the subject as the key, and date and from as the value.

Then, Hadoop collects all key-value pairs, sorts them, and then invokes the reducer once for each key. Since we use the e-mail subject as the key, each reducer will receive all the information about each e-mail thread. Then, each reducer walks through all the e-mails and finds out who sent the first e-mail and how many replies have been received by each e-mail thread.

```
public void reduce(Text key, Iterable<Text> values, Context context)
    throws IOException, InterruptedException
{

```

```

TreeMap<Long, String>replyData = new TreeMap<Long, String>();
for (Text val : values)
{
    String[] tokens = val.toString().split("#");
    if(tokens.length != 2)
    {
        throw new IOException("Unexpected token " + val.toString());
    }
    String from = tokens[1];
    Date date = dateFormatter.parse(tokens[0]);
    replyData.put(date.getTime(), from);
}
String owner = replyData.get(replyData.firstKey());
intreplyCount = replyData.size();
intselfReplies = 0;
for(String from: replyData.values())
{
    if(owner.equals(from))
    {
        selfReplies++;
    }
}
replyCount = replyCount - selfReplies;
context.write(new Text(owner), new Text(replyCount+"#" + selfReplies));
}

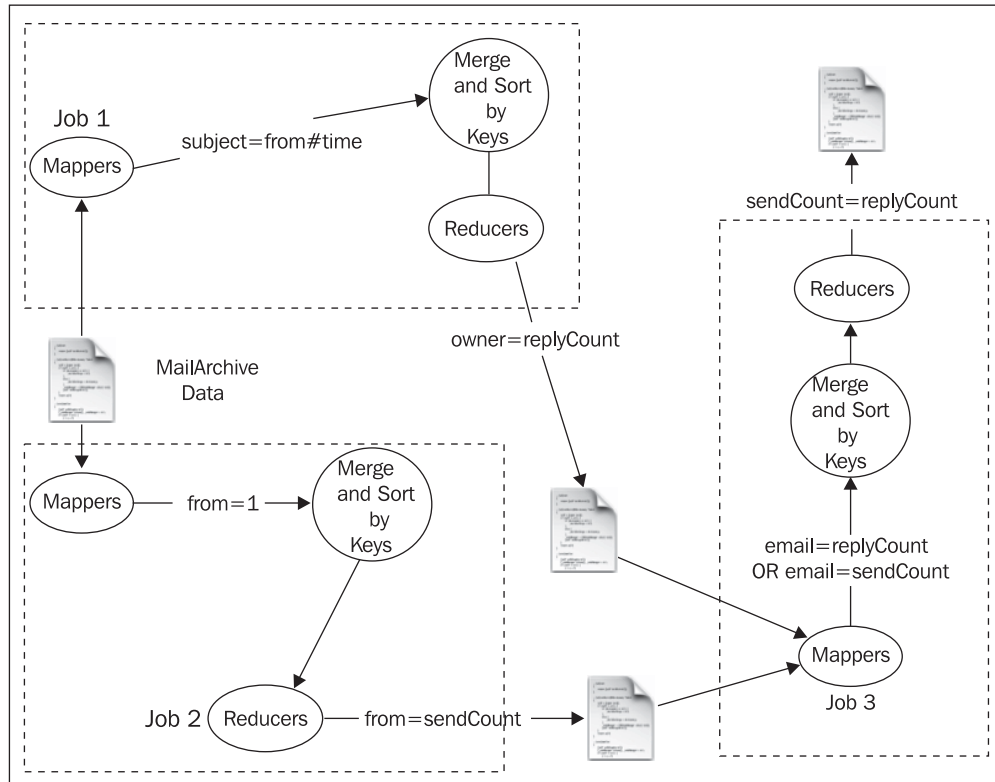
```

Joining two datasets using MapReduce

As we have observed already, Hadoop is very good at reading through a dataset and calculating the analytics. However, often we will have to merge two datasets for analyzing the data. This recipe will explain how to join two datasets using Hadoop.

As an example, this recipe will use the Tomcat developer archives dataset. A common belief among the open source community is that, the more a developer is involved with the community (for example, by replying to threads and helping others and so on), the more quickly he will receive a response to his queries. In this recipe we will test this hypothesis using the Tomcat developer mailing list.

To test this hypothesis, we will run the MapReduce jobs as explained in the following figure:



We would start with e-mail archives in the MBOX format, and we will read the mail using the MBOX format class explained in the earlier recipe. Then, the Hadoop job will receive the sender of the e-mail (from), e-mail subject, and the date the e-mail was sent, as inputs.

1. In the first job, mapper will emit the subject as key, and the sender's e-mail address and date as the value. Then, the reducer step will receive all values with the same subject and it will output the subject as the key, and the owner and reply count as the value. We have executed this job in the earlier recipe.
2. In the second job, the mapper step emits the sender's e-mail address as the key and one as the value. Then, the reducer step will receive all the e-mails sent from the same address to the same reducer. Using this data, each reducer will emit the e-mail address as the key and the number of e-mails sent from that e-mail address as the value.
3. Finally, the third job reads both the output from earlier jobs, joins the results, and emits the number of replies sent by each e-mail address and the number of replies received by each e-mail address as the output.

Getting ready

- ▶ This recipe assumes that you have followed the first chapter and have installed Hadoop. We will use the `HADOOP_HOME` variable to refer to the Hadoop installation folder.
- ▶ Start Hadoop by following the instructions in the first chapter.
- ▶ This recipe assumes you are aware of how Hadoop processing works. If you have not already done so, you should follow the recipe *Writing a WordCount MapReduce sample, bundling it and running it using standalone Hadoop* from Chapter 1, *Getting Hadoop Up and Running in a Cluster*.

How to do it...

The following steps show how to use MapReduce to join two datasets:

1. If you have not already done so, run the previous recipe, which will set up the environment and run the first job as explained in the figure.
2. Run the second MapReduce job through the following command from `HADOOP_HOME`:

```
> bin/hadoop jar hadoop-cookbook-chapter6.jar chapter6.
MLSendReplyProcessor/data/input2 /data/output7
```
3. Read the results by running the following command:

```
>bin/hadoopdfs -cat /data/output7/*
```
4. Create a new folder `input3` and copy both results from earlier jobs to that folder in HDFS:

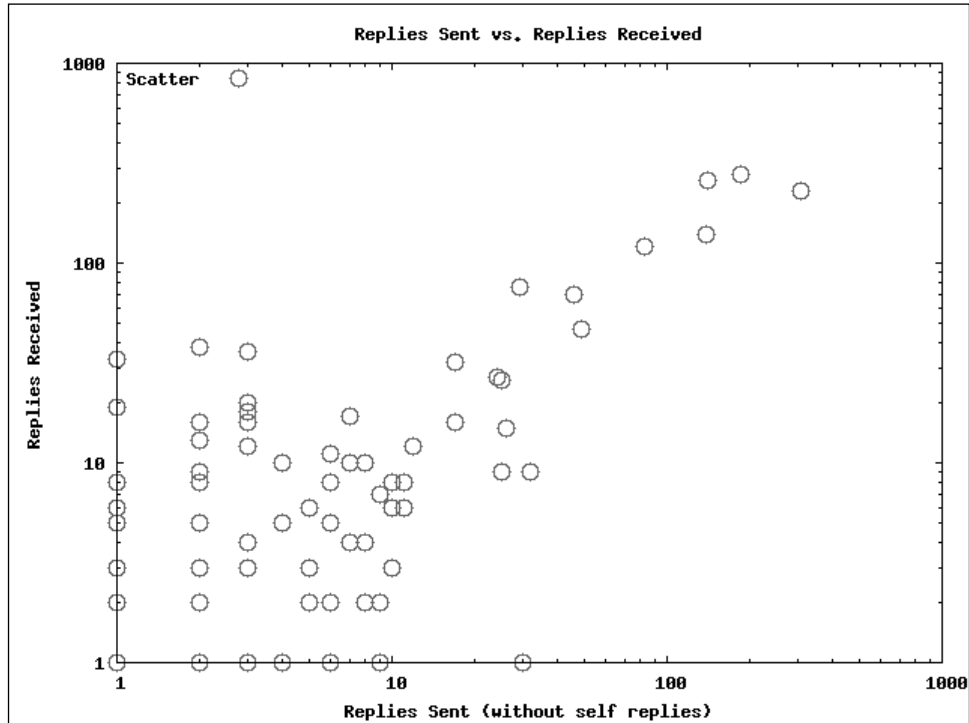
```
> bin/hadoopdfs -mkdir /data/input3
> bin/hadoopdfs -cp /data/output6/part-r-00000 /data/input3/1.
data
> bin/hadoopdfs -cp /data/output7/part-r-00000 /data/input3/2.data
```
5. Run the third MapReduce job through the following command from `HADOOP_HOME`:

```
> bin/hadoop jar hadoop-cookbook-chapter6.jar chapter6.
MLJoinSendReceiveReplies /data/input3 /data/output8
```
6. Download the results of the last recipe to the local computer by running the following command from `HADOOP_HOME`:

```
> bin/hadoopdfs -get /data/output8/part-r-00000 8.data
```
7. Copy all the `*.plot` files from `CHAPTER_6_SRC` to `HADOOP_HOME`.
8. Generate the plot by running the following command from `HADOOP_HOME`:

```
>gnuplot sendvsreceive.plot
```

9. It will generate a file called `sendreceive.png`, which will look like following:



The graph confirms our hypothesis, and like before, the data approximately follows a power law distribution.

How it works...

You can find the source for the recipe from `src/chapter6/MLSendReplyProcessor.java` and `src/chapter6/MLJoinSendReceiveReplies.java`. We have already discussed the working of the first job in the earlier recipe.

The following code snippet shows the `map()` function for the second job. It receives the sender's e-mail, subject, and date separated by # as input, which parses the input and outputs the sender's e-mail as the key and the date the e-mail was sent, as the value:

```
public void map(Object key, Text value, Context context)
    throws IOException, InterruptedException
{
    String[] tokens = value.toString().split("#");
    String from = tokens[0]; String date = tokens[2];
    context.write(new Text(from), new Text(date));
}
```

The following code snippet shows the `reduce()` function for the second job. Each `reduce()` function receives the time of all the e-mails sent by one sender. The reducer counts the number of replies sent by each sender, and outputs the sender's name as the key and the replies sent, as the value:

```
public void reduce(Text key, Iterable<Text> values,
    Context context ) throws IOException, InterruptedException
{
    int sum = 0;
    for (Text val : values)
    {
        sum = sum +1;
    }
    context.write(key, new IntWritable(sum));
}
```

The following code snippet shows the `map()` function for the third job. It reads the outputs of the first and second jobs and writes them as the key-value pairs:

```
public void map(Object key, Text value, Context context) throws
    IOException, InterruptedException {
    String[] tokens = value.toString().split("\\s");
    String from = tokens[0];
    String replyData = tokens[1];
    context.write(new Text(from), new Text(replyData));
}
```

The following code snippet shows the `reduce()` function for the third job. Since, both the output of the first and the second job has the same key, the sent replies and received replies for a given user are sent to the same reducer. The reducer does some adjustments to remove self-replies, and outputs the sent replies and received replies as the key and value respectively of the reducer, thus joining the two datasets:

```
public void reduce(Text key, Iterable<Text> values, Context context)
    throws IOException, InterruptedException
{
    StringBuffer buf = new StringBuffer("[");
    try
    {
        int sendReplyCount = 0;
        int receiveReplyCount = 0;
        for (Text val : values)
        {
            String strVal = val.toString();
            if(strVal.contains("#"))
            {

```

```
String[] tokens = strVal.split("#");
Int repliesOnThisThread = Integer.parseInt(tokens[0]);
Int selfRepliesOnThisThread = Integer.parseInt(tokens[1]);
receiveReplyCount = receiveReplyCount + repliesOnThisThread;
sendReplyCount = sendReplyCount - selfRepliesOnThisThread;
}
else
{
    sendReplyCount = sendReplyCount + Integer.parseInt(strVal);
}
}
context.write(new IntWritable(sendReplyCount),
    new IntWritable(receiveReplyCount)); buf.append("]");
}
catch (NumberFormatException e)
{
    System.out.println("ERROR "+ e.getMessage());
}
}
```

Here the final job is an example of using the MapReduce to join two datasets. The idea is to send all the values that need to be joined under the same key to the same reducer, and join the data there.

Where to buy this book

You can buy Hadoop MapReduce Cookbook from the Packt Publishing website:

<http://www.packtpub.com/hadoop-mapreduce-cookbook/book>.

Free shipping to the US, UK, Europe and selected Asian countries. For more information, please read our [shipping policy](#).

Alternatively, you can buy the book from Amazon, BN.com, Computer Manuals and most internet book retailers.



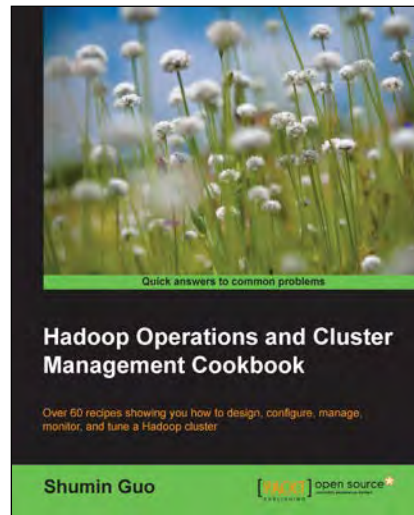
www.PacktPub.com

For More Information:

www.packtpub.com/hadoop-mapreduce-cookbook/book

Hadoop Operations and Cluster Management Cookbook

Shumin Guo



Chapter No. 3 "Configuring a Hadoop Cluster"

In this package, you will find:

A Biography of the author of the book

A preview chapter from the book, Chapter NO.3 "Configuring a Hadoop Cluster"

A synopsis of the book's content

Information on where to buy this book

About the Author

Shumin Guo is a PhD student of Computer Science at Wright State University in Dayton, OH. His research fields include Cloud Computing and Social Computing. He is enthusiastic about open source technologies and has been working as a System Administrator, Programmer, and Researcher at State Street Corp. and LexisNexis.

I would like to sincerely thank my wife, Min Han, for her support both technically and mentally. This book would not have been possible without encouragement from her.

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

Hadoop Operations and Cluster Management Cookbook

Today, many organizations are facing the Big Data problem. Managing and processing Big Data can incur a lot of challenges for traditional data processing platforms such as relational database systems. Hadoop was designed to be a distributed and scalable system for dealing with Big Data problems. A Hadoop-based Big Data platform uses Hadoop as the data storage and processing engine. It deals with the problem by transforming the Big Data input into expected output.

Hadoop Operations and Cluster Management Cookbook provides examples and step-by-step recipes for you to administrate a Hadoop cluster. It covers a wide range of topics for designing, configuring, managing, and monitoring a Hadoop cluster. The goal of this book is to help you manage a Hadoop cluster more efficiently and in a more systematic way.

In the first three chapters, you will learn practical recipes to configure a fully distributed Hadoop cluster. The subsequent management, hardening, and performance tuning chapters will cover the core topics of this book. In these chapters, you will learn practical commands and best practices to manage a Hadoop cluster. The last important topic of the book is the monitoring of a Hadoop cluster. And, we will end this book by introducing steps to build a Hadoop cluster using the AWS cloud.

What This Book Covers

Chapter 1, Big Data and Hadoop, introduces steps to define a Big Data problem and outlines steps to build a Hadoop-based Big Data platform.

Chapter 2, Preparing for Hadoop Installation, describes the preparation of a Hadoop cluster configuration. Topics include choosing the proper cluster hardware, configuring the network, and installing the Linux operating system.

Chapter 3, Configuring a Hadoop Cluster, introduces recipes to configure a Hadoop cluster in pseudo-distributed mode as well as in fully distributed mode. We will also describe steps to verify and troubleshoot a Hadoop cluster configuration.

Chapter 4, Managing a Hadoop Cluster, shows you how to manage a Hadoop cluster. We will learn cluster maintenance tasks and practical steps to do the management. For example, we will introduce the management of an HDFS filesystem, management of MapReduce jobs, queues and quota, and so on.

Chapter 5, Hardening a Hadoop Cluster, introduces recipes to secure a Hadoop cluster. We will show you how to configure ACL for authorization and Kerberos for authentication, configure NameNode HA, recover from a failed NameNode, and so on.

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

Chapter 6, Monitoring a Hadoop Cluster, explains how to monitor a Hadoop cluster with various tools, such as Ganglia and Nagios.

Chapter 7, Tuning a Hadoop Cluster for Best Performance, introduces best practices to tune the performance of a Hadoop cluster. We will tune the memory profile, the MapReduce scheduling strategy, and so on to achieve best performance for a Hadoop cluster.

Chapter 8, Building a Hadoop Cluster with Amazon EC2 and S3, shows you how to configure a Hadoop cluster in the Amazon cloud. We will explain steps to register, connect, and start VM instances on EC2. We will also show you how to configure a customized AMI for a Hadoop cluster on EC2.

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

3

Configuring a Hadoop Cluster

In this chapter, we will cover:

- ▶ Choosing a Hadoop version
- ▶ Configuring Hadoop in pseudo-distributed mode
- ▶ Configuring Hadoop in fully-distributed mode
- ▶ Validating Hadoop installation
- ▶ Configuring ZooKeeper
- ▶ Installing HBase
- ▶ Installing Hive
- ▶ Installing Pig
- ▶ Installing Mahout

Introduction

After finishing all the preparing tasks, we are ready to configure a Hadoop cluster in this chapter. First, we will give you some tips on choosing a proper Hadoop release version. Then we will show you how to configure a Hadoop cluster in pseudo-distributed and fully-distributed mode. Pseudo-distributed mode is a very good starting point if you have no experience configuring a Hadoop cluster. In this mode, we will configure all the Hadoop daemons to run on a single machine, which can give us the first feeling of a working Hadoop cluster while minimizing configuration difficulties. Next, we will show you how to validate a Hadoop cluster. The importance of validating a Hadoop cluster configuration will never be overemphasized. We typically use this step to confirm that the Hadoop cluster is running as expected. The last few recipes will show you how to install a few components in the cluster.

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

Choosing a Hadoop version

As an open source project, Hadoop has been under active development over the past few years. New versions are being released regularly. These new releases either fix bugs contributed by the community, leading to a more stable Hadoop software stack, or add new features for the purpose of more full-fledged and enterprise-level distribution.

In this section, we are going to review the history of releases of Hadoop, pointing out features of these releases. More importantly, we will give tips on choosing a proper Hadoop distribution.

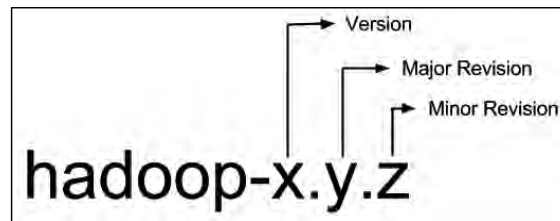
Getting ready

In general, the **release version number** of a Hadoop distribution consists of three parts: the **version** number, the **major revision** number, and the **minor revision** number.



Sometimes the revision number can have a fourth part, for example, 0.20.203.0, but this is relatively rare.

A Hadoop release name can be described with the following figure:



How to do it...

The following table shows features of major Hadoop releases:

Feature\Version	2.0.x	1.1.x	0.23.x	0.20.x
Stable		Yes		Yes
MRv1		Yes		Yes
MRv2	Yes		Yes	
Kerberos security	Yes	Yes	Yes	
HDFS federation	Yes		Yes	

Feature\Version	2.0.x	1.1.x	0.23.x	0.20.x
NameNode HA	Yes		Yes	
HDFS append	Yes	Yes	Yes	
HDFS symbolic links	Yes	Yes	Yes	

The table tells us that Hadoop is evolving rapidly, with new features such as security, **HDFS federation**, and **NameNode HA** being added over time. Another lesson we can learn from the table is that the most recent stable release, Version 1.1.x, does not contain all the features. And although release Version 2.0.x is the most feature-rich Hadoop release, it is still in alpha state requiring further improvements.

So, which version should you choose for your deployment? Generally, we need to consider two properties: stability and features. For a production deployment, we definitely want to deploy a stable release and we want to use the release that contains all the required features. Clearly, our current optimal and only choice is Version 1.1.x or specifically Version 1.1.2 as of this book's writing.

See also

- More information about Hadoop releases can be found at <http://hadoop.apache.org/releases.html>

Configuring Hadoop in pseudo-distributed mode

Pseudo-distributed mode refers to a Hadoop cluster configuration that contains only one node. This mode can be helpful for debugging and validation purposes. In this recipe, we will outline steps to configure Hadoop in pseudo-distributed mode.

Getting ready

Before configuring Hadoop in pseudo-distributed mode, we assume that we have a machine, for example, the master node of the Hadoop cluster, with Linux installed. We also assume that all the necessary tools have been installed and properly configured.

- The most important dependent software is Java, which is the programming language and library that Hadoop is based on. To check that Java has been properly installed, we can use the following command:

```
$ java -version
```

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

You should have output similar to the following:

```
java version "1.7.0_13"  
Java(TM) SE Runtime Environment (build 1.7.0_13-b20)  
Java HotSpot(TM) 64-Bit Server VM (build 23.7-b01, mixed mode)
```

- ▶ If you have installed OpenJDK other than the Oracle's official Java, the output will be similar to the following:

```
Java version "1.7.0_09-icedtea"  
OpenJDK Runtime Environment (fedora-2.3.4.fc17-x86_64)  
OpenJDK 64-Bit Server VM (build 23.2-b09, mixed mode)
```



If you have installed OpenJDK, please refer to the *Installing Java and other tools* recipe from *Chapter 2, Preparing for Hadoop installation*.

- ▶ Download the desired Hadoop distribution. In this book, we assume that we're using Hadoop release 1.1.2. To download a Hadoop release, please visit the following URL:

```
http://www.apache.org/dyn/closer.cgi/hadoop/common/
```

Choose the proper mirror site (or use the suggested link on top of the mirror). Start downloading by clicking on the proper Hadoop release. We suggest downloading a .gzip archived file with the filename ending with `tar.gz`.

- ▶ Alternatively, we can download a Hadoop release with the following command under Linux:

```
wget http://mirror.quintex.com/apache/hadoop/common/hadoop-1.1.2/  
hadoop-1.1.2.tar.gz -P ~
```

- ▶ Last, we assume that the `ssh` password-less login has been properly configured.

How to do it...

Perform the following steps to configure Hadoop in pseudo-distributed mode:

1. Copy the Hadoop archive to the `/usr/local` directory:

```
sudo cp hadoop-1.1.2.tar.gz /usr/local
```

2. Decompress the Hadoop package archive:

```
cd /usr/local  
sudo tar xvf hadoop-1.1.2.tar.gz
```

The uncompressed archive file will contain the following files and folders:

CHANGES.txt	c++	hadoop-examples-1.1.2.jar
lib		
LICENSE.txt	conf	hadoop-miniclust-1.1.2.jar
libexec		
NOTICE.txt	contrib	hadoop-test-1.1.2.jar
sbin		
README.txt	hadoop-ant-1.1.2.jar	hadoop-tools-1.1.2.jar
share		
bin	hadoop-client-1.1.2.jar	ivy
src		
build.xml	hadoop-core-1.1.2.jar	ivy.xml
webapps		



The folder contains several .jar files and folders such as bin, sbin, and conf. The .jar files hadoop-core-1.1.2.jar and hadoop-tools-1.1.2.jar contain the core classes of Hadoop. The files hadoop-examples-1.1.2.jar and hadoop-test-1.1.2.jar contain sample MapReduce jobs.

The conf folder contains cluster configuration files, the bin folder contains commands and scripts to start and stop a cluster, and the sbin folder contains scripts to perform specific tasks.

3. Make a soft link for Hadoop root directory:

```
sudo ln -s hadoop-1.1.2 hadoop
```

4. Use your favorite text editor to open the file ~/.bashrc and add the following contents:

```
export JAVA_HOME=/usr/java/latest
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$JAVA_HOME/bin:HADOOP_HOME/bin
```



We are assuming Oracle Java has been installed under the /usr/java/latest directory.

5. Reload the configuration file ~/.bashrc with the following command:

```
. ~/.bashrc
```

6. Use your favorite text editor to open the file \$HADOOP_HOME/conf/hadoop-env.sh, and change the JAVA_HOME environment variable to the following:

```
export JAVA_HOME=/usr/Java/latest
```

7. Use your favorite text editor to open the file `$HADOOP_HOME/conf/core-site.xml`, and add the following content:

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://localhost:54310</value>
  </property>

  <property>
    <name>mapred.job.tracker</name>
    <value>localhost:54311</value>
  </property>

  <property>
    <name>hadoop.tmp.dir</name>
    <value>/hadoop/tmp/</value>
  </property>
</configuration>
```

8. Use your favorite text editor to open the file `$HADOOP_HOME/conf/hdfs-site.xml`, and add the following content to the file:

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>

  <property>
    <name>dfs.data.dir</name>
    <value>/hadoop/data/</value>
  </property>
</configuration>
```

9. Use your favorite text editor to open the file `$HADOOP_HOME/conf/mapred-site.xml`, and add the following content:

```
<configuration>
  <property>
    <name>mapred.system.dir</name>
    <value>/hadoop/mapred</value>
  </property>
</configuration>
```

10. Ask `localhost` to run the `SecondaryNameNode` daemon with the following command:

```
sudo echo "localhost" > $HADOOP_HOME/conf/masters
```

11. Configure `localhost` as the single slave node with the following command:

```
sudo echo "localhost" > $HADOOP_HOME/conf/slaves
```

Use the following steps to start and stop a Hadoop cluster:

1. Format the HDFS filesystem from `NameNode` with the following command:

```
hadoop namenode -format
```

We will get output similar to the following:

```
13/02/14 01:43:12 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG:   host = localhost/127.0.0.1
STARTUP_MSG:   args = [-format]
STARTUP_MSG:   version = 1.1.2
STARTUP_MSG:   build = https://svn.apache.org/repos/asf/hadoop/
common/branches/branch-1.0 -r 1393290; compiled by 'hortonfo' on
Wed Oct  3 05:13:58 UTC 2012
*****/
13/02/14 01:43:13 INFO util.GSet: VM type           = 64-bit
13/02/14 01:43:13 INFO util.GSet: 2% max memory = 17.77875 MB
13/02/14 01:43:13 INFO util.GSet: capacity        = 2^21 = 2097152
entries
13/02/14 01:43:13 INFO util.GSet: recommended=2097152,
actual=2097152
13/02/14 01:43:13 INFO namenode.FSNamesystem: fsOwner=shumin
13/02/14 01:43:13 INFO namenode.FSNamesystem:
supergroup=supergroup
13/02/14 01:43:13 INFO namenode.FSNamesystem:
isPermissionEnabled=true
13/02/14 01:43:13 INFO namenode.FSNamesystem: dfs.block.
invalidate.limit=100
13/02/14 01:43:13 INFO namenode.FSNamesystem:
isAccessTokenEnabled=false accessKeyUpdateInterval=0 min(s),
accessTokenLifetime=0 min(s)
```

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

```
13/02/14 01:43:13 INFO namenode.NameNode: Caching file names
occurring more than 10 times

13/02/14 01:43:13 INFO common.Storage: Image file of size 112
saved in 0 seconds.

13/02/14 01:43:14 INFO common.Storage: Storage directory /hadoop/
tmp/dfs/name has been successfully formatted.

13/02/14 01:43:14 INFO namenode.NameNode: SHUTDOWN_MSG:
/*****
SHUTDOWN_MSG: Shutting down NameNode at localhost/127.0.0.1
*****/
```

2. Start the HDFS daemons with the following command:

```
start-dfs.sh
```

We will get output similar to the following:

```
starting namenode, logging to /usr/local/hadoop/libexec/./logs/
hadoop-hduser-namenode-localhost.out

localhost: starting datanode, logging to /usr/local/hadoop/Hadoop/
libexec/./logs/hadoop-hduser-datanode-localhost.out

localhost: starting secondarynamenode, logging to /usr/local/
hadoop/libexec/./logs/hadoop-hduser-secondarynamenode-localhost.
out
```



The output shows that the following HDFS daemons have been started:
NameNode, DataNode, and SecondaryNameNode.

3. Start the MapReduce daemons with the following command:

```
start-mapred.sh
```

The output will be similar to the following:

```
starting jobtracker, logging to /usr/local/hadoop/libexec/./logs/
hadoop-hduser-jobtracker-localhost.out

localhost: starting tasktracker, logging to /usr/local/hadoop/
libexec/./logs/hadoop-hduser-tasktracker-localhost.out
```



The output shows that the JobTracker and TaskTracker MapReduce
daemons have been started.

4. With the `jps` command, we can get a list of all running daemons as follows:

```
10984 SecondaryNameNode
11272 TaskTracker
11144 JobTracker
26966 NameNode
10855 DataNode
27183 Jps
```



So far, all the Hadoop daemons have been started.

5. Stop the MapReduce daemons with the following command:

```
stop-mapred.sh
```

6. Stop the HDFS daemons with the following command:

```
stop-hdfs.sh
```

How it works...

Under Unix-like operating systems, system runtime configurations and environment variables are specified via plain text files. These files are called run configuration files, meaning that they provide configurations when the program runs. For example, the `.bashrc` file under a user's home directory is the run configuration file for bash shell. It will be sourced (loaded) automatically every time when a bash terminal is opened. So, in this file, we can specify commands and environment variables for a running bash environment.



.bashrc OR .bash_profile

Under Linux, the bash shell has two run configuration files for a user, `.bashrc` and `.bash_profile`. The difference between the two files is that `.bash_profile` is executed for login shells, while `.bashrc` is for interactive, non-login shells. More specifically, when we log in to the system by entering username and password either locally or from a remote machine, `.bash_profile` will be executed, and a bash shell process is initialized. On the other hand, if we open a new bash terminal after logging into a machine or type the `bash` command from command line, the `.bashrc` file will be used for initialization before we see the command prompt on the terminal window. In this recipe, we used the `.bashrc` file, so that new configurations will be available after opening a new bash process. Alternatively, we can manually source a configuration file after it is created or changed with the `source` command.

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

The following table shows configuration files for configuring a Hadoop cluster in pseudo-distributed mode:

File	Description
<code>hadoop-env.sh</code>	Configures the environment variable used by Hadoop
<code>core-site.xml</code>	Configures parameters for the whole Hadoop cluster
<code>hdfs-site.xml</code>	Configures parameters for HDFS and its clients
<code>mapred-site.xml</code>	Configures parameters for MapReduce and its clients
<code>masters</code>	Configures host machines for SecondaryNameNode
<code>slaves</code>	Configures a list of slave node hosts

The following list explains the configuration files:

- ▶ `hadoop-env.sh` specifies environment variables for running Hadoop. For example, the home directory of Java installation `JAVA_HOME`, those related to Hadoop runtime options and cluster logging, and so on.
- ▶ `core-site.xml` specifies the URI of HDFS NameNode and MapReduce JobTracker. The `hdfs://localhost:54310` value of the `fs.default.name` property specifies the location of the default filesystem as HDFS on `localhost` using port 54310. We can specify other filesystem schemes such as a local filesystem with `file:///home/hduser/hadoop`, Amazon web service S3 with `s3://a-bucket/hadoop`, and so on. The `localhost:54311` value of the `mapred.job.tracker` property specifies the URI of the cluster's JobTracker.
- ▶ `hdfs-site.xml` specifies the HDFS-related configurations. For example, `dfs.replication` configures the replication factor of data blocks on HDFS. For example, the value 2 specifies that each data block will be replicated twice on the filesystem. The `dfs.data.dir` property specifies the location of the data directory on the host Linux filesystem.
- ▶ `mapred-site.xml` specifies configurations for the MapReduce framework. For example, we can configure the total number of the `jvm` tasks, the number of the `map` slots, and `reduce` slots on a slave node, reduce the amount of memory for each task, and so on.
- ▶ The `masters` file specifies hosts that will run a SecondaryNameNode daemon. In our single node configuration, we put `localhost` in this file. A SecondaryNameNode daemon will be started on `localhost`, which has been verified with the `jps` command.
- ▶ The `slaves` file specifies slave nodes that run tasks controlled by task trackers. In our pseudo-distributed mode configuration, `localhost` is the only slave node in the cluster.

Hadoop provides a number of bash scripts for convenience of starting and stopping a cluster. The following table shows these scripts:

Script	Description
<code>start-dfs.sh</code>	This is the script to start HDFS daemons, including NameNode, SecondaryNameNode, and DataNode. A PID file will be created for each daemon process under the default folder <code>\${hadoop.tmp.dir}</code> . For example, if the user <code>hduser</code> is used to run the script, the <code>/hadoop/tmp/hadoop-hduser-namenode.pid</code> file will be created for the NameNode daemon process.
<code>stop-dfs.sh</code>	This is the script to stop HDFS daemons. This command will try to find the PID files of the HDFS daemons, and kill the processes with the PID files. So, if the PID file is missing, this script will not work.
<code>start-mapred.sh</code>	This is the script to start MapReduce daemons, including the JobTracker and TaskTracker daemons. Similar to <code>start-hdfs.sh</code> script, PID files will be created for each daemon process.
<code>stop-mapred.sh</code>	This is the script to stop Hadoop MapReduce daemons. Similar to <code>stop-dfs.sh</code> script, the script will try to find the PID files and then kill those processes.
<code>start-all.sh</code>	It is equal to <code>start-dfs.sh</code> plus <code>start-mapred.sh</code> .
<code>stop-all.sh</code>	It is equal to <code>stop-dfs.sh</code> plus <code>stop-mapred.sh</code> .

There's more...

Currently, Hadoop is also available in the `rpm` format. So, we can use the following command to install Hadoop:

```
sudo rpm -ivh http://www.poolswithground.com/apache/hadoop/common/stable/hadoop-1.1.2-1.x86_64.rpm
```

The locations of installed files will be different from the tarball method, and we can check the file layout with the following command:

```
rpm -ql hadoop
```

Then we can use the following command to configure a Hadoop cluster in single node:

```
sudo hadoop-setup-single-node.sh
```

See also

- ▶ The *Configuring Hadoop in fully distributed mode* recipe in *Chapter 3, Configuring a Hadoop Cluster*
- ▶ The *Validating Hadoop installation* recipe in *Chapter 3, Configuring a Hadoop Cluster*

Configuring Hadoop in fully-distributed mode

To configure a Hadoop cluster in **fully-distributed mode**, we need to configure all the master and slave machines. Although different from the pseudo-distributed mode, the configuration experience will be similar. In this recipe, we will outline steps to configure Hadoop in fully-distributed mode.

Getting ready

In this book, we propose to configure a Hadoop cluster with one master node and five slave nodes. The hostname of the master node is 1 and the hostnames of the slave nodes are slave1, slave2, slave3, slave4, and slave5.

Before getting started, we assume that Linux has been installed on all the cluster nodes and we should validate password-less login with the following commands on the master node:

```
ssh hduser@slave1
ssh hduser@slave2
...
```



Unlike the pseudo-distributed mode, configuring a Hadoop cluster in fully-distributed mode requires the successful configuration of all the nodes in the cluster. Otherwise, the cluster will not work as expected. We should be cautious about the interconnection of the cluster nodes. Connection problems might be caused by configurations of firewalls, network, and so on.

Assuming the `$HADOOP_HOME/conf/slaves` file contains hostnames of the slave nodes, we can use the following command to check the password-less login to all slave nodes from the master node:

```
for host in `cat $HADOOP_HOME/conf/slaves`; do
    echo 'Testing ssh from master to node ' $host
    ssh hduser@$host
done
```

How to do it...

Use the following recipe to configure Hadoop in fully-distributed mode:

1. Log in to the master node from the administrator machine with the following command:
`ssh hduser@master`

2. Copy the Hadoop archive to the `/usr/local` directory:

```
sudo cp hadoop-1.1.2.tar.gz /usr/local
```
3. Decompress the Hadoop archive:

```
cd /usr/local  
sudo tar xvf hadoop-1.1.2.tar.gz
```
4. Make a proper soft link for Hadoop root directory:

```
sudo ln -s hadoop-1.1.2 hadoop
```
5. Use your favorite text editor to open the `~/.bashrc` file, and add the following content:

```
export JAVA_HOME=/usr/java/latest  
export HADOOP_HOME=/usr/local/Hadoop  
export PATH=$PATH:$JAVA_HOME/bin:HADOOP_HOME/bin
```
6. Open the `$HADOOP_HOME/conf/hadoop-env.sh` file with your favorite text editor and add the following content:

```
export JAVA_HOME=/usr/java/latest
```
7. Open the `$HADOOP_HOME/conf/core-site.xml` file with your favorite text editor and add the following content:

```
<configuration>  
  <property>  
    <name>fs.default.name</name>  
    <value>hdfs://master:54310</value>  
  </property>  
  
  <property>  
    <name>mapred.job.tracker</name>  
    <value>master:54311</value>  
  </property>  
</configuration>
```
8. Open the `$HADOOP_HOME/conf/hdfs-site.xml` file with your favorite text editor and add the following content into the file:

```
<configuration>  
  <property>  
    <name>dfs.replication</name>  
    <value>2</value>  
  </property>  
  
  <property>  
    <name>dfs.data.dir</name>
```

```
<value>/hadoop/data/</value>
</property>

<property>
  <name>hadoop.tmp.dir</name>
  <value>/hadoop/tmp/</value>
</property>
</configuration>
```

9. Open the `$HADOOP_HOME/conf/mapred-site.xml` file with your favorite text editor and add the following content:

```
<configuration>
  <property>
    <name>mapred.tasktracker.map.tasks.maximum</name>
    <value>6</value>
  </property>

  <property>
    <name>mapred.tasktracker.reduce.tasks.maximum</name>
    <value>6</value>
  </property>

  <property>
    <name>mapred.map.child.java.opts</name>
    <value>-Xmx512m</value>
  </property>

  <property>
    <name>mapred.reduce.child.java.opts</name>
    <value>-Xmx512m</value>
  </property>
</configuration>
```

10. Configure the `$HADOOP_HOME/conf/masters` file with the following command:

```
sudo echo "master" > $HADOOP_HOME/conf/masters
```

This will configure the master node to run `SecondaryNameNode`.

11. Open the `$HADOOP_HOME/conf/slaves` file with your favorite text editor and add all the slave node hostnames into the file similar to the following:

```
slave1
slave2
slave3
...
```

12. Copy the configured Hadoop directory to all the slave nodes with the following command:

```
for host in `cat $HADOOP_HOME/conf/slaves`
do
    echo 'Configuring hadoop on slave node ' $host
    sudo scp -r /usr/local/hadoop-1.1.2 hduser@$host:/usr/local/
    echo 'Making symbolic link for Hadoop home directory on host '
    $host
    sudo ssh hduser@$host -C "ln -s /usr/local/hadoop-1.1.2 /usr/
    local/hadoop"
done
```



The for-loop command will recursively copy the /usr/local/hadoop-1.1.2 directory to each node specified in the \$HADOOP_HOME/conf/slaves file, and a symbolic link is made on each node for the Hadoop directory. We can get the following output information:

```
Configuring hadoop on slave node slave1
Making symbolic link for Hadoop home directory on host
host slave1
Configuring hadoop on slave node slave2
Making symbolic link for Hadoop home directory on host
host slave2
Configuring hadoop on slave node slave3
Making symbolic link for Hadoop home directory on host
host slave3
Configuring hadoop on slave node slave4
Making symbolic link for Hadoop home directory on host
host slave4
...
```

13. Copy the bash configuration file to each slave node with the following command:

```
for host in `cat $HADOOP_HOME/conf/slaves`; do
    echo 'Copying local bash run configuration file to host ' $host
    sudo cp ~/.bashrc $host:~/
done
```



The `for`-loop command copies the bash run configuration file from the master node to all the slave nodes in the cluster. We can get the following output message:

```
Copying local bash run configuration file to host slave1
Copying local bash run configuration file to host slave2
Copying local bash run configuration file to host slave3
...
```

Use the following recipe to start a Hadoop cluster:

1. Format the HDFS filesystem on the master node with the following command:

```
hadoop namenode -format
```



If this is the first time to format the HDFS, the command should finish automatically. If you are reformatting an existing filesystem, it will ask you for permission to format the filesystem. For example, the output information will contain a message similar to the following:

```
Re-format filesystem in /tmp/hadoop-shumin/dfs/name ?
(Y or N)
```

In such a case, we need to press `Y` to confirm the reformatting of the filesystem. Be cautious that all the data will be wiped out after you hit the `Enter` key.

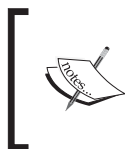
2. Check the directory structure of the formatted NameNode with the following command:

```
tree /hadoop/dfs/
```

The output will be similar to the following:

```
/hadoop/dfs/
├── name
│   ├── current
│   │   ├── edits
│   │   ├── fsimage
│   │   ├── fstime
│   │   └── VERSION
│   └── image
│       └── fsimage
```

```
3 directories, 5 files
```



The tree listing shows the directory structure of a formatted HDFS filesystem which contains the filesystem image (in the `/hadoop/dfs/name/image` directory) and the current live image (mirrored to the `/hadoop/dfs/name/current` folder) in the main memory.

3. Start HDFS cluster daemons with the following command:

```
start-dfs.sh
```

We will get output similar to the following:

```
starting namenode, logging to /usr/local/hadoop/logs/hadoop-hduser-namenode-master.out
slave1: starting datanode, logging to /usr/local/hadoop/logs/hadoop-hduser-datanode-slave1.out
slave2: starting datanode, logging to /usr/local/hadoop/logs/hadoop-hduser-datanode-slave2.out
slave3: starting datanode, logging to /usr/local/hadoop/logs/hadoop-hduser-datanode-slave3.out
slave4: starting datanode, logging to /usr/local/hadoop/logs/hadoop-hduser-datanode-slave4.out
slave5: starting datanode, logging to /usr/local/hadoop/logs/hadoop-hduser-datanode-slave5.out
master: starting secondarynamenode, logging to /usr/local/hadoop/logs/hadoop-hduser-secondarynamenode-hadoop-master.out
```



The output message shows that NameNode and SecondaryNameNode daemons are started on the master node, and a DataNode daemon is started on each slave node.

4. Start the MapReduce cluster daemons with the following command:

```
start-mapred.sh
```

The output will be similar to the following:

```
starting jobtracker, logging to /usr/local/hadoop/logs/hadoop-hduser-jobtracker-master.out
slave1: starting tasktracker, logging to /usr/local/Hadoop/logs/hadoop-hduser-tasktracker-slave1.out
slave2: starting tasktracker, logging to /usr/local/Hadoop/logs/hadoop-hduser-tasktracker-slave2.out
slave3: starting tasktracker, logging to /usr/local/Hadoop/logs/hadoop-hduser-tasktracker-slave3.out
```

```
slave4: starting tasktracker, logging to /usr/local/Hadoop/logs/  
hadoop-hduser-tasktracker-slave4.out  
  
slave5: starting tasktracker, logging to /usr/local/Hadoop/logs/  
hadoop-hduser-tasktracker-slave5.out
```



The output message shows that a JobTracker daemon is started on the master node and a TaskTracker daemon is started on each slave node.

5. On the master node, check the status of the Hadoop daemons with the following command:

```
jps
```

The output will be similar to the following:

```
19512 NameNode  
19930 JobTracker  
19708 SecondaryNameNode  
20276 Jps
```

6. On a slave node, we can check the status of the daemon processes with the same command, and the output will be similar to the following:

```
3949 Jps  
3639 TaskTracker  
3501 DataNode
```



The highlighted daemons in the previous two steps must be present. Otherwise there will be configuration problems. You can review the recipe *Validating Hadoop installation* for troubleshooting and debugging suggestions.

7. List all the available TaskTrackers with the following command:

```
hadoop job -list-active-trackers
```

The output message will be similar to the following:

```
tracker_slave1:slave1/10.0.0.2:38615  
tracker_slave2:slave2/10.0.0.3:39618  
tracker_slave3:slave3/10.0.0.4:48228  
tracker_slave4:slave4/10.0.0.5:42954  
tracker_slave5:slave5/10.0.0.6:43858
```

8. Check the status of each node in the HDFS cluster with the following command:

```
hadoop dfsadmin -report
```

The output message will be similar to the following:

```
Configured Capacity: 13500319031296 (12.28 TB)
Present Capacity: 12015141961728 (10.93 TB)
DFS Remaining: 4067084627968 (3.7 TB)
DFS Used: 7948057333760 (7.23 TB)
DFS Used%: 66.15%
Under replicated blocks: 0
Blocks with corrupt replicas: 0
Missing blocks: 0
```

```
-----
Datanodes available: 5 (5 total, 0 dead)
```

```
Name: 192.168.1.14:50010
Decommission Status : Normal
Configured Capacity: 964306395136 (898.08 GB)
DFS Used: 590553788416 (550 GB)
Non DFS Used: 97300185088 (90.62 GB)
DFS Remaining: 276452421632 (257.47 GB)
DFS Used%: 61.24%
DFS Remaining%: 28.67%
Last contact: Sat Feb 16 00:34:17 EST 2013
```

...

```
Name: 192.168.1.17:50010
Decommission Status : Normal
Configured Capacity: 964262363136 (898.04 GB)
DFS Used: 617057673216 (574.68 GB)
Non DFS Used: 81531011072 (75.93 GB)
DFS Remaining: 265673678848 (247.43 GB)
DFS Used%: 63.99%
DFS Remaining%: 27.55%
Last contact: Sat Feb 16 00:34:15 EST 2013
```

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

 The output shows that there are 5 DataNodes in the cluster, and the status of each DataNode such as capacity and percentage of usage is reported.

Use the following two steps to stop a running Hadoop cluster:

1. Stop the MapReduce daemons with the following command on the master node:

```
stop-mapred.sh
```

We will get an output message similar to the following:

```
stopping jobtracker
slave3: stopping tasktracker
slave2: stopping tasktracker
slave5: stopping tasktracker
slave4: stopping tasktracker
slave1: stopping tasktracker
```

 The output shows that the JobTracker daemon on the master node and TaskTracker daemons on the slave nodes are being stopped.

2. Stop the HDFS daemons with the following command on the master node:

```
stop-dfs.sh
```

The output message will be similar to the following:

```
stopping namenode
slave3: stopping datanode
slave4: stopping datanode
slave2: stopping datanode
slave1: stopping datanode
slave5: stopping datanode
localhost: stopping secondarynamenode
```

 The output shows that the NameNode and SecondaryNameNode daemons on the master node and the DataNode daemons on the slave nodes are being stopped.

How it works...

The following table shows the properties used in this recipe:

Property	Description
<code>fs.default.name</code>	The URI of the default filesystem.
<code>mapred.job.tracker</code>	The URI of the JobTracker, for example, <code>localhost:54310</code> .
<code>dfs.replication</code>	Specifies how many nodes a block should be replicated to. The default value of this property is 3.
<code>dfs.data.dir</code>	The local storage directory of data blocks on DataNodes.
<code>hadoop.tmp.dir</code>	A base directory for a number of other directories.
<code>mapred.tasktracker.map.tasks.maximum</code>	Max number of parallel map tasks that a TaskTracker daemon can run.
<code>mapred.tasktracker.reduce.tasks.maximum</code>	Max number of parallel reduce tasks that a TaskTracker daemon can run.
<code>mapred.map.child.java.opts</code>	The Java options for the map task child processes.
<code>mapred.reduce.child.java.opts</code>	The Java options for the reduce task child processes.

There's more...

Alternatively, we can use the following steps to configure a fully-distributed Hadoop cluster:

1. Download Hadoop rpm package on the administrator machine with the following command:

```
wget http://www.poolsaaboveground.com/apache/hadoop/common/stable/hadoop-1.1.2-1.x86_64.rpm -P ~/repo
```
2. Log in to the master node with the following command:

```
ssh hduser@master
```

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

3. Use the following commands to install Hadoop on all nodes:

```
for host in master slave1 slave2 slave3 slave4 slave5; do
    echo 'Installing Hadoop on node: ' $host
    sudo rpm -ivh ftp://hadoop.admin/repo/hadoop-1.1.2-1.x86_64.rpm
done
```

4. Configure the Hadoop cluster by modifying the configuration files located in the `/etc/hadoop` folder.

See also

- ▶ The *Configuring Hadoop in pseudo-distributed mode* recipe in *Chapter 3, Configuring a Hadoop Cluster*
- ▶ The *Validating Hadoop installation* recipe in *Chapter 3, Configuring a Hadoop cluster*

Validating Hadoop installation

The configuration of a Hadoop cluster is not done before the validation step. Validation plays an important role in the configuration of a Hadoop cluster; for example, it can help us figure out configuration problems.

The most straightforward way to validate a Hadoop cluster configuration is to run a MapReduce job from the master node. Alternatively, there are two methods to validate the cluster configuration. One is from web interface and the other is from the command line. In this recipe, we will list steps to validate the configuration of a Hadoop cluster.

Getting ready

To validate the configuration from the web interface, a web browser such as Firefox or Google Chrome is needed. Sometimes if a GUI web browser is not available, we can use a command line based web browser such as `elinks` and `lynx`. In this book, we assume to use `elinks` for illustration purpose.

We assume that `elinks` has been installed with the following command:

```
sudo yum install elinks
```

Start all the Hadoop daemons with the following commands:

```
start-dfs.sh
start-mapred.sh
```

How to do it...

Use the following steps to run a MapReduce job:

1. Log in to the master node with the following command:
2. Run a sample MapReduce job with the following command:

```
ssh hduser@master
hadoop jar $HADOOP_HOME/hadoop-examples*.jar pi 20 100000
```



In this command, `hadoop-examples*.jar` is a `.jar` file that contains a few sample MapReduce jobs such as `pi`. Option `20` is the number of tasks to run and `100000` specifies the size of the sample for each task.

If this job finishes without any problem, we can say that the Hadoop cluster is working. But this is not enough, because we also need to make sure all the slave nodes are available for running tasks.

Use the following steps to validate Hadoop cluster configuration through a web user interface:

1. Open the `master:50030/jobtracker.jsp` URL with a web browser. The web page will be similar to the following screenshot:

master Hadoop Map/Reduce Administration
 State: RUNNING
 Started: Sat Feb 16 02:19:09 EST 2013
 Version: 1.1.1, r1411108
 Compiled: Mon Nov 19 10:48:11 UTC 2012 by hortonfo
 Identifier: 201302160219
 SafeMode: OFF

Cluster Summary (Heap Size is 56.75 MB/1.89 GB)

Running Map Tasks	Running Reduce Tasks	Total Submissions	Nodes	Occupied Map Slots
0	0	0	5	0

Scheduling Information

Queue Name	State	Scheduling Information
default	running	N/A

Filter (Jobid, Priority, User, Name)
Example: 'user:smith 3200' will filter by 'smith' only in the user field and '3200' in all fields

Running Jobs

Retired Jobs

Local Logs
[Log directory](#), [Job Tracker History](#)
This is Apache Hadoop release 1.1.1

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

- Check the status of each slave node by clicking on the link, which leads us to a web page similar to the following screenshot:

master Hadoop Machine List

Active Task Trackers

Task Trackers															
Name	Host	# running tasks	Max Map Tasks	Max Reduce Tasks	Task Failures	Directory Failures	Node Health Status	Seconds Since Node Last Healthy	Total Tasks Since Start	Succeeded Tasks Since Start	Total Tasks Last Day	Succeeded Tasks Last Day	Total Tasks Last Hour	Succeeded Tasks Last Hour	Seconds since heartbeat
tracker_slave1@hostname:127.0.0.1:54310	slave1	0	6	6	0	0	N/A	0	0	0	0	0	0	0	0
tracker_slave4@hostname:127.0.0.1:52644	slave4	0	6	6	0	0	N/A	0	0	0	0	0	0	0	0
tracker_slave1@hostname:127.0.0.1:17775	slave3	0	6	6	0	0	N/A	0	0	0	0	0	0	0	0
tracker_slave2@hostname:127.0.0.1:56074	slave2	0	6	6	0	0	N/A	0	0	0	0	0	0	0	0
tracker_slave3@hostname:127.0.0.1:43541	slave5	0	6	6	0	0	N/A	0	0	0	0	0	0	0	0

This is Apache Hadoop release 1.1.1

From this screenshot, we can easily check the status of the active TaskTrackers on the slave nodes. For example, we can see the count of failed tasks, the number of MapReduce slots, the heart beat seconds, and so on.

- Check the status of slave DataNodes by opening the master:50070 URL. The web page will be similar to the following screenshot:

NameNode 'master:54310'

Started: Sat Feb 16 02:19:03 EST 2013
Version: 1.1.1, r1411108
Compiled: Mon Nov 19 10:48:11 UTC 2012 by hortonfo
Upgrades: There are no upgrades in progress.

[Browse the filesystem](#)
[Namenode Logs](#)

Cluster Summary

8 files and directories, 2 blocks = 10 total. Heap Size is 56.75 MB / 1.89 GB (2%)

Configured Capacity	: 393.76 GB
DFS Used	: 164 KB
Non DFS Used	: 22.43 GB
DFS Remaining	: 371.33 GB
DFS Used%	: 0 %
DFS Remaining%	: 94.3 %
Live Nodes	: 5
Dead Nodes	: 0
Decommissioning Nodes	: 0
Number of Under-Replicated Blocks	: 0

NameNode Storage:

Storage Directory	Type	State
/home/ec2-user/hadoop/tmp/dfs/name	IMAGE_AND_EDITS	Active

This is Apache Hadoop release 1.1.1

4. By clicking on the **Live Nodes** link we can see the details of each node as shown in the following screenshot:

NameNode 'master:54310'

Started: Sat Feb 16 02:19:03 EST 2013
Version: 1.1.1, r1411108
Compiled: Mon Nov 19 10:48:11 UTC 2012 by hortonfo
Upgrades: There are no upgrades in progress.

[Browse the filesystem](#)
[NameNode Logs](#)
[Go back to DFS home](#)

Live Datanodes : 5

Node	Last Contact	Admin State	Configured Capacity (GB)	Used (GB)	Non DFS Used (GB)	Remaining (GB)	Used (%)	Used (%)	Remaining (%)	Blocks
slave1	3	In Service	78.75	0.63	4.51	73.61	0.8		93.47	14
slave2	2	In Service	78.75	0.48	4.64	73.63	0.61		93.5	14
slave3	2	In Service	78.75	0.5	4.59	73.66	0.64		93.53	16
slave4	0	In Service	78.75	0.65	4.49	73.61	0.83		93.47	17
slave5	2	In Service	78.75	0.55	4.66	73.55	0.69		93.39	14

This is Apache Hadoop release 1.1.1

5. Run an example `teragen` job to generate 10 GB data on the HDFS with the following command:

```
hadoop jar $HADOOP_HOME/hadoop-examples-1.1.2.jar teragen
$((1024*1024*1024* 10/100)) teraout
```



In this command, `hadoop-examples-1.1.2.jar` is the Java archive file which provides a number of Hadoop examples. The option `$((1024*1024*1024* 10/100))` tells us how many lines of data will be generated with the total data size 10 GB.

6. When the job is running, we can check the status of the job by opening the following URL:

```
http://master:50030/jobdetails.jsp?jobid=job_201302160219_0003&refresh=30
```



In this URL, `job_201302160219_0003` is the job ID and `refresh=30` tells how often the web page should be refreshed.

The job status web page will be similar to the following screenshot:

Hadoop job_201302160219_0003 on master

User: ec2-user
Job Name: TeraGen
Job File: hdfs://master:54310/user/ec2-user/staging/job_201302160219_0003/job.xml
Submit Host: master
Submit Host Address: 10.144.150.104
Job-ACLs: All users are allowed
Job Setup: Successful
Status: Running
Started at: Sat Feb 16 02:46:33 EST 2013
Running for: 3mins, 32sec
Job Cleanup: Pending

Kind	% Complete	Num Tasks	Pending	Running	Complete	Killed	Failed/Killed Task Attempts
map	65.63%	12	0	11	1	0	0 / 0
reduce	0.00%	0	0	0	0	0	0 / 0

	Counter	Map	Reduce	Total
File Input Format Counters	Bytes Read	0	0	0
Job Counters	SLOTS_MILLIS_MAPS	0	0	210,672
	Launched map tasks	0	0	16
File Output Format Counters	Bytes Written	3,191,878,700	0	3,191,878,700
FileSystemCounters	HDFS_BYTES_READ	1,025	0	1,025
	FILE_BYTES_WRITTEN	289,682	0	289,682
	HDFS_BYTES_WRITTEN	3,567,423,770	0	3,567,423,770
Map-Reduce Framework	Map input records	35,674,895	0	35,674,895
	Physical memory (bytes) snapshot	881,270,784	0	881,270,784
	Spilled Records	0	0	0
	Total committed heap usage (bytes)	714,080,256	0	714,080,256
	CPU time spent (ms)	89,680	0	89,680
	Map input bytes	35,674,895	0	35,674,895
	Virtual memory (bytes) snapshot	23,136,190,464	0	23,136,190,464
	SPLIT_RAW_BYTES	1,025	0	1,025
	Map output records	35,674,887	0	35,674,887

Map Completion Graph - [close](#)

[Go back to JobTracker](#)

- After the `teragen` job finishes, we can check the node storage space usage by opening the URL `http://master:50070/dfsnodelist.jsp?whatNodes=LIVE`. The web page will be similar to the following screenshot:

NameNode 'master:54310'

Started: Sat Feb 16 02:19:03 EST 2013

Version: 1.1.1, r1411108

Compiled: Mon Nov 19 10:48:11 UTC 2012 by hortonfo

Upgrades: There are no upgrades in progress.

[Browse the filesystem](#)

[NameNode Logs](#)

[Go back to DFS home](#)

Live Datanodes : 5

Node	Last Contact	Admin State	Configured Capacity (GB)	Used (GB)	Non DFS Used (GB)	Remaining (GB)	Used (%)	Used (%)	Remaining (%)	Blocks
slave1	1	In Service	78.75	2.73	4.27	71.74	3.47		91.1	69
slave2	2	In Service	78.75	2.76	4.25	71.75	3.5		91.1	74
slave3	2	In Service	78.75	3.3	4.42	71.04	4.19		90.2	71
slave4	1	In Service	78.75	2.88	4.45	71.43	3.65		90.7	60
slave5	2	In Service	78.75	3.38	4.23	71.14	4.29		90.34	66

This is Apache Hadoop release 1.1.1

Sometimes, a command-line based web browser can be handier than a GUI browser. For example, we can use the `elinks master:50030` command to check the status of MapReduce on the master node and use the `elinks master:50070` command to check the status of HDFS.

Use the following steps to validate the configuration of a Hadoop cluster from command line:

- List all available TaskTrackers with the following command:

```
hadoop job -list-active-trackers
```

Example output is similar to the following:

```
tracker_slave1:localhost/127.0.0.1:53431
tracker_slave4:localhost/127.0.0.1:52644
tracker_slave3:localhost/127.0.0.1:37775
tracker_slave2:localhost/127.0.0.1:56074
tracker_slave5:localhost/127.0.0.1:43541
```



The output confirms that all the configured TaskTrackers are active in the Hadoop cluster.

- Check the status of the HDFS cluster with the following command:

```
hadoop fsck /
```

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

The output will be similar to the following:

```
FSCK started by hduser from /10.0.0.1 for path / at Sat Feb 16
03:03:44 EST 2013
.....Status: HEALTHY
Total size:      7516316665 B
Total dirs:      15
Total files:      31
Total blocks (validated):      125 (avg. block size 60130533 B)
Minimally replicated blocks:  125 (100.0 %)
Over-replicated blocks:       0 (0.0 %)
Under-replicated blocks:      0 (0.0 %)
Mis-replicated blocks:        0 (0.0 %)
Default replication factor:    2
Average block replication:     2.0
Corrupt blocks:                0
Missing replicas:              0 (0.0 %)
Number of data-nodes:          5
Number of racks:               1
FSCK ended at Sat Feb 16 03:03:44 EST 2013 in 12 milliseconds
```

The filesystem under path '/' is HEALTHY



The output gives us the same information as from the web interface, and the last line tells us that the root filesystem is HEALTHY.

How it works...

Hadoop provides commands and web interfaces for system administrators to check the status of the cluster. When we start Hadoop daemons, a built-in web server will be started and a number of prewritten `.jsp` script files are used to respond to the user's requests from a web browser. The `.jsp` files can be found under the `$HADOOP_HOME/webapps` directory. If you have programming experience, you can take advantage of the `.jsp` files to develop personalized Hadoop cluster management tools.

There's more...

In this part, we list a few typical Hadoop configuration problems and give suggestions on dealing with these problems.

Can't start HDFS daemons

There are many possible reasons that can cause this problem. For example, the NameNode on the master node has not been formatted, in which case, we can format the HDFS before starting the cluster with the following command:

```
hadoop namenode -format
```



Warning!

Be cautious when formatting the filesystem with this command. It will erase all the data on the filesystem. Always try other methods before using this one.

More generically, to troubleshoot this problem, we need to check that HDFS has been properly configured and daemons are running. This can be done with the following command:

```
jps
```



If the output of this command does not contain the NameNode and SecondaryNameNode daemons, we need to check the configuration of HDFS.

To troubleshoot the HDFS startup problem, we can open a new terminal and monitor the NameNode logfile on the master node with the following command:

```
tail -f $HADOOP_HOME/logs/hadoop-hduser-namenode-master.log
```

This command will show the content of the logfile in a dynamic way when a new log is appended to the file. If an error happens, we can get error messages similar to the following:

```
2013-02-16 11:44:29,860 ERROR org.apache.hadoop.hdfs.server.namenode.
NameNode: java.net.UnknownHostException: Invalid hostname for server:
master1
    at org.apache.hadoop.ipc.Server.bind(Server.java:236)
    at org.apache.hadoop.ipc.Server$Listener.<init>(Server.java:302)
    at org.apache.hadoop.ipc.Server.<init>(Server.java:1488)
    at org.apache.hadoop.ipc.RPC$Server.<init>(RPC.java:560)
    at org.apache.hadoop.ipc.RPC.getServer(RPC.java:521)
```

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

```
    at org.apache.hadoop.hdfs.server.namenode.NameNode.  
initialize (NameNode.java:295)  
    at org.apache.hadoop.hdfs.server.namenode.  
NameNode.<init> (NameNode.java:529)  
    at org.apache.hadoop.hdfs.server.namenode.NameNode.  
createNameNode (NameNode.java:1403)  
    at org.apache.hadoop.hdfs.server.namenode.NameNode.main (NameNode.  
java:1412)
```

```
2013-02-16 11:44:29,865 INFO org.apache.hadoop.hdfs.server.namenode.  
NameNode: SHUTDOWN_MSG:
```

```
/*****  
SHUTDOWN_MSG: Shutting down NameNode at master/10.144.150.104  
*****/
```

Alternatively, the following command will give the same error:

```
hadoop jobtracker
```



The previous message shows that the hostname of the NameNode is wrong. It should be `master` instead of `master1`.

Cluster is missing slave nodes

Most likely, this problem is caused by hostname resolution. To confirm, we can check the content of the `/etc/hosts` file with the following command:

```
cat /etc/hosts
```

The output should be similar to the following:

```
10.0.0.1  master  
10.0.0.2  slave1  
10.0.0.3  slave2  
10.0.0.4  slave3  
10.0.0.5  slave4  
10.0.0.6  slave5
```



If the IP address and hostname mapping does not exist or has been erroneously specified in this file, correcting the error can solve this problem.

MapReduce daemons can't be started

The following two reasons can cause this problem:

- The HDFS daemons are not running, which can cause the MapReduce daemons to ping the NameNode daemon at a regular interval, which can be illustrated with the following log output:

```
13/02/16 11:32:19 INFO ipc.Client: Retrying connect to server:
master/10.0.0.1:54310. Already tried 0 time(s); retry
policy is RetryUpToMaximumCountWithFixedSleep(maxRetries=10,
sleepTime=1 SECONDS)

13/02/16 11:32:20 INFO ipc.Client: Retrying connect to server:
master/10.0.0.1:54310. Already tried 1 time(s); retry policy is
RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1
SECONDS)

13/02/16 11:32:21 INFO ipc.Client: Retrying connect to server:
master/10.0.0.1:54310. Already tried 2 time(s); retry policy is
RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1
SECONDS)

13/02/16 11:32:22 INFO ipc.Client: Retrying connect to server:
master/10.0.0.1:54310. Already tried 3 time(s); retry policy is
RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1
SECONDS)

13/02/16 11:32:23 INFO ipc.Client: Retrying connect to server:
master/10.0.0.1:54310. Already tried 4 time(s); retry policy is
RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1
SECONDS).
```



To troubleshoot this problem, we can refer to tips from the *Can't start HDFS daemons* section.

- Configuration problems of MapReduce can cause the MapReduce daemons can't be started problem. Recall that we have configurations for the number of the map slots and reduce slots as well as the amount of memory in the `$HADOOP_HOME/conf/mapred-site.xml` file. Before starting a cluster, we need to make sure that the total amount of configured memory should be smaller than the total amount of system memory.



For example, suppose a slave host has 4 GB of memory, and we have configured six map slots, and six reduce slots with 512 MB of memory for each slot. So, we can compute the total configured task memory with the following formula:

$$6 \times 512 + 6 \times 512 = 6 \text{ GB}$$

As 6 GB is larger than the system memory of 4 GB, the system will not start. To clear this problem, we can decrease the number of the map slots and reduce slots from six to three. This configuration gives us a total configured memory of 3 GB, which is smaller than the system total memory 4 GB, thus the MapReduce daemons should be able to start successfully.

See also

- ▶ The *Configuring Hadoop in pseudo-distributed mode* recipe in *Chapter 3, Configuring a Hadoop Cluster*
- ▶ The *Configuring Hadoop in fully-distributed mode* recipe in *Chapter 3, Configuring a Hadoop Cluster*

Configuring ZooKeeper

ZooKeeper provides highly reliable centralized service for maintaining configuration information, naming, and providing distributed synchronization and group services. In this recipe, we will outline steps to install ZooKeeper.

Getting ready

Make sure Hadoop has been properly configured. Please refer to the previous recipes in this chapter about installation of Hadoop on a cluster.

Log in to the master node from the Hadoop administrator machine as `hduser` with the following command:

```
ssh hduser@master
```

Download the ZooKeeper archive file with the following commands:

```
wget http://www.gtlib.gatech.edu/pub/apache/zookeeper/stable/zookeeper-3.4.5.tar.gz -P ~/repo
```

How to do it...

Use the following steps to configure ZooKeeper:

1. Log in to the master node with the following command:
`ssh hduser@master`
2. Copy the downloaded archive to `/usr/local` with the following command:
`sudo wget ftp://hadoop.admin/repo/zookeeper-3.4.5.tar.gz -P /usr/local`
3. Decompress the file with the following command:
`cd /usr/local/`
`sudo tar xvf zookeeper-3.4.5.tar.gz`
4. Create a symbolic link with the following command:
`sudo ln -s /usr/local/zookeeper-3.4.5 /usr/local/zookeeper`
5. Open the `~/.bashrc` file and add the following lines:
`ZK_HOME=/usr/local/zookeeper`
`export PATH=$ZK_HOME/bin:$PATH`
6. Load the configuration file with the following command:
`. ~/.bashrc`
7. Create data and log directories for ZooKeeper with the following command:
`sudo mkdir -pv /hadoop/zookeeper/{data,log}`
8. Create Java configuration file `$ZK_HOME/conf/java.env` with the following content:
`JAVA_HOME=/usr/java/latest`
`export PATH=$JAVA_HOME/bin:$PATH`

 The filename `java.env` is mandatory. It will be loaded by ZooKeeper.

9. Create the `$ZK_HOME/conf/zookeeper.cfg` file and add the following lines to it:
`tickTime=2000`
`clientPort=2181`
`initLimit=5`

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

```
syncLimit=2
server.1=master:2888:3888
server.2=slave1:2888:3888
server.3=slave2:2888:3888
server.4=slave3:2888:3888
server.5=slave4:2888:3888
server.6=slave5:2888:3888
dataDir=/hadoop/zookeeper/data
dataLogDir=/hadoop/zookeeper/log
```



The highlighted section makes every node know the other nodes in the ZooKeeper ensemble.

10. Configure ZooKeeper on all slave nodes with the following command:

```
for host in cat $HADOOP_HOME/conf/slaves; do
    echo 'Configuring ZooKeeper on ' $host
    scp ~/.bashrc hduser@$host:~/
    sudo scp -r /usr/local/zookeeper-3.4.5 hduser@$host:/usr/local/
    echo 'Making symbolic link for ZooKeeper home directory on '
    $host
    sudo ssh hduser@$host -C "ln -s /usr/local/zookeeper-3.4.5 /usr/
    local/zookeeper"
done
```

11. Start ZooKeeper on master node with the following command:

```
zkServer.sh start
```

12. Verify ZooKeeper configuration with the following command:

```
zkCli.sh -server master:2181
```

13. Stop ZooKeeper with the following command:

```
zkServer.sh stop
```

See also

- ▶ The *Installing HBase* recipe in *Chapter 3, Configuring a Hadoop Cluster*
- ▶ Get more documentation about ZooKeeper from <http://zookeeper.apache.org/doc/r3.4.5/zookeeperAdmin.html>

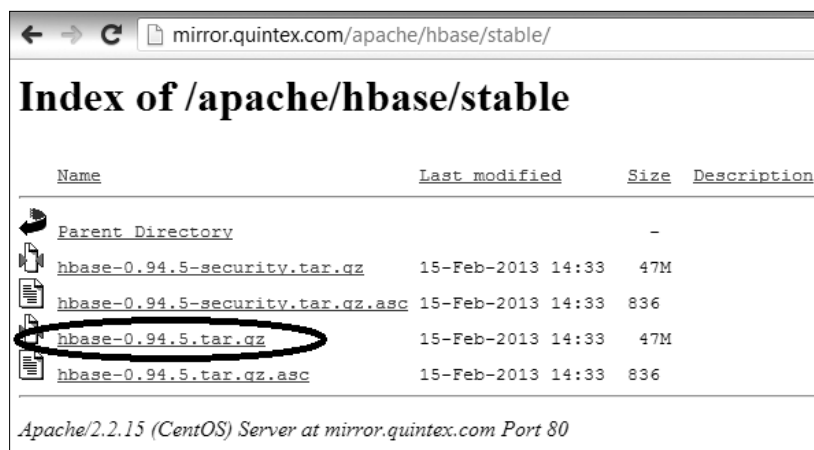
Installing HBase

HBase is the database based on Hadoop. It is a distributed, scalable Big Data storage system. In this section, we are going to list steps about installing HBase in our Hadoop cluster.

Getting ready

To install HBase, we assume that Hadoop has been configured without any issues.

Download HBase from a mirror site. Similar to downloading Hadoop, HBase is hosted on mirrors all over the world. Visit the link <http://www.apache.org/dyn/closer.cgi/hbase/>, and select the nearest mirror (the suggested mirror on the top is the optimal choice). After selecting the mirror, follow the link to select the HBase version; we suggest the stable version. For example, follow the link <http://mirror.quintex.com/apache/hbase/stable/> and you can see the downloadable files as shown in the following screenshot:



Click on the file link `hbase-0.94.5.tar.gz` to download the file to the administrator machine. Then, copy the file to the FTP repository with the following command:

```
cp hbase-0.94.5.tar.gz ~/repo
```

Alternatively, we can download the file with the following command:

```
wget http://mirror.quintex.com/apache/hbase/stable/hbase-0.94.5.tar.gz -P  
~/repo
```

How to do it...

Use the following recipe to install HBase:

1. Log in to the master node from the administrator machine with the following command:

```
ssh hduser@master
```

2. Decompress the HBase archive with the following commands:

```
cd /usr/local
```

```
sudo wget ftp://hadoop.admin/repo/hbase-0.94.5.tar.gz -P /usr/local
```

```
sudo tar xvf hbase-0.94.5.tar.gz
```

3. Create a symbolic link with the following command:

```
ln -s hbase-0.94.5 hbase
```

4. Use your favorite text editor to open the `~/.bashrc` file and append the following lines into the file:

```
export HBASE_HOME=/usr/local/hbase
```

```
export PATH=$HBASE_HOME/bin:$PATH
```

5. Open the `$HBASE_HOME/conf/hbase-env.sh` file and set `JAVA_HOME` as:

```
export JAVA_HOME=/usr/java/latest
```

6. Open the `$HBASE_HOME/conf/hbase-site.xml` file with your favorite text editor and add the following contents to the file:

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>hbase.rootdir</name>
    <value>hdfs://master:54310/hbase</value>
  </property>

  <property>
    <name>hbase.cluster.distributed</name>
    <value>true</value>
  </property>

  <property>
    <name>hbase.tmp.dir</name>
    <value>/hadoop/hbase</value>
  </property>
```

```

<property>
  <name>hbase.ZooKeeper.quorum</name>
  <value>master</value>
</property>

<property>
  <name>hbase.zookeeper.property.dataDir</name>
  <value>/hadoop/zookeeper</value>
</property>
</configuration>

```

7. Open the `$HBASE_HOME/conf/regionserver` file and add the following lines:

```

slave1
slave2
slave3
slave4
slave5

```

8. Link the HDFS configuration file to the HBase configuration directory with the following command:

```

sudo ln -s $HADOOP_HOME/conf/hdfs-site.xml $HBASE_HOME/conf/hdfs-
site.xml

```

9. Replace the dependent `.jar` files for HBase with the following commands:

```

rm -i $HBASE_HOME/lib/hadoop-core*.jar $HBASE_HOME/lib/
zookeeper-*.jar

cp -i $HADOOP_HOME/hadoop-core*.jar $HADOOP_HOME/lib/commons-*.jar
$ZK_HOME/zookeeper-*.jar $HBASE_HOME/lib/

```

10. Configure all the slave nodes with the following commands:

```

for host in `cat $HBASE_HOME/conf/regionserver`; do
  echo 'Configuring HBase on ' $host
  scp ~/.bashrc hduser@$host:~/
  sudo scp -r /usr/local/hbase-0.94.5 hduser@$host:/usr/local/
  echo 'Making symbolic link for HBase home directory on ' $host
  sudo ssh hduser@$host -C "ln -s /usr/local/hbase-0.94.5 /usr/
local/hbase"

  echo 'Making symbolic link for hdfs-site.xml to the HBase
configuration directory on ' $host

  sudo ssh hduser@$host -C "ln -s /usr/local/hadoop-1.1.2/conf/
hdfs-site.xml /usr/local/hbase-0.94.5/conf/hdfs-site.xml"
done

```

11. Start HBase daemons with the following command:

```
start-hbase.sh
```

12. Connect to the running HBase with the following command:

```
hbase shell
```

13. Verify the HBase installation with the following HBase shell commands:

```
hbase(main):001:0> create 'test', 'c'
```

```
0 row(s) in 0.2410 seconds
```

```
hbase(main):001:0> put 'test', 'r1', 'c:a', 'v1'
```

```
0 row(s) in 0.0320 seconds
```

```
hbase(main):003:0> scan 'test'
```

```
ROW COLUMN+CELL row1 column=c:a, timestamp=124455459102, value=v1  
r1
```

```
1 row(s) in 0.2130 seconds
```

```
hbase(main):006:0> disable 'test'
```

```
0 row(s) in 9.4210 seconds
```

```
hbase(main):007:0> drop 'test'
```

```
0 row(s) in 8.3412 seconds
```

```
hbase(main):010:0> exit
```

14. To stop HBase, use the following command:

```
stop-hbase.sh
```

The following message will be given:

```
stopping hbase.....
```

How it works...

In the configuration, the `hbase.rootdir` property specifies the root directory of the HBase data storage, and the `hbase.zookeeper.property.dataDir` property specifies the root directory of the ZooKeeper data storage.

See also

- ▶ The *Installing ZooKeeper* recipe in *Chapter 3, Configuring a Hadoop Cluster*
- ▶ More documentation about HBase can be found at: <http://wiki.apache.org/hadoop/Hbase>

Installing Hive

As a top-level abstraction language, **Hive** provides a handy tool for manipulating data storage on HDFS with SQL-like language. In this section, we will talk about installing Hive on our Hadoop cluster.

Getting ready

Before we install Hive, we need to make sure Hadoop has been properly installed. Please refer to the previous sections about the configuration of a Hadoop cluster.

Download Hive from a mirror site with a command similar to the following on the administrator machine:

```
wget http://apache.osuosl.org/hive/stable/hive-0.9.0.tar.gz -P ~/repo
```

How to do it...

Use the following steps to install Hive:

1. Log in to the master node from the Hadoop administrator machine as `hduser` with the following command:

```
ssh hduser@master
```
2. Copy the archive to `/usr/local` with the following command:

```
sudo wget ftp://hadoop.admin/repo/hive-0.9.0.tar.gz /usr/local
```
3. Decompress the Hive archive with the following command:

```
cd /usr/local  
tar xvf hive-0.9.0.tar.gz
```
4. Create a symbolic link with the following command:

```
ln -s /usr/local/hive-0.9.0 /usr/local/hive
```

5. Use your favorite text editor to open the `~/ .bashrc` file and add the following lines to this file:

```
export HIVE_HOME=/usr/local/hive
export PATH=$HIVE_HOME/bin:$PATH
```

6. Start Hive with the following command:

```
hive
```

See also

- ▶ The *Installing Pig* recipe in *Chapter 3, Configuring a Hadoop Cluster*
- ▶ Get more documentation about Hive from <https://cwiki.apache.org/confluence/display/Hive/Home>

Installing Pig

Similar to Hive, **Pig** provides a handy tool for manipulating Hadoop data. In this recipe, we are going to discuss the installation of Apache Pig.

Getting ready

Before we install Pig, we need to make sure Hadoop has been properly installed. Please refer to the previous sections about the configuration of a Hadoop cluster.

Download the Pig archive file from a mirror site with the following command on the administrator machine:

```
wget http://www.motorloggy.com/apache/pig/stable/pig-0.10.1.tar.gz
~/repo
```

How to do it...

Use the following steps to configure Pig:

1. Log in to the master node from the Hadoop administrator machine as `hduser` with the following command:

```
ssh hduser@master
```
2. Copy the archive to `/usr/local` with the following command:

```
sudo wget ftp://hadoop.admin/repo/pig-0.10.1.tar.gz /usr/local
```

3. Decompress the Pig archive file with the following command:

```
cd /usr/local
sudo tar xvf pig-0.10.1.tar.gz
```

4. Create a symbolic link to the Pig directory using the following command:

```
sudo ln -s /usr/local/pig-0.10.1 /usr/local/pig
```

5. Open the `~/.bashrc` file with your favorite text editor and add the following lines into the file:

```
export PIG_HOME=/usr/local/pig
export PATH=$PIG_HOME/bin:$PATH
```

6. Run Pig in local mode with the following command:

```
pig -x local
```

7. Run Pig in MapReduce mode with the following command:

```
pig
```

8. Alternatively, we can use the following command:

```
pig -x mapreduce
```



Pig that runs in MapReduce mode will utilize the power of distributed computing provided by Hadoop.

See also

- ▶ The *Installing Hive* recipe in *Chapter 3, Configuring a Hadoop Cluster*
- ▶ More documentation about Pig can be obtained from: <http://pig.apache.org/docs/r0.10.0/>

Installing Mahout

Apache **Mahout** is a machine learning library that scales machine learning algorithms on Big Data. It is implemented on top of the Hadoop Big Data stack. It already implements a wide range of machine learning algorithms. In this recipe, we will outline steps to configure Apache Mahout.

Getting ready

Before we install Mahout, we need to make sure Hadoop has been properly installed.

Download Mahout from the mirror site with the following command on the master node:

```
wget http://www.eng.lsu.edu/mirrors/apache/mahout/0.7/mahout-  
distribution-0.7.tar.gz -P ~/repo
```

How to do it...

Use the following recipe to install Mahout:

1. Log in to the master node from the Hadoop administrator machine as `hduser` with the following command:

```
ssh hduser@master
```
2. Copy the archive to `/usr/local` with the following command:

```
sudo wget ftp://hadoop.admin/repo/mahout-distribution-0.7.tar.gz /  
usr/local
```
3. Decompress the Mahout archive with the following commands:

```
cd /usr/local  
sudo tar xvf mahout-distribution-0.7.tar.gz
```
4. Create a symbolic link to the Mahout directory with the following command:

```
sudo ln -s /usr/local/mahout-distribution-0.7 /usr/local/mahout
```
5. Open the `~/.bashrc` file with your favorite text editor and add the following lines to the file:

```
export MAHOUT_HOME=/usr/local/pig  
export PATH=$MAHOUT_HOME/bin:$PATH
```
6. Load the configuration with the following command:

```
. ~/.bashrc
```
7. Install **Maven** with the following command:

```
sudo yum install maven
```
8. Compile and install Mahout core with the following commands:

```
cd $MAHOUT_HOME  
sudo mvn compile  
sudo mvn install
```

 The `install` command will run all the tests by default; we can ignore the tests to speed up the installation process with command `sudo mvn -DskipTests install`.

9. Compile the Mahout examples with the following commands:

```
cd examples
sudo mvn compile
```

Use the following steps to verify Mahout configuration:

1. Download sample data with the following command:

```
wget http://archive.ics.uci.edu/ml/databases/synthetic_control/
synthetic_control.data -P ~/
```

2. Start the Hadoop cluster with commands:

```
start-dfs.sh
start-mapred.sh
```

3. Put the downloaded data into HDFS with the following commands:

```
hadoop fs -mkdir testdata
hadoop fs -put ~/synthetic_control.data testdata
```

4. Run the `kmeans` clustering with the following command:

```
mahout org.apache.mahout.clustering.syntheticcontrol.kmeans.Job
```

See also

- More documentation about Mahout can be obtained from <https://cwiki.apache.org/confluence/display/MAHOUT/Mahout+Wiki>.

Where to buy this book

You can buy Hadoop Operations and Cluster Management Cookbook from the Packt Publishing website: <http://packtpub.com/hadoop-operations-and-cluster-management-cookbook/book>.

Free shipping to the US, UK, Europe and selected Asian countries. For more information, please read our [shipping policy](#).

Alternatively, you can buy the book from Amazon, BN.com, Computer Manuals and most internet book retailers.



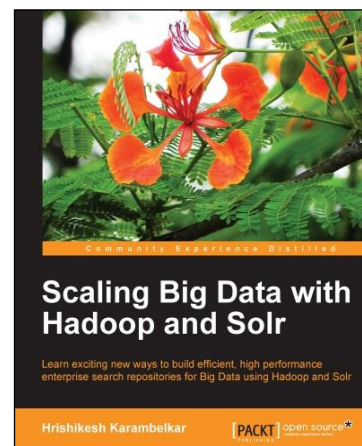
www.PacktPub.com

For More Information:

www.packtpub.com/hadoop-operations-and-cluster-management-cookbook/book

Scaling Big Data with Hadoop and Solr

Hrishikesh Karambelkar



Chapter No. 2 "Understanding Solr"

In this package, you will find:

A Biography of the author of the book

A preview chapter from the book, Chapter NO.2 "Understanding Solr"

A synopsis of the book's content

Information on where to buy this book

About the Author

Hrishikesh Karambelkar is a software architect with a blend of entrepreneurial and professional experience. His core expertise involves working with multiple technologies such as Apache Hadoop and Solr, and architecting new solutions for the next generation of a product line for his organization. He has published various research papers in the domains of graph searches in databases at various international conferences in the past. On a technical note, Hrishikesh has worked on many challenging problems in the industry involving Apache Hadoop and Solr.

While writing the book, I spend my late nights and weekends bringing in the value for the readers. There were few who stood by me during good and bad times, my lovely wife Dhanashree, my younger brother Rupesh, and my parents. I dedicate this book to them. I would like to thank the Apache community users who added a lot of interesting content for this topic, without them, I would not have got an opportunity to add new interesting information to this book.

For More Information:

www.packtpub.com/scaling-big-data-with-hadoop-and-solr/book

Scaling Big Data with Hadoop and Solr

This book will provide users with a step-by-step guide to work with Big Data using Hadoop and Solr. It starts with a basic understanding of Hadoop and Solr, and gradually gets into building efficient, high performance enterprise search repository for Big Data.

You will learn various architectures and data workflows for distributed search system. In the later chapters, this book provides information about optimizing the Big Data search instance ensuring high availability and reliability.

This book later demonstrates two real world use cases about how Hadoop and Solr can be used together for distributed enterprise search.

What This Book Covers

Chapter 1, Processing Big Data Using Hadoop and MapReduce, introduces you with Apache Hadoop and its ecosystem, HDFS, and MapReduce. You will also learn how to write MapReduce programs, configure Hadoop cluster, the configuration files, and the administration of your cluster.

Chapter 2, Understanding Solr, introduces you to Apache Solr. It explains how you can configure the Solr instance, how to create indexes and load your data in the Solr repository, and how you can use Solr effectively for searching. It also discusses interesting features of Apache Solr.

Chapter 3, Making Big Data Work for Hadoop and Solr, brings the two worlds together; it drives you through different approaches for achieving Big Data work with architectures and their benefits and applicability.

Chapter 4, Using Big Data to Build Your Large Indexing, explains the NoSQL and concepts of distributed search. It then gets you into using different algorithms for Big Data search covering shards and indexing. It also talks about SolrCloud configuration and Lily.

Chapter 5, Improving Performance of Search while Scaling with Big Data, covers different levels of optimizations that you can perform on your Big Data search instance as the data keeps growing. It discusses different performance improvement techniques which can be implemented by the users for their deployment.

For More Information:

www.packtpub.com/scaling-big-data-with-hadoop-and-solr/book

Appendix A, Use Cases for Big Data Search, describes some industry use cases and case studies for Big Data using Solr and Hadoop.

Appendix B, Creating Enterprise Search Using Apache Solr, shares a sample Solr schema which can be used by the users for experimenting with Apache Solr.

Appendix C, Sample MapReduce Programs to Build the Solr Indexes, provides a sample MapReduce program to build distributed Solr indexes for different approaches.

For More Information:

www.packtpub.com/scaling-big-data-with-hadoop-and-solr/book

2

Understanding Solr

The exponential growth of data coming from various applications over the past decade has created many challenges. Handling such massive data demanded focus on the development of scalable search engines. It also triggered development of data analytics. Apache Lucene along with Mahout and Solr were developed to address these needs. Out of these, Mahout was moved as a separate Apache top-level project, and Apache Solr was merged into the Lucene project itself.

Apache Solr is an open source enterprise search application which provides user abilities to search structured as well as unstructured data across the organization. It is based on the Apache Lucene libraries for information retrieval. Apache Lucene is an open source information retrieval library used widely by various organizations. Apache Solr is completely developed on Java stack of technologies. Apache Solr is a web application, and Apache Lucene is a library consumed by Apache Solr for performing search. We will try to understand Apache Solr in this chapter, while covering the following topics:

- Installation of an Apache Solr
- Understanding the Apache Solr architecture
- Configuring a Solr instance
- Understanding various components of Solr in detail
- Understanding data loading

For More Information:

www.packtpub.com/scaling-big-data-with-hadoop-and-solr/book

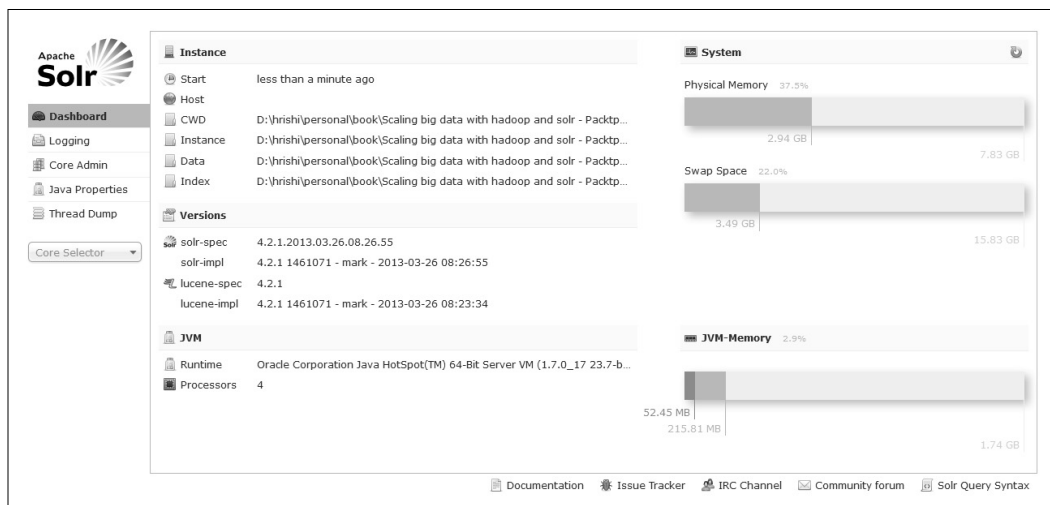
Installing Solr

Apache Solr comes by default with a demo server based on Jetty, which can be downloaded and run. However, you can choose to customize it, and deploy it in your own environment. Before installation, you need to make sure that you have JDK 1.5 or above on your machines. You can download the stable installer from <http://lucene.apache.org/solr/> or from its nightly builds running on the same site. You may also need a utility called **curl** to run your samples. There are commercial versions of Apache Solr available from a company called **LucidWorks** (<http://www.lucidworks.com>). Solr being a web-based application can run on many operating systems such as *nix and Windows.



Some of the older versions of Solr have failed to run properly due to locale differences on host systems. If your system's default locale, or character set is non-english (that is, en/en-US), for safety, you can override your system defaults for Solr by passing `-Duser.language` and `-Duser.country` in your Jetty to ensure smooth running of Solr.

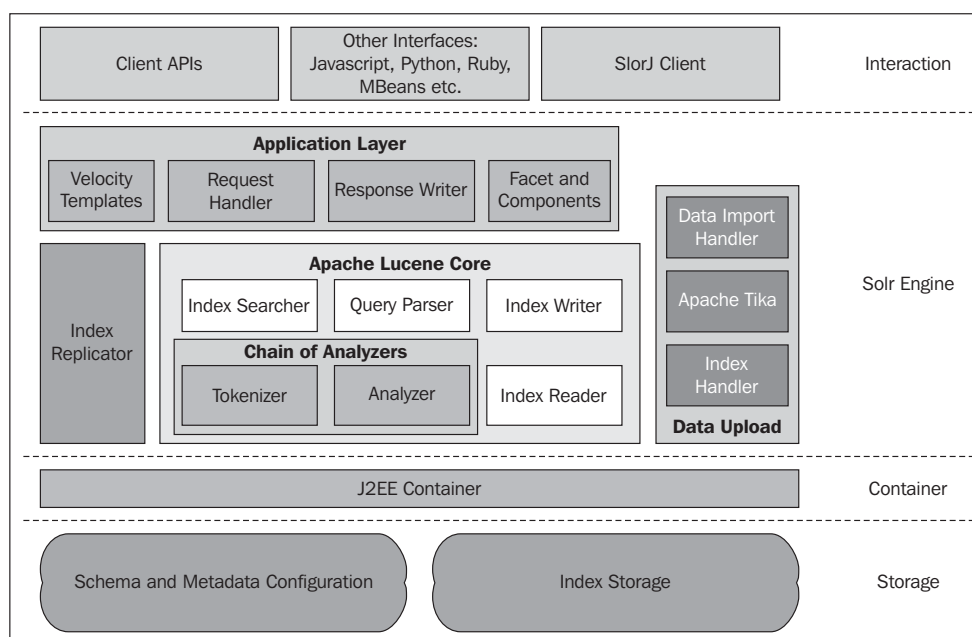
If you are planning to run Solr in your own container, you need to deploy `solr.war` from the distribution to your container. You can simply check whether your instance is running or not by accessing its admin page available at <http://localhost:8983/solr/admin>. The following screenshot shows the Solr window:



If you are building Solr from source, then you need Java SE 6 JDK (Java Development Kit), Apache Ant distribution (1.8.2 or higher), and Apache Ivy (2.2.0 or higher). You can compile the source by simply navigating to Solr directory and running Ant from the directory.

Apache Solr architecture

Apache Solr is composed of multiple modules, some of them being separate projects in themselves. Let's understand the different components of Apache Solr architecture. The following diagram depicts the Apache Solr conceptual architecture:



Apache Solr can run as a single core or multicore. A Solr core is nothing but the running instance of a Solr index along with its configuration. Earlier, Apache Solr had a single core which in turn limited the consumers to run Solr on one application through a single schema and configuration file. Later support for creating multiple cores was added. With this support, now, one can run one Solr instance for multiple schemas and configurations with unified administrations. You can run Solr in multicore with the following command:

```
java -Dsolr.solr.home=multicore -jar start.jar
```

Storage

The storage of Apache Solr is mainly used for storing metadata and the actual index information. It is typically a file store locally, configured in the configuration of Apache Solr. The default Solr installation package comes with a Jetty server, the respective configuration can be found in the `solr.home/conf` folder of Solr install.

There are two major configuration files in Solr described as follows:

File name	Description
<code>solrconfig.xml</code>	This is the main configuration file of your Solr install. Using this you can control each and every thing possible; write from caching, specifying customer handlers, codes, and commit options.
<code>schema.xml</code>	This file is responsible for defining a Solr schema for your application. For example, Solr implementation for log management would have schema with log-related attributes such as log levels, severity, message type, container name, application name, and so on.
<code>solr.xml</code>	Using <code>solr.xml</code> , you can configure Solr cores (single or multiple) for your setup. It also provides additional parameters such as zookeeper timeout, transient cache size, and so on.

Apache Solr (underlying Lucene) indexing is a specially designed data structure, stored in the file system as a set of index files. The index is designed with specific format in such a way to maximize the query performance.

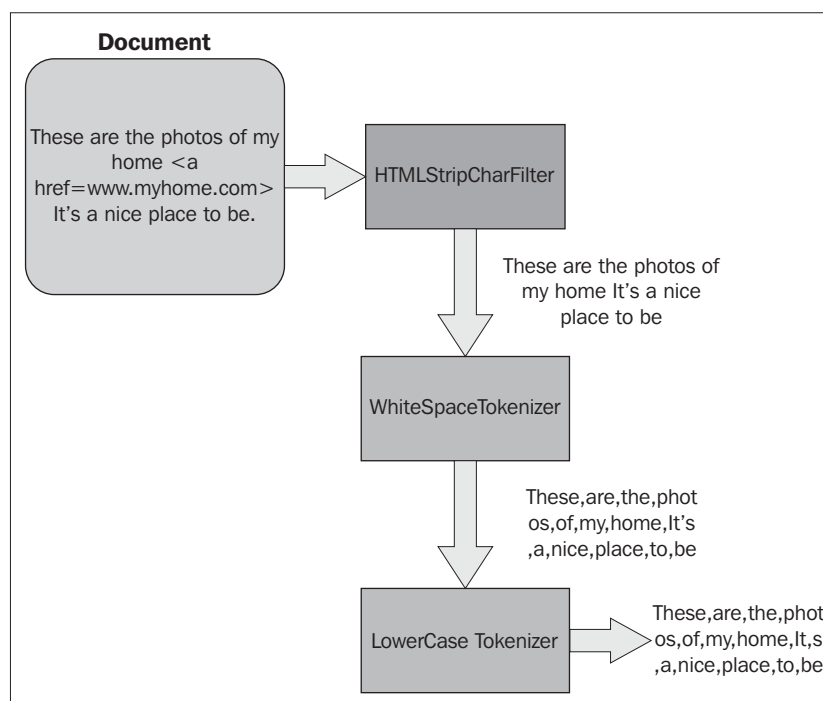
Solr engine

A Solr engine is nothing but the engine responsible for making Solr what it is today. A Solr engine with metadata configuration together forms the Solr core. When Solr runs in a replication mode, the index replicator is responsible for distributing indexes across multiple slaves. The master server maintains index updates, and slaves are responsible for talking with master to get them replicated. Apache Lucene core gets packages as library with Apache Solr application. It provides core functionality for Solr such as index, query processing, searching data, ranking matched results, and returning them back.

The query parser

Apache Lucene comes with variety of query implementations. Query parser is responsible for parsing the queries passed by the end search as a search string. Lucene provides `TermQuery`, `BooleanQuery`, `PhraseQuery`, `PrefixQuery`, `RangeQuery`, `MultiTermQuery`, `FilteredQuery`, `SpanQuery`, and so on as query implementations. `IndexSearcher` is a basic component of Solr searched with a default base searcher class. This class is responsible for returning ordered matched results of searched keywords ranked as per the computed score. `IndexReader` provides access to indexes stored in the file system. It can be used for searching for an index. Similar to `IndexReader`, `IndexWriter` allows you to create and maintain indexes in Apache Lucene.

Tokenizer breaks field data into lexical units or tokens. Filter examines field of tokens from Tokenizer and either it keeps them, transforms them, discards them, or creates new ones. Tokenizer and Filter together form chain or pipeline of analyzers. There can only be one Tokenizer per Analyzer. The output of one chain is fed to another. Analyzing process is used for indexing as well as querying by Solr. They play an important role in speeding up the query as well as index time; they also reduce the amount of data that gets generated out of these operations. You can define your own custom analyzers depending upon your use case. The following diagram shows the example of a filter:



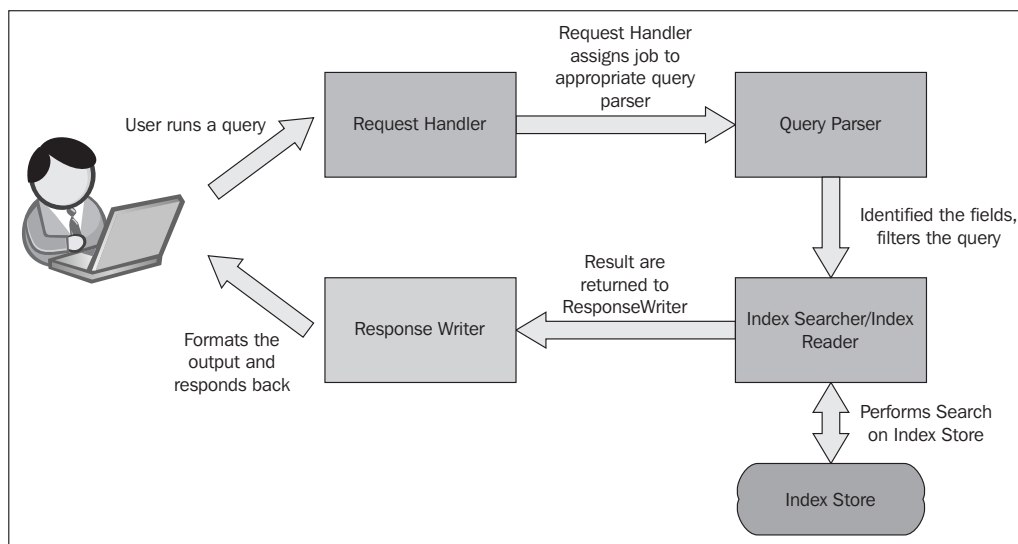
Application layer represents Apache Solr web application. It consists of different UI templates, request/response handlers, and different faceting provided by Solr.

Faceted browsing is one of the main features of Apache Solr; it helps users reach the right set of information they wanted to get. The facets and components deal with providing the faceted search capabilities on top of Lucene. When a user fires a search query on Solr, it actually gets passed on to a request handler. By default, Apache Solr provides `DisMaxRequestHandler`. This handler is designed to work for simple user queries. It can only search one field by default. You can visit [here](http://lucene.apache.org/solr/4.10.0/query-api.html#dismax) to find more details about this handler. Based on the request, request handler calls query parser.

Query parser is responsible for parsing the queries, and converting it into Lucene query objects. There are different types of parsers available (Lucene, DisMax, eDisMax, and so on). Each parser offers different functionalities and it can be used based on the requirements. Once a query is parsed, it hands it over to index searcher or reader. The job of **index reader** is to run the queries on index store, and gather the results to response writer.

Response Writer is responsible for responding back to the client; it formats the query response based on search outcomes from the Lucene engine.

The following diagram displays complete process flow when a search is fired from a client:



Apache Solr ships with an example schema that runs using Apache velocity. Apache velocity is a fast, open source template engine which quickly generates an HTML-based frontend. Users can customize these templates as per their requirements.

Index handler is one type of update handler that handles the task of addition, updation, and deletion of document for indexing. Apache Solr supports updates through index handler through JSON, XML, and text format.

Data Import Handler (DIH) provides a mechanism for integrating different data sources with Apache Solr for indexing. The data sources could be relational databases or web-based sources (for example, RSS, ATOM feeds, and e-mails).



Although DIH is part of Solr development, the default installation does not include it in the Solr application.

Apache Tika, a project in itself extends capabilities of Apache Solr to run on top of different types of files. When assigned a document to Tika, it automatically determines the type of file (that is, Word, Excel, or PDF) and extracts the content. Tika also extracts document metadata such as author, title, creation date, and so on, which if provided in schema go as text field in Apache Solr.

Interaction

Apache Solr, although a web-based application, can be integrated with different technologies. So, if a company has Drupal-based e-commerce site, they can integrate Apache Solr application and provide its rich faceted search to the user.

Client APIs and SolrJ client

Apache Solr client provides different ways of talking with Apache Solr web application. This enables Solr to easily get integrated with any application. Using client APIs, consumers can run search, and perform different operations on indexes. SolrJ or Solr Java client is an interface of Apache Solr with Java. SolrJ client enables any Java application to talk directly with Solr through its extensive library of APIs. Apache SolrJ is part of Apache Solr package.

Other interfaces

Apache Solr can be integrated with other various technologies using its API library and standards-based interfacing. JavaScript-based clients can straightaway talk with Solr using JSON-based messaging. Similarly, other technologies can simply connect to Apache Solr running instance through HTTP, and consume its services either through JSON, XML, and text formats.

Configuring Apache Solr search

Apache Solr allows extensive configuration to meet the needs of the consumer. Configuring the instance revolves around the following:

- Defining a schema
- Configuring Solr parameters

Let's look at all these steps to understand the configuration of Apache Solr.

Defining a Schema for your instance

Apache Solr lets you define the structure of your data to extend support for searching across the traditional keyword search. You can allow Solr to understand the structure of your data (coming from various sources) by defining fields in the schema definition file. These fields once defined, will be made available at the time of data import or data upload. The schema is stored in the `schema.xml` file in the `etc` folder of Apache Solr.

Apache Solr ships with a default `schema.xml` file, which you have to change to fit your needs. In schema configuration, you can define field types, (for example, `String`, `Integer`, `Date`), and map them to respective Java classes. Apache Solr ships with default data types for text, integer, date, and so on.

```
<field name="id" type="string" indexed="true" stored="true"
      required="true"/>
```

This enables users to define the custom type in case if they wish to. Then you can define the fields with name and type pointing to one of the defined types. A field in Solr will have the following major attributes:

Name	Description
default	Sets default value, if not read while importing a document
indexed	True, when it has to be indexed (that is, can be searched, sorted, and facet creation)
stored	When true, a field is stored in the index store, and it will be accessible while displaying results
compressed	When true, the field will be zipped (using gzip). It is applicable for text-based fields
multiValued	If a field contains multiple values in the same import cycle of the document/row
omitNorms	When true, it omits the norms associated with field (such as length normalization and index boosting)
termVectors	When true, it also persists metadata related to document, and returns that when queried.

Each Solr instance should have a unique identifier field (ID) although it's not mandatory condition. In addition to static fields, you can also use Solr dynamic fields for getting the flexibility in case if you do not know the schema upfront. Use the `<dynamicField>` declaration for creating a field rule to allow Solr understands which data type to be used. In the following sample code, any field imported, and identified as `*_no` (for example, `id_no`, `book_no`) will in turn be read as `integer` by Solr.

```
<dynamicField name="*_no" type="integer" indexed="true"
  stored="true"/>
```

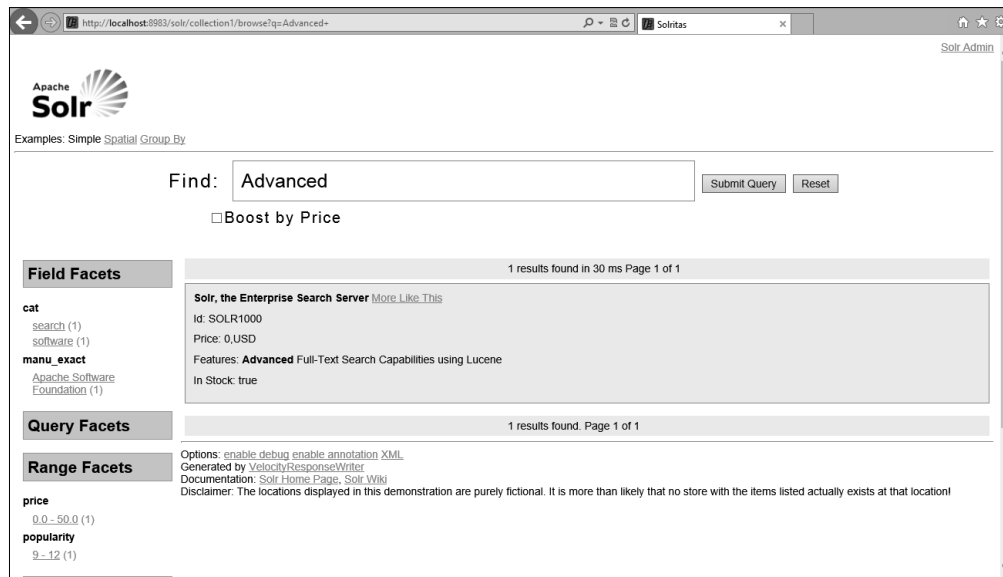
You can also index same data into multiple fields by using the `<copyField>` directive. This is typically needed when you want to have multi-indexing for same data type, for example, if you have data for refrigerator with company followed by model number (WHIRLPOOL-1000LTR, SAMSUNG-980LTR), you can have these indexed separately by applying your own tokenizers to different field. You might generate indexes for two different fields, company name and model number. You can define Tokenizers specific to your field types. Similarly, a Lucene class responsible for scoring the matched results. Solr allows you to override default similarity behavior through the `<similarity>` declaration. Similarity can be configured at the global level; however with Solr 4.0, it extends similarity to be configured at the field level.

Configuring a Solr instance

Once a schema is configured, next step would be to configure the instance itself. To configure the instance, you need to touch upon many files, some of them are configuration files, and some of them are metadata files. The entire configuration is part of `/conf` directory where Solr instance is setup. You can simply run examples by going to the `examples/example-docs` directory and running the following code:

```
java -jar post.jar solr.xml monitor.xml
```

Now, try accessing your instance by typing `http://localhost:8983/solr/collection1/browse`, and you will be able to see the following screenshot when you search on **Advanced**:



Configuration files

There are two major configurations that go in the Solr configuration: namely, `solrconfig.xml` and `solr.xml`. Among these, `solr.xml` is responsible for maintaining configuration for logging, cloud setup, and Solr core primarily, whereas `solrconfig.xml` focuses more on the Solr application front. Let's look at the `solrconfig.xml` file, and understand all the important declarations you'd be using frequently.

Directive	Description
<code>luceneMatchVersion</code>	It tells which version of Lucene/Solr the <code>solrconfig.xml</code> configuration file is set to. When upgrading your Solr instances, you need to modify this attribute.
<code>lib</code>	In case if you create any plugins for Solr, you need to put a library reference here, so that it gets picked up. The libraries are loaded in the same sequence that of the configuration order. The paths are relative; you can also specify regular expressions. For example, <pre><lib dir="../../../contrib/velocity/lib" regex=".*\.jar" /></pre>

Directive	Description
<code>dataDir</code>	By default Solr uses <code>./data</code> directory for storing indexes, however this can be overridden by changing the directory for data using this directive.
<code>indexConfig</code>	This directive is of <code>xsd complexType</code> , and it allows you to change the settings of some of the internal indexing configuration of Solr.
<code>filter</code>	You can specify different filters to be run at the time of index creation.
<code>writeLockTimeout</code>	This directive denotes maximum time to wait for the write Lock for <code>IndexWriter</code> .
<code>maxIndexingThreads</code>	It denotes maximum number of index threads that can run in the <code>IndexWriter</code> class; if more threads arrive, they have to wait. The default value is 8.
<code>ramBufferSizeMB</code>	It specifies the maximum RAM you need in the buffer while index creation, before the files are flushed to filesystem.
<code>maxBufferedDocs</code>	It specifies the limited number of documents buffered.
<code>lockType</code>	When indexes are generated and stored in the file, this mechanism decides which file locking mechanism to be used to manage concurrent read/writes. There are three types: single (one process at a time), native (native operating system driven), and simple (based on locking using plain files).
<code>unlockOnStartup</code>	When true, it will release all the write locks held in the past.
<code>Jmx</code>	Solr can expose statistics of runtime through MBeans. It can be enabled or disabled through this directive.
<code>updateHandler</code>	This directive is responsible for managing the updates to Solr. The entire configuration for <code>updateHandler</code> goes as a part of this directive.
<code>updateLog</code>	You can specify the directory and other configuration for transaction logs during index updates.
<code>autoCommit</code>	It enables automatic commit, when updates are done. This could be based on the documents or time.
<code>Listener</code>	Using this directive, you can subscribe to update events when <code>IndexWriter</code> is updating the index. The listeners can be run either at the time of <code>postCommit</code> or <code>postOptimize</code> .
<code>Query</code>	This directive is mainly responsible for controlling different parameters at the query time.

Directive	Description
<code>requestDispatcher</code>	By setting parameters in this directive, you can control how a request will be processed by <code>SolrDispatchFilter</code> .
<code>requestHandler</code>	It is described separately in the next section.
<code>searchComponent</code>	It is described separately in the next section.
<code>updateRequestProcessor chain</code>	The <code>updateRequestProcessor</code> chain defines how update requests are processed; you can define your own <code>updateRequestProcessor</code> to perform things such as cleaning up data, optimizing text fields, and so on.
<code>queryResponseWriter</code>	Each request for query is formatted and written back to user through <code>queryResponseWriter</code> . You can extend your Solr instance to have responses for XML, JSON, PHP, Ruby, Python, and CSVs by enabling respective predefined writers. If you have a custom requirement for a certain type of response, it can easily be extended.
<code>queryParser</code>	The <code>queryParser</code> directive tells Apache Solr which query parser to be used for parsing the query and creating the Lucene query objects. Apache Solr contains predefined query parsers such as <code>Lucene</code> (default), <code>DisMax</code> (based on weights of fields), and <code>eDisMax</code> (similar to <code>DisMax</code> , with some additional features).

Request handlers and search components

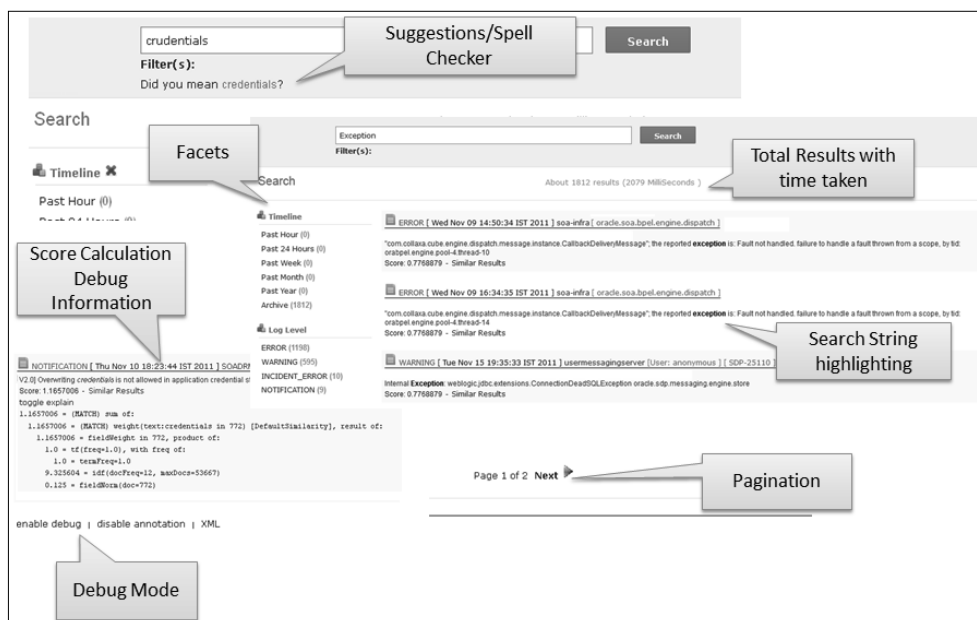
Apache Solr gets requests for searching on data or index generation. In such cases, `RequestHandler` is the directive through which you can define different ways of tackling these requests. One request handler is assigned with one relative URL where it would serve the request. A request handler may or may not provide search facility. In case if it provides, it is also called `searchHandler`. `RealTimeGetHandler` provides latest stored fields of any document. `UpdateRequestHandler` is responsible for updating the process of the index. Similarly, `CSVRequestHandler` and `JsonUpdateRequestHandler` takes the responsibility of updating the indexes with CSV and JSON formats, respectively. `ExtractingRequestHandler` uses Apache Tika to extract the text out of different file formats. By default, there are some important URLs configured with Apache Solr which are listed as follows:

URL	Purpose
/select	SearchHandler in text
/query	SearchHandler for JSON-based requests
/get	RealTimeGetHandler in JSON format
/browse	SearchHandler faceted web-based search, primary interface
/update/extract	ExtractingRequestHandler
/update/csv	CSVRequestHandler
/update/json	JsonUpdateRequestHandler
/analysis/*	For analyzing the field, documents. It makes use of FieldAnalysisRequestHandler
/admin	AdminHandler for providing administration of Solr. AdminHandler has multiple subhandlers defined; /admin/ping is used for health checkup
/debug/dump	DumpRequestHandler echoes the request content back to the client
/replication	Supports replicating indexes across different Solr servers, used by masters and slaves for data sharing. It makes use of ReplicationHandler

A `searchComponent` is a one of the main feature of Apache Solr. It brings the capability of enhancing new features to Apache Solr. You can use `searchComponent` in your `searchHandler`. It has to be defined separately from `requestHandler`. These components can be defined, and then they can be used in any of the `requestHandler` directives. Some components also allow access through either `searchComponent`, or directly as a separate request handler. You can alternatively specify your query parser in the context of your `requestHandler`. Different parsers can be used for this. The default parser is the Lucene-based standard parser.

Facet

Facets are one of the primary features of Apache Solr. Your search results can be organized in different formats through facets. This is an effective way of helping users to drill down to right set of information. The following screenshot shows one of a customized instance of Apache Solr with facets on the left-hand side:



Using facets, you can filter down your query. Facets can be created on your schema-based fields. So, considering the log-based search, you can create facets based on the log severity. There are different types of facets:

Facet	Description
Field-value	You can have your schema fields as facet component here. It shows the count of top fields.
Range	Range faceting is mostly used on date/numeric fields, and it supports range queries. You can specify start and end dates, gap in the range, and so on.
Date	This is a deprecated faceting, and it is now being handled in the range faceting itself.
Pivot	Pivot gives you the ability to perform simple math on your data. With this facet, you can summarize your results, and then you can get them sorted, and take average. This gives you hierarchical results (also sometimes called hierarchical faceting).

MoreLikeThis

The Solr-based search results are enhanced with the `MoreLikeThis` component, because they provide a better user-browsing experience by allowing the user to choose similar results. This component can be accessed either through `requestHandler`, or through `searchcomponent`.

Highlight

The matched search string can be highlighted in the search results when a user fires a query to Apache Solr through the `highlight` search component.

SpellCheck

Searching in Solr can be extended further with the support for spell checks using the `spellcheck` component. You can get support for multiple dictionaries together per field. This is very useful in case of multilingual data. It also has a `Suggestor` that responds to user with **Did you mean** type of suggestions. Additionally, `Suggestor` with `autocomplete` feature starts providing users options right at the time when user is typing search query enhancing the overall experience.

Metadata management

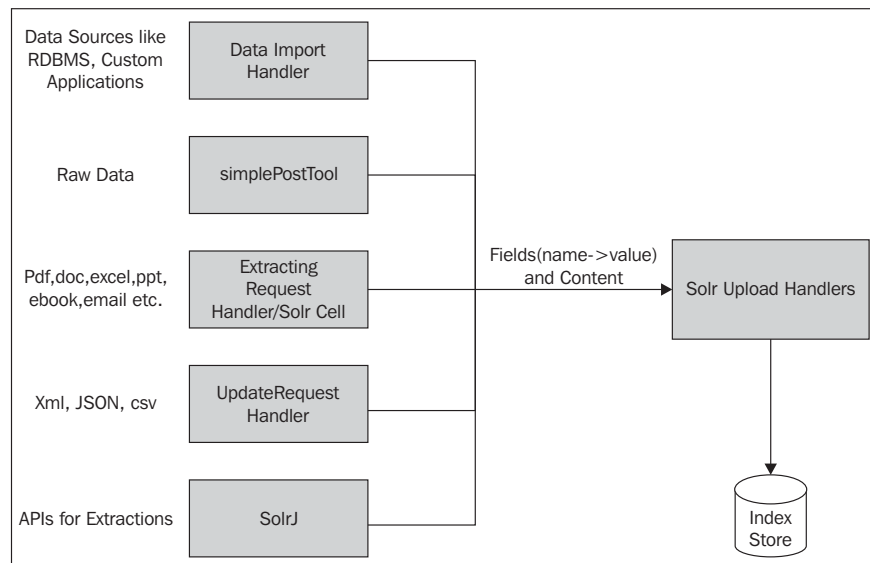
We have already seen the `solr.xml`, `solrconfig.xml`, and `schema.xml` configuration files. Besides these, there are other files where a metadata can be specified. These files again appear in the `conf` directory of Apache Solr.

File name	Description
<code>protwords.txt</code>	In this file, you can specify protected words that you do not wish to get stemmed. For example, a stemmer might stem the word <code>catfish</code> to <code>cat</code> or <code>fish</code> .
<code>currency.txt</code>	Current stores mapping between exchange rates across different countries; this file is helpful when you have your application accessed by people from different country.
<code>elevate.txt</code>	With this file, you can influence the search results and make your own results among the top ranked results. This overrides Lucene's standard ranking scheme taking into account the elevations from this file.
<code>spellings.txt</code>	In this file, you can provide spelling suggestions to the end user.

File name	Description
synonyms.txt	Using this file, you can specify your own synonyms. For example, Cost => money, Money => dollars.
stopwords.txt	Stopwords are those words which will not be indexed and used by Solr in the applications; this is particularly helpful when you wish to get rid of certain words, for example, in the string, Jamie and Joseph , the word and can be marked as a stopwords.

Loading your data for search

Once a Solr instance is configured, next step is to index your data, and then simply use the instance for querying and analyzing. Apache Solr/Lucene is designed in such a way that it allows you to plugin any type of data from any data source in the world. If you have structured data, it makes sense to extract the structured information, create exhaustive Solr schema, and feed in the data to Solr, effectively adding different data dimensions to your search. Data Import Handler (DIH) is used mainly for indexing structured data. It is mainly associated with data sources such as relational databases, XML databases, RSS feeds, and ATOM feeds. DIH uses multiple entity processors to extract the data from various data sources, transform them, and finally generate indexes out of it. For example, in a relational database, a table or a view can be viewed as an entity. DIH allows you to write your own custom entity processors. There are different ways to load the data in Apache Solr as shown in the following diagram:



ExtractingRequestHandler/Solr Cell

Solr Cell is one of the most powerful handlers for uploading any type of data. If you wish you can run Solr on a set of files/unstructured data containing different formats such as MS Office, PDF, e-book, e-mail, text, and so on. In Apache Tika, text extraction is based purely on how exhaustive any file is. Therefore, if you have a PDF of scanned images containing text, Apache Tika won't be able to extract any of the text out of it. In such cases, you need to use **Optical Character Recognition (OCR)**-based software to bring in such functionality for Solr. You can simply try this on your downloaded curl utility, and then running it on your document:

```
curl 'http://localhost:8983/solr/update/extract?
literal.id=doc1&commit=true' -F "myfile=@<your document name
with extension>"
```

Index handlers such as SimplePostTool, UpdateRequestHandler, and SolrJ provide addition, updation, and deletion of documents to index them for XML, JSON, and CSV format. UpdateRequestHandler provide web-based URL for uploading the document. This can be done through curl utility.



Curl/wget utilities can be used for uploading data to Solr in your environment. They are command line based; you can also use the FireCURL plugin to upload data through your Firefox browser.

Simple post tool is a command-line tool for uploading the raw data to Apache Solr. You can simply run it on any file or type in your input through STDIN to load it in Apache Solr.

SolrJ

SolrJ or (SolrJava) is a tool that can be used by your Java-based application to connect to Apache Solr for indexing. It provides a user-friendly interface hiding connection details from consumer application. Using SolrJ, you can index your documents and perform your queries. There are two major ways to do so; one is using the EmbeddedSolrServer interface. If you are using Solr in an embedded application, this is the recommended interface suited for you. It does not use HTTP-based connection. The other way is to use the HTTPSolrServer interface, which talks with Solr server through HTTP protocol. This is suited if you have a remote client-server based application. You can use ConcurrentUpdateSolrServer for bulk uploads whereas CloudSolrServer for communicating with Solr running in a cloud setup.

In analyzing and querying your data, we have already seen how Apache Solr effectively uses different request handlers to provide consumers with extensive ways of getting search results. Each request handler uses its own query parser, which extracts the parameters and their values from the query string, and forms the Lucene query objects. Standard query parser allows greater precision over search data; `DisMaxQueryParser` and `ExtendedDisMaxQueryParser` provide a Google-like searching syntax while searching. Depending upon which request handler is called, the query syntax is changed. Let's look at some of the important terms:

Term	Meaning
<code>q?<string></code>	Can support wildcard (*:*), for example, <code>title:Scaling*</code>
<code>fl=id,book-name</code>	Field list that a search response will return
<code>sort=author asc</code>	Results/facets to be sorted on authors in ascending order
<code>price[* TO 100]&rows=10&start=5</code>	Limits the result to 10 rows at a time, starting at fifth matched result
<code>hl=true&hl.fl=name,features</code>	Enables highlighting on field list name and features
<code>&q=*:*&facet=true&facet.field=year</code>	Enables faceted search on field year
<code>Publish-date:[NOW-1YEAR/DAY TO NOW/DAY]</code>	Published date between last year (same day) until today
<code>description:"Java sql"~10</code>	Called proximity search. Searches for the descriptions containing Java and sql in a single document with a proximity of 10 words maximum
<code>"open jdk" NOT "Sun JDK"</code>	Searches for the open jdk term in the document
<code>&q=id:938099893&mlt=true</code>	Searches for a specific ID, and also searches for similar results (more like this)

Summary

We have gone through various details of Apache Solr in this chapter. We reviewed the architecture, the configuration, the data loading, and its features. In the next chapter, we will look into how you can bring the two worlds of Apache Solr and Apache Hadoop together to work with Big Data.

Where to buy this book

You can buy Scaling Big Data with Hadoop and Solr from the Packt Publishing website:
<http://www.packtpub.com/scaling-big-data-with-hadoop-and-solr/book>

Free shipping to the US, UK, Europe and selected Asian countries. For more information, please read our [shipping policy](#).

Alternatively, you can buy the book from Amazon, BN.com, Computer Manuals and most internet book retailers.



www.PacktPub.com

For More Information:

www.packtpub.com/scaling-big-data-with-hadoop-and-solr/book